

\$ how to git

The GitHub CLI Best-Practices Book

Edition 2026.1

2026

Thomas Björk

Contents

What the GitHub CLI is	1
gh vs git: two different tools	1
Installing gh	1
The command shape	2
Auth is the prerequisite for everything	2
How gh knows which repository you mean	2
Getting help	2
See also	3
auth	4
Mental model	4
Synopsis	4
Everyday usage	4
Key options	5
Best practices	6
Pitfalls & gotchas	6
Worked examples	7
Recovery	8
See also	8
config	9
Mental model	9
Synopsis	9
Everyday usage	9
Key options	10
Best practices	11
Pitfalls & gotchas	11
Worked examples	12
Recovery	13
See also	13
alias	14
Mental model	14
Synopsis	14
Everyday usage	14
Key options	15
Best practices	16
Pitfalls & gotchas	16
Worked examples	17
Recovery	18
See also	18
repo	19
Mental model	19
Synopsis	19
Everyday usage	19
Key options	21
Best practices	24
Pitfalls & gotchas	25
Worked examples	25

Recovery	27
See also	27
browse	28
Mental model	28
Synopsis	28
Everyday usage	28
Key options	29
Best practices	30
Pitfalls & gotchas	30
Worked examples	31
Recovery	32
See also	32
search	33
Mental model	33
Synopsis	33
Everyday usage	33
Key options	35
Best practices	38
Pitfalls & gotchas	38
Worked examples	39
Recovery	41
See also	41
pr	42
Mental model	42
Synopsis	42
Everyday usage	42
Key options	45
Best practices	48
Pitfalls & gotchas	49
Worked examples	49
Recovery	51
See also	52
issue	53
Mental model	53
Synopsis	53
Everyday usage	53
Key options	57
Best practices	59
Pitfalls & gotchas	60
Worked examples	60
Recovery	62
See also	62
label	63
Mental model	63
Synopsis	63
Everyday usage	63
Key options	64

Best practices	65
Pitfalls & gotchas	66
Worked examples	66
Recovery	67
See also	68
project	69
Mental model	69
Synopsis	69
Everyday usage	70
Key options	73
Best practices	74
Pitfalls & gotchas	75
Worked examples	76
Recovery	77
See also	78
run	79
Mental model	79
Synopsis	79
Everyday usage	79
Key options	82
Best practices	84
Pitfalls & gotchas	85
Worked examples	85
Recovery	86
See also	87
workflow	88
Mental model	88
Synopsis	88
Everyday usage	88
Key options	89
Best practices	90
Pitfalls & gotchas	91
Worked examples	91
Recovery	93
See also	93
cache	94
Mental model	94
Synopsis	94
Everyday usage	94
Key options	95
Best practices	96
Pitfalls & gotchas	96
Worked examples	97
Recovery	98
See also	98
release	100
Mental model	100

Synopsis	100
Everyday usage	100
Key options	103
Best practices	105
Pitfalls & gotchas	106
Worked examples	107
Recovery	108
See also	109
gist	110
Mental model	110
Synopsis	110
Everyday usage	110
Key options	112
Best practices	113
Pitfalls & gotchas	114
Worked examples	115
Recovery	116
See also	116
attestation	117
Mental model	117
Synopsis	117
Everyday usage	117
Key options	118
Best practices	120
Pitfalls & gotchas	120
Worked examples	121
Recovery	123
See also	123
secret	124
Mental model	124
Synopsis	124
Everyday usage	124
Key options	126
Best practices	127
Pitfalls & gotchas	127
Worked examples	128
Recovery	129
See also	130
variable	131
Mental model	131
Synopsis	131
Everyday usage	131
Key options	132
Best practices	133
Pitfalls & gotchas	134
Worked examples	134
Recovery	136

See also	136
ruleset	137
Mental model	137
Synopsis	137
Everyday usage	137
Key options	138
Best practices	139
Pitfalls & gotchas	139
Worked examples	140
Recovery	141
See also	141
ssh-key	143
Mental model	143
Synopsis	143
Everyday usage	143
Key options	144
Best practices	144
Pitfalls & gotchas	145
Worked examples	145
Recovery	147
See also	147
gpg-key	148
Mental model	148
Synopsis	148
Everyday usage	148
Key options	149
Best practices	149
Pitfalls & gotchas	150
Worked examples	150
Recovery	151
See also	152
org	153
Mental model	153
Synopsis	153
Everyday usage	153
Key options	154
Best practices	154
Pitfalls & gotchas	154
Worked examples	155
Recovery	156
See also	156
api	157
Mental model	157
Synopsis	157
Everyday usage	157
Key options	158
Best practices	159

Pitfalls & gotchas	160
Worked examples	161
Recovery	163
See also	163
extension	164
Mental model	164
Synopsis	164
Everyday usage	164
Key options	166
Best practices	167
Pitfalls & gotchas	168
Worked examples	169
Recovery	170
See also	171
status	172
Mental model	172
Synopsis	172
Everyday usage	172
Key options	173
Best practices	173
Pitfalls & gotchas	173
Worked examples	174
Recovery	175
See also	175
completion	176
Mental model	176
Synopsis	176
Everyday usage	176
Key options	177
Best practices	177
Pitfalls & gotchas	177
Worked examples	178
Recovery	179
See also	180
Getting out of jams: gh	181
HTTP 403 / “Permission denied” on a repo command	181
Not logged in / “authentication required” / expired token	181
Missing token scopes (“requires the X scope”)	182
SSO authorization required for an org’s resources	183
HTTPS git push fails because gh credential helper uses the wrong account	183
Rate limiting (HTTP 429 / X-RateLimit headers)	184
gh pr create — “no commits between base and head” / wrong base branch	184
Merge blocked by required checks / branch protection	185
gh api — distinguishing 404 vs 403 vs 422	186
Targeting a GitHub Enterprise Server host	187

What the GitHub CLI is

`gh` is GitHub's official command-line tool. Where `git` manages a repository on your machine and speaks the git protocol to remotes, `gh` speaks GitHub's REST and GraphQL APIs. It lets you open pull requests, file issues, trigger workflows, manage releases, and inspect repository settings without leaving the terminal.

gh vs git: two different tools

The distinction matters because people often confuse them.

	git	gh
What it talks to	Any git server (GitHub, GitLab, Bitbucket, self-hosted, ...)	GitHub's API only
What it manages	The content of repositories (commits, branches, history)	GitHub concepts (PRs, issues, Actions, gists, ...)
Auth required?	Only for private repos or pushes	Always — every call hits the API
Works offline?	Yes, for local operations	No

A typical workflow uses both: `git` to manage the work, `gh` to collaborate around it on GitHub.

```
git switch -c feature/new-search # git: create a local branch
git commit -m "Implement search" # git: record a change
git push                        # git: send it to GitHub
gh pr create --fill             # gh: open a pull request on GitHub
```

Installing gh

macOS (Homebrew)

```
brew install gh
```

Linux (Debian / Ubuntu)

```
(type -p wget >/dev/null || (sudo apt update && sudo apt install wget -y)) \
&& sudo mkdir -p -m 755 /etc/apt/keyrings \
&& out=$(mktemp) && wget -nv -O$out https://cli.github.com/packages/githubcli-
archive-keyring.gpg \
&& cat $out | sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null \
&& sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg \
&& echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/
githubcli-archive-keyring.gpg] https://cli.github.com/packages stable main" | sudo
tee /etc/apt/sources.list.d/github-cli.list > /dev/null \
&& sudo apt update \
&& sudo apt install gh -y
```

Windows (winget)

```
winget install --id GitHub.cli
```

Verify the installation:

```
gh --version
```

```
gh version 2.68.0 (2026-01-15)
```

The command shape

Every `gh` invocation follows the same pattern:

```
gh <command> <subcommand> [flags] [arguments]
```

Examples:

```
gh pr list           # list pull requests in the current repo
gh issue create --title "Bug" # open a new issue
gh repo clone owner/repo    # clone a repository
gh run watch 12345678      # stream a workflow run's output
```

Run `gh help` for a full list of top-level commands, or `gh <command> --help` for a specific command's subcommands and flags.

Auth is the prerequisite for everything

Every `gh` command contacts GitHub's API, which requires authentication. If you skip `gh auth login`, every command will fail with a `not logged in` error.

Authenticate once, then forget about it — `gh` stores credentials securely in the system keychain.

```
gh auth login
```

See *auth* for the full login workflow, managing multiple accounts, and using tokens in CI.

How gh knows which repository you mean

When you run a `gh` command inside a git working directory, `gh` reads the `origin` remote URL and maps it to a GitHub repository. This means `gh pr list` with no arguments works when you are inside a cloned repository — it already knows which one you mean.

Override with `--repo`:

```
gh pr list --repo owner/other-repo
```

Or set `GH_REPO` in the environment to target a specific repository for the duration of a script.

Getting help

```
gh help           # top-level command list
gh <command> --help # flags and subcommands for a command
gh help environment # environment variables gh respects
gh help formatting  # --json, --jq, --template output
```

The `gh help formatting` page is particularly useful when you want to pipe `gh` output into scripts.

See also

- *auth* — authenticating, switching accounts, and using tokens.
- *pr* — creating and reviewing pull requests.
- *repo* — cloning, forking, and managing repositories.

auth

Authenticate `gh` with GitHub accounts, manage stored credentials, and switch between multiple accounts on the same machine.

Mental model

`gh` needs an OAuth token (or a personal access token) to call GitHub's API. The `auth` command group manages those tokens: it stores them in the system credential store (macOS Keychain, Windows Credential Manager, or a platform-native secret store), retrieves them transparently on every API call, and lets you add, remove, refresh, and switch between multiple accounts.

A separate credential is stored for each (*hostname, username*) pair. You can authenticate against multiple accounts on the same host (e.g., a personal account and a work account on `github.com`) or against GitHub Enterprise Server instances alongside `github.com`.

`gh` marks one account per host as **active**. All commands use the active account for that host unless you override with `GH_TOKEN` or `--repo` with a different owner.

Synopsis

```
gh auth login      [--web] [--with-token] [-h hostname] [-p git-protocol] [-s scopes]
gh auth logout     [-h hostname] [-u user]
gh auth status     [-h hostname] [-a] [--show-token]
gh auth switch     [-h hostname] [-u user]
gh auth refresh    [-h hostname] [-s scopes] [--remove-scopes scopes] [--reset-scopes]
gh auth setup-git  [-h hostname]
gh auth token      [-h hostname] [-u user]
```

Everyday usage

Log in interactively (opens a browser to authorise):

```
gh auth login
```

Log in to a specific GitHub Enterprise host:

```
gh auth login --hostname github.mycompany.com
```

Check which accounts are authenticated and which is active:

```
gh auth status
```

Switch the active account on `github.com`:

```
gh auth switch
```

Log out of a specific account:

```
gh auth logout --hostname github.com --user myusername
```

Key options

login

Flag	What it does	When to use it
<code>-w / --web</code>	Open a browser to complete OAuth	The default interactive flow
<code>--with-token</code>	Read a personal access token from stdin	Headless / CI environments
<code>-p / --git-protocol</code>	Set <code>https</code> or <code>ssh</code> for git operations	Choose once at login time
<code>-h / --hostname</code>	Target a specific host	GitHub Enterprise Server
<code>-s / --scopes</code>	Request additional OAuth scopes beyond the minimum	When you need extra API access
<code>--skip-ssh-key</code>	Do not offer to generate/upload an SSH key	When you use HTTPS exclusively
<code>--insecure-storage</code>	Store the token in plain text	Systems without a keychain

status

Flag	What it does	When to use it
<code>-a / --active</code>	Show only the active account	Quick sanity check
<code>-h / --hostname</code>	Check only one host	Multi-host setups
<code>-t / --show-token</code>	Print the stored token	Debugging, piping to other tools

switch

Flag	What it does	When to use it
<code>-u / --user</code>	Specify the account to switch to	More than two accounts on one host
<code>-h / --hostname</code>	Target a specific host	Multi-host setups

refresh

Flag	What it does	When to use it
<code>-s / --scopes</code>	Add more OAuth scopes	A command needs a scope you did not grant at login
<code>-r / --remove-scopes</code>	Remove scopes from the token	Reduce permissions after you no longer need them
<code>--reset-scopes</code>	Reset to the minimum required set	Clean up accumulated scopes

token

Flag	What it does	When to use it
<code>-h / --hostname</code>	Target a specific host	Multi-host setups
<code>-u / --user</code>	Target a specific account	When multiple accounts are stored

Best practices

Use `--web` for interactive logins. The browser OAuth flow is the most secure option because you never handle a token string yourself. The device-code flow is the fallback for environments without a browser.

Use `--with-token` in CI. Automation should not open browsers. Pass a personal access token or a short-lived `GITHUB_TOKEN` from Actions via stdin:

```
echo "$GH_TOKEN" | gh auth login --with-token
```

Or skip `auth login` entirely and set the `GH_TOKEN` environment variable. `gh` will use it transparently without storing anything:

```
export GH_TOKEN="ghp_..."
gh pr list # uses GH_TOKEN
```

Set `--git-protocol ssh` at login if you use SSH keys. This configures the git credential helper (`gh auth setup-git`) to use SSH URLs when cloning and pushing, which is consistent with a key-based workflow.

Grant only the scopes you need. The minimum required scopes are `repo`, `read:org`, and `gist`. Add extras with `gh auth refresh --scopes <scope>` only when a specific command demands them. Fewer scopes means a stolen token does less damage.

Run `gh auth setup-git` once per machine. This command registers `gh` as git's credential helper so that HTTPS clones and pushes authenticate automatically — no password prompts:

```
gh auth setup-git
```

Pitfalls & gotchas

The active account is per-host and persists across sessions. Switching is written to `~/.config/gh/hosts.yml`, so it sticks — a brand-new shell does *not* reset you to some default account. If you switch to account A to run a command and forget to switch back, every later command (in this and future terminal sessions) targeting account B's repositories will fail or act as account A. Get in the habit of confirming with `gh auth status --active`.

`gh auth refresh` works only on the active account. To refresh credentials for an inactive account, switch to it first, refresh, then switch back:

```
gh auth switch --user work-account
gh auth refresh --scopes admin:org
gh auth switch --user personal-account
```

`--with-token` expects the token on stdin, not as an argument. The following is wrong:

```
gh auth login --with-token ghp_... # WRONG — token is not a positional arg
```

Pass it via a pipe or heredoc:

```
echo "ghp_..." | gh auth login --with-token
```

Fine-grained personal access tokens and `--with-token` . Fine-grained tokens are scoped to specific repositories. If you use one with `--with-token` , `gh` commands targeting other repositories will fail with permission errors. For fine-grained token usage, set `GH_TOKEN` in the environment rather than storing the token with `auth login` .

Worked examples

Logging in for the first time

```
gh auth login
```

```
? What account do you want to log into? GitHub.com
? What is your preferred protocol for Git operations? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser
! First copy your one-time code: ABCD-1234
Press Enter to open github.com in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as ada-lovelace
```

Managing two GitHub accounts on one machine

Many developers maintain a personal account and a work account on github.com. Log in to each in turn:

```
gh auth login --hostname github.com # log in as personal-user; complete browser
flow
gh auth login --hostname github.com # log in as work-user; complete browser flow
```

Check what is stored:

```
gh auth status
```

```
github.com
✓ Logged in to github.com account personal-user (keyring)
- Active account: true
✓ Logged in to github.com account work-user (keyring)
- Active account: false
```

Switch the active account before working on a work repository:

```
gh auth switch --user work-user
gh pr list --repo my-company/backend # now runs as work-user
```

Or switch back to personal:

```
gh auth switch --user personal-user
```

If the host has exactly two accounts, `gh auth switch` with no flags alternates between them.

Using gh in a GitHub Actions workflow

In Actions, the workflow token is available as `GITHUB_TOKEN`. Pass it through `GH_TOKEN` rather than using `auth login`:

```
- name: List open PRs
  env:
    GH_TOKEN: ${github.token}
  run: gh pr list --state open
```

No credential storage, no cleanup required.

Adding a scope after login

You run `gh release create` and get an error about missing `write:packages` scope. Add it without logging out:

```
gh auth refresh --scopes write:packages
```

This opens a browser to re-authorise with the new scope.

Recovery

If `gh auth status` shows `authentication failed`, the stored token has expired or been revoked. Log in again:

```
gh auth logout
gh auth login
```

To recover from a corrupted credential store, pass `--insecure-storage` to store the token in plain text as a fallback while you diagnose the keychain issue.

See also

- *What the GitHub CLI is* — overview of `gh` and how it differs from `git`.
- *config* — `gh` configuration beyond authentication.
- *repo* — `gh repo clone` uses the active auth credential.

config

Read and write `gh`'s persistent configuration settings — controlling everything from your preferred editor and git protocol to pager, browser, and accessibility options.

Mental model

`gh` stores its configuration in `~/.config/gh/config.yml` (global) and `~/.config/gh/hosts.yml` (per-host overrides). Every key-value pair in those files drives a specific behaviour: which editor opens when `gh pr create` needs a body, which protocol is used for `git clone`, whether the interactive prompt appears at all.

The `config` command group is the safe, structured way to read and write those files. You never have to hand-edit YAML; `gh config get` and `gh config set` treat the files as a typed key-value store. Changes are immediate — the next `gh` invocation picks them up.

Most keys are global, but a handful can be scoped to a specific host with `--host`. A per-host value takes precedence over the global value for that host; for all other hosts the global value still applies. This lets you use SSH on `github.com` while staying on HTTPS for a GitHub Enterprise Server instance.

Synopsis

```
gh config list                [--host <hostname>]
gh config get <key>          [--host <hostname>]
gh config set <key> <value>  [--host <hostname>]
gh config clear-cache
```

Everyday usage

Print every current setting and its value:

```
gh config list
```

Read a single key:

```
gh config get git_protocol
```

Switch to SSH for all git operations:

```
gh config set git_protocol ssh
```

Set your preferred editor:

```
gh config set editor "code --wait"
```

Override git protocol for one host only:

```
gh config set git_protocol ssh --host github.com
```

Check what protocol is configured for a specific host:

```
gh config get git_protocol --host github.com
```

Clear the CLI's local API response cache (useful when stale data is returned):

```
gh config clear-cache
```

Key options

list

Flag	What it does	When to use it
<code>-h / --host</code>	Show only settings for the named host	Multi-host setups; see what overrides are in effect

get

Flag	What it does	When to use it
<code>-h / --host</code>	Read the per-host value instead of the global value	Verify host-specific overrides

set

Flag	What it does	When to use it
<code>-h / --host</code>	Write a per-host value	Different protocols or editors per host

Recognised keys

Key	Allowed values	Default	What it controls
<code>git_protocol</code>	<code>https</code> , <code>ssh</code>	<code>https</code>	Protocol for <code>git clone</code> and <code>push</code>
<code>editor</code>	any executable (quoted if it has arguments)	system <code>\$EDITOR</code>	Editor for multi-line input
<code>prompt</code>	<code>enabled</code> , <code>disabled</code>	<code>enabled</code>	Interactive prompts in the terminal
<code>prefer_editor_prompt</code>	<code>enabled</code> , <code>disabled</code>	<code>disabled</code>	Open the editor for prompts instead of inline questions
<code>pager</code>	any executable	system <code>\$PAGER</code>	Pager for long output
<code>http_unix_socket</code>	filesystem path	—	Route HTTP calls through a Unix socket (proxy / VM use)
<code>browser</code>	any executable	system <code>\$BROWSER</code>	Browser opened by <code>gh browse</code> and similar
<code>color_labels</code>	<code>enabled</code> , <code>disabled</code>	<code>disabled</code>	Render label colours using RGB hex in truecolor terminals

<code>accessible_colors</code>	<code>enabled</code> , <code>disabled</code>	<code>disabled</code>	Use 4-bit accessible colour palette instead of defaults
<code>accessible_prompter</code>	<code>enabled</code> , <code>disabled</code>	<code>disabled</code>	Use an accessible prompt mode (screen-reader friendly)
<code>spinner</code>	<code>enabled</code> , <code>disabled</code>	<code>enabled</code>	Show animated progress spinner
<code>telemetry</code>	<code>enabled</code> , <code>disabled</code> , <code>log</code>	<code>enabled</code>	CLI usage telemetry reporting

Best practices

Set `editor` to a command that blocks until closed. `gh` waits for the editor to exit before continuing. Graphical editors typically return immediately unless you pass the right flag:

```
gh config set editor "code --wait"      # VS Code
gh config set editor "subl --wait"     # Sublime Text
gh config set editor "idea --wait"     # JetBrains (via toolbox CLI)
```

Terminal editors (`vim`, `nvim`, `nano`, `emacs`) block naturally and need no extra flag.

Use `--host` for per-host overrides, not separate config files. If you work with both `github.com` and a GitHub Enterprise Server instance, set `git_protocol` separately for each:

```
gh config set git_protocol ssh --host github.com
gh config set git_protocol https --host github.mycompany.com
```

This keeps a single config file with clear, auditable overrides.

Disable the prompt in automation scripts. Interactive prompts hang in CI pipelines. If you invoke `gh` in scripts outside of Actions (where `GH_TOKEN` implicitly disables them), turn prompts off globally:

```
gh config set prompt disabled
```

Keep telemetry choices deliberate. The `telemetry` key has three values: `enabled` sends usage data, `disabled` sends nothing, and `log` writes locally to `~/.config/gh/telemetry.jsonl` without sending. If you want to review what would be sent before opting in, start with `log`.

Run `gh config list` after any machine setup. The output is concise and shows every active setting, making it easy to confirm that a freshly-cloned dotfiles installation applied correctly.

Pitfalls & gotchas

`gh config set` does not validate values beyond basic types. Setting `git_protocol` to `ftp` or `prompt` to `yes` will be silently accepted and written — but the next `gh` command

will either error or fall back to the default. Always use the exact values listed in

```
gh config --help.
```

Global vs. per-host resolution can surprise you. Running `gh config get git_protocol` returns the *global* value, even if a per-host override exists. To see the effective value for a particular host, always pass `--host`:

```
gh config get git_protocol --host github.com
```

The editor must be on `$PATH` at the time `gh` is invoked. Setting `editor` to an absolute path works, but a relative name like `code` only works if the binary is in `$PATH` in the shell that runs `gh`. When this goes wrong, `gh pr create` (and similar) will fail with

```
exec: "code": executable file not found.
```

`config clear-cache` is not the same as revoking credentials. The cache stores API responses (repository metadata, etc.) for speed. Clearing it does not touch tokens or auth state — use `gh auth logout` for that.

`prefer_editor_prompt` has no effect when `prompt` is disabled. Both keys affect the prompt experience, but `prompt disabled` takes priority. Enable `prompt` first before toggling `prefer_editor_prompt`.

Worked examples

First-time setup on a new machine

After running `gh auth login` (see *auth*), configure your preferred tools before you start working:

```
gh config set git_protocol ssh
gh config set editor "nvim"
gh config set pager "less -FRX"
```

Verify everything at once:

```
gh config list
```

```
git_protocol=ssh
editor=nvim
prompt=enabled
prefer_editor_prompt=disabled
pager=less -FRX
http_unix_socket=
browser=
color_labels=disabled
accessible_colors=disabled
accessible_prompter=disabled
spinner=enabled
telemetry=enabled
```

Working with two hosts that need different git protocols

Your personal account on `github.com` uses SSH keys; your company's GitHub Enterprise Server only allows HTTPS.

```
gh config set git_protocol ssh --host github.com
gh config set git_protocol https --host github.mycompany.com
```

Confirm the overrides:

```
gh config list --host github.com
```

```
git_protocol=ssh
```

```
gh config list --host github.mycompany.com
```

```
git_protocol=https
```

Now `gh repo clone personal/project` uses SSH, and operations against `github.mycompany.com` use HTTPS, without any extra flags on every command.

Recovering from a stale cache

`gh pr list` shows a PR as open even though you merged it moments ago. Clear the cache and re-run:

```
gh config clear-cache
gh pr list
```

The fresh API call returns current state.

Recovery

`gh config set` overwrites the current value. If you set a key incorrectly, set it again with the correct value:

```
gh config set editor vim # restore a known-good editor
```

To remove a per-host override entirely, there is no `gh config unset` command — open `~/.config/gh/hosts.yml` and delete the relevant key manually, or set the per-host value to match the global default.

If `~/.config/gh/config.yml` becomes corrupted, delete it and re-run `gh config set` for each key you need. `gh` recreates the file with defaults on its next invocation.

See also

- *auth* — store and manage the credentials that `gh config` never touches directly.
- *alias* — define shorthand commands that can embed config-driven behaviour.
- *completion* — shell completion setup, which depends on the active shell detected via config.

alias

Create, manage, and share keyboard-friendly shortcuts for any `gh` command or shell pipeline.

Mental model

`gh alias` lets you give a short name to a command you type often. When you invoke the alias, `gh` expands it — inserting any extra arguments you passed — before running it. Think of it as an abbreviation table that lives in `~/.config/gh/config.yml`.

There are two flavours of alias:

- **Plain alias.** The expansion is another `gh` sub-command with optional fixed flags. `gh` handles argument passing itself.
- **Shell alias.** The expansion starts with `!` or you pass `--shell`. `gh` hands the whole expression to `sh -c`, which lets you pipe, redirect, and combine arbitrary commands.

Positional placeholders (`$1` , `$2` , ...) let a plain or shell alias accept dynamic values. Without placeholders, any extra arguments are appended at the end of the expansion.

Aliases are personal; they are stored in your local config file, not in the repository. Use `gh alias import` / `gh alias list` to move them between machines.

Synopsis

```
gh alias set    <alias> <expansion> [--clobber] [--shell]
gh alias delete {<alias> | --all}
gh alias list
gh alias import [<filename> | -] [--clobber]
```

Everyday usage

Defining a simple alias

Shorten a flag-heavy command you type every day:

```
gh alias set bugs 'issue list --label=bug'
gh bugs           # expands to: gh issue list --label=bug
```

Using positional placeholders

Accept a dynamic value at invocation time:

```
gh alias set epicsBy 'issue list --author="$1" --label=epic'
gh epicsBy alice      # expands to: gh issue list --author="alice" --
label=epic
```

Multi-word (namespace) alias

Use a quoted alias name to create a sub-command under an existing group:

```
gh alias set 'issue mine' 'issue list --assignee @me'
gh issue mine      # feels like a built-in subcommand
```

Shell alias with piping

Prefix the expansion with `!` to pass it through `sh`:

```
gh alias set igrep '!gh issue list --label="$1" | grep "$2"'
gh igrep bug "crash" # lists issues labelled "bug", filtered by "crash"
```

Or use the `--shell` flag with an expansion that does not start with `!`:

```
gh alias set --shell openpr 'gh pr view $(gh pr list --json number -q "[0].number") --web'
```

Listing all aliases

```
gh alias list
```

```
bugs      issue list --label=bug
epicsBy    issue list --author="$1" --label=epic
igrep      !gh issue list --label="$1" | grep "$2"
```

(`gh alias ls` is an accepted abbreviation.)

Deleting a single alias

```
gh alias delete bugs
```

Deleting every alias at once

```
gh alias delete --all
```

Exporting and importing aliases

Export from one machine and import on another:

```
# On the source machine
gh alias list > aliases.yml

# On the target machine
gh alias import aliases.yml
```

The YAML produced by `gh alias list` is directly consumable by `gh alias import`. Pass `-` to read from stdin:

```
cat aliases.yml | gh alias import -
```

Key options

set

Flag	What it does	When to use it
<code>--clobber</code>	Overwrite an existing alias with the same name	Updating a definition without deleting first

<code>-s / --shell</code>	Treat the expansion as a shell expression via <code>sh</code>	Pipelines, redirections, multi-command sequences
---------------------------	---	--

delete

Flag	What it does	When to use it
<code>--all</code>	Remove every stored alias in one step	Resetting a machine or wiping a stale dotfile

import

Flag	What it does	When to use it
<code>--clobber</code>	Overwrite any conflicting aliases already defined	Importing a canonical team alias file over personal definitions

Best practices

Read the expansion string from stdin with `-` to avoid quoting headaches. Shell quoting of `$1` inside single and double quotes is notoriously tricky. Pipe the expansion string in instead:

```
echo 'issue list --author="$1" --label=epic' | gh alias set epicsBy -
```

The `-` tells `gh alias set` to read the expansion from stdin, so your shell never sees the inner `$`.

Namespace aliases that extend a command group. Using a multi-word alias like `'pr review-queue'` keeps your personal shortcuts discoverable alongside the real subcommands and avoids name collisions with top-level commands.

Commit your `aliases.yml` to a dotfiles repository. Export with `gh alias list > aliases.yml`, commit the file, and import on every new machine with `gh alias import aliases.yml`. A single source of truth is easier to audit than per-machine config files.

Prefer plain aliases over shell aliases when no piping is needed. Plain aliases are faster (no shell spawn), safer (no injection risk), and easier for `gh` to complete. Reserve `--shell` for cases that genuinely need a pipeline or a conditional.

Use `--clobber` when scripting alias setup. If a setup script runs `gh alias set` on a machine that already has the alias, the command fails without `--clobber`. Add the flag to make your bootstrap script idempotent.

Pitfalls & gotchas

A plain alias expansion cannot start with `!`. The exclamation mark is the sentinel that marks a shell alias. If your expansion happens to start with `!` (e.g., a negated `jq` expression), use `--shell` instead.

`$1` in a plain alias is handled by `gh`, not by `sh`. Quoting rules differ from shell: inside a plain alias expansion, `"$1"` works as a placeholder without a shell process. Do not expect arithmetic (`${(…)}`), command substitution (`${(…)}`), or variable expansion (`$HOME`) to work in a plain alias — they require `--shell`.

Alias names cannot shadow built-in `gh` commands. Attempting to name an alias `pr` or `repo` will be rejected. Use a different name or a multi-word alias such as `'pr draft'`.

`gh alias list` output is YAML, not shell. The multiline block syntax (`| -`) is valid YAML. Do not try to source or eval the output in bash; pass it through `gh alias import` instead.

Without `--clobber`, redefining an alias is an error. If you run `gh alias set pv 'pr view'` twice, the second invocation fails with a conflict message. Either delete first or add `--clobber`.

Worked examples

Building a personal productivity kit

A developer works primarily on issues and pull requests and wants fast, finger-friendly shortcuts:

```
# List your open issues
gh alias set mine 'issue list --assignee @me --state open'

# List PRs that need your review
gh alias set review-queue 'pr list --search "review-requested:@me"'

# Open the current branch's PR in the browser
gh alias set pv 'pr view --web'

# Quick-create a draft PR from the current branch
gh alias set draft 'pr create --draft --fill'
```

Day-to-day use:

```
gh mine          # what am I working on?
gh review-queue  # who is waiting on me?
gh pv            # jump to the PR in the browser
```

Sharing aliases across a team

The team agrees on a canonical set of shortcuts for a monorepo. A maintainer creates `aliases.yml` and commits it:

```
# aliases.yml
deploy-check: 'run list --workflow=deploy.yml --limit=5'
flaky:        'issue list --label=flaky-test --state=open'
release-next: '!gh release list --limit=1 | awk "{print \$1}"'
```

Each developer imports it once:

```
gh alias import aliases.yml
```

To update an alias without removing it first:

```
gh alias import --clobber aliases.yml
```

To inspect what is currently defined after importing:

```
gh alias list
```

Debugging a shell alias

A shell alias is not behaving as expected. Inspect the stored expansion:

```
gh alias list | grep igrep
```

```
igrep !gh issue list --label="$1" | grep "$2"
```

Run the expanded command manually to isolate the problem:

```
gh issue list --label="bug" | grep "crash"
```

If the alias needs updating, overwrite it with `--clobber`:

```
gh alias set --shell --clobber igrep 'gh issue list --label="$1" --state=open |  
grep "$2"'
```

Recovery

If an alias definition is broken and causes errors on invocation, delete it and redefine:

```
gh alias delete igrep  
gh alias set --shell igrep 'gh issue list --label="$1" | grep "$2"'
```

If you have accidentally deleted aliases you need, restore them from your dotfiles

`aliases.yml`:

```
gh alias import aliases.yml
```

If you do not have a backup, check your shell history for the original `gh alias set` invocations:

```
history | grep 'alias set'
```

Alias definitions are stored in `~/.config/gh/config.yml` under the `aliases:` key. In an emergency you can inspect or hand-edit that file, but prefer the `gh alias` commands so the format stays valid.

See also

- *auth* — `gh` must be authenticated before any alias expansion that calls the API can succeed.
- *config* — the config file that stores aliases (`~/.config/gh/config.yml`); use `gh alias list` to inspect current definitions.
- *extension* — for richer, distributable plugins that go beyond what a shell alias can express.

repo

Create, clone, fork, view, and manage GitHub repositories without leaving the terminal.

Mental model

Every `gh repo` subcommand talks to GitHub's API on your behalf. Think of this command group as the GitHub web UI's repository tab — condensed into a set of composable shell commands.

There are two layers to understand:

1. **Remote repositories** live on GitHub's servers. `gh repo create`, `edit`, `archive`, `delete`, `rename`, `list`, and `view` all operate on the remote.
2. **Local checkouts** live on your disk. `gh repo clone`, `fork`, `sync`, and `set-default` bridge the remote and your working directory.

`gh` identifies the *current* repository by reading the `origin` remote (or whichever remote is set as the default — see `set-default`). If a subcommand takes an optional `[<repository>]` argument you can always spell it out as `OWNER/REPO` or pass a full GitHub URL; omitting it targets the current directory's repository.

All API calls use the active `gh auth` credential (see *auth*). For delete operations you need the extra `delete_repo` OAuth scope; `gh` will tell you when to run `gh auth refresh`.

Synopsis

```
gh repo create    [<name>] [flags]
gh repo clone     <repository> [<directory>] [-- <gitflags>...]
gh repo fork      [<repository>] [-- <gitflags>...] [flags]
gh repo view      [<repository>] [flags]
gh repo list      [<owner>] [flags]
gh repo edit      [<repository>] [flags]
gh repo set-default [<repository>] [flags]
gh repo sync      [<destination-repository>] [flags]
gh repo rename    [<new-name>] [flags]
gh repo archive   [<repository>] [flags]
gh repo unarchive [<repository>] [flags]
gh repo delete    [<repository>] [flags]
```

Everyday usage

Creating a new repository

Start a brand-new public project and immediately clone it:

```
gh repo create my-project --public --clone
```

Bootstrap from code you already have locally:

```
cd ~/code/my-project
gh repo create my-project --private --source=. --remote=origin --push
```

Run with no arguments for a guided interactive wizard:

```
gh repo create
```

Cloning a repository

```
# Clone by OWNER/REPO shorthand
gh repo clone cli/cli

# Clone to a specific directory
gh repo clone cli/cli workspace/gh-cli

# Clone with extra git flags (shallow clone)
gh repo clone cli/cli -- --depth=1
```

When you clone a fork, `gh` automatically adds the parent as a remote named `upstream` and marks it as the default repository for `gh` commands.

Forking a repository

Fork on GitHub and clone the fork in one step:

```
gh repo fork cli/cli --clone
```

Run inside an existing clone to fork the current repository:

```
gh repo fork
```

Fork into an organization rather than your personal account:

```
gh repo fork cli/cli --org my-company --clone
```

Viewing a repository

Print the README and description in the terminal:

```
gh repo view
gh repo view torvalds/linux
```

Open in a browser instead:

```
gh repo view --web
```

Extract structured data:

```
gh repo view --json name,description,stargazerCount
```

Listing repositories

```
# Your own repos (default 30)
gh repo list

# Another user or org
gh repo list my-company --limit 100
```

```
# Only private repos
gh repo list --visibility private

# Only forks
gh repo list --fork

# Filter by primary language
gh repo list --language go
```

Setting a default repository

Inside a multi-remote clone, tell `gh` which GitHub repository to use for PR and issue commands:

```
# Interactive picker
gh repo set-default

# Set explicitly
gh repo set-default owner/repo

# Check what is currently set
gh repo set-default --view

# Clear the setting
gh repo set-default --unset
```

Syncing a fork

Pull the latest changes from the parent repository into your fork's default branch:

```
# Sync your local clone from its upstream parent
gh repo sync

# Sync a specific branch
gh repo sync --branch develop

# Sync a remote fork (no local clone required)
gh repo sync your-handle/your-fork
```

Key options

create

Flag	What it does	When to use it
<code>--public</code> / <code>--private</code> / <code>--internal</code>	Set repository visibility	Required for non-interactive creation
<code>-c</code> / <code>--clone</code>	Clone the new repo after creation	Jump straight into coding
<code>-s</code> / <code>--source</code>	Path to an existing local repo	Publish local work to GitHub

<code>--push</code>	Push local commits after creating	Combine with <code>--source</code> to upload history
<code>-d / --description</code>	Short description	Shown on GitHub and in <code>gh repo list</code>
<code>-g / --gitignore</code>	Apply a gitignore template	Saves manual setup
<code>-l / --license</code>	Apply an OSS license	Run <code>gh repo license list</code> for valid keywords
<code>-p / --template</code>	Base the repo on a template repo	Org-wide project starters
<code>--add-readme</code>	Create an initial README	Needed before cloning an otherwise empty repo
<code>-r / --remote</code>	Name for the new remote	When <code>origin</code> is already taken
<code>-t / --team</code>	Grant an org team access	Org repos
<code>--disable-issues</code>	Turn off the Issues tab	Docs-only or mirror repos
<code>--disable-wiki</code>	Turn off the Wiki tab	Keeps the repo tab bar clean

clone

Flag	What it does	When to use it
<code>-u / --upstream-remote-name</code>	Name for the upstream remote when cloning a fork (default <code>upstream</code>)	Rename to match your convention
<code>--no-upstream</code>	Skip adding an upstream remote	When you do not want the parent wired in

fork

Flag	What it does	When to use it
<code>--clone</code>	Clone the fork locally	Almost always — saves a second command
<code>--remote</code>	Add a git remote for the fork without cloning	Already have a local clone
<code>--remote-name</code>	Name for the fork's remote (default <code>origin</code>)	When <code>origin</code> is taken
<code>--org</code>	Create the fork in an organization	Contributing under a team namespace
<code>--fork-name</code>	Rename the fork on creation	Avoid collision with existing repos
<code>--default-branch-only</code>	Include only the default branch	Smaller fork for quick PRs

view

Flag	What it does	When to use it
<code>-w / --web</code>	Open in browser	When you want the full GitHub UI
<code>-b / --branch</code>	View a specific branch	Inspect a branch's README
<code>--json fields</code>	Output structured JSON	Scripting, dashboards
<code>-q / --jq</code>	Filter JSON with a jq expression	Quick field extraction in pipelines

list

Flag	What it does	When to use it
<code>-L / --limit</code>	Maximum repos to return (default 30)	Orgs with many repos
<code>--visibility</code>	Filter by <code>public</code> , <code>private</code> , or <code>internal</code>	Audit or scope work
<code>--fork</code>	Show only forks	Find forks to clean up
<code>--source</code>	Show only non-forks	Original repos only
<code>--archived</code>	Show only archived repos	Housekeeping
<code>--no-archived</code>	Omit archived repos	Active work list
<code>-l / --language</code>	Filter by primary language	Polyglot accounts
<code>--topic</code>	Filter by topic	Repos tagged with a keyword
<code>--json fields</code>	Output structured JSON	Scripting

edit

Flag	What it does	When to use it
<code>-d / --description</code>	Change the description	Keep repo metadata current
<code>--default-branch</code>	Change the default branch	After a main-branch rename
<code>--visibility</code>	Change visibility (<code>public</code> / <code>private</code> / <code>internal</code>)	Requires <code>--accept-visibility-change-consequences</code>
<code>--delete-branch-on-merge</code>	Auto-delete head branch after merge	Cleaner PR workflow
<code>--enable-auto-merge</code>	Allow PRs to auto-merge when checks pass	CI-driven repos
<code>--enable-squash-merge / --enable-rebase-merge / --enable-merge-commit</code>	Toggle merge strategies	Enforce a team standard
<code>--enable-issues / --enable-wiki / --enable-discussions</code>	Toggle feature tabs	Slim down unused tabs
<code>--add-topic / --remove-topic</code>	Manage repository topics	Discoverability
<code>--template</code>	Make the repo a template	Publish a project starter

sync

Flag	What it does	When to use it
<code>-b / --branch</code>	Branch to sync (default: default branch)	Keep a non-default branch current
<code>-s / --source</code>	Source repository	Sync from something other than the parent
<code>--force</code>	Hard reset the destination to match the source	After divergent history on a fork

set-default

Flag	What it does	When to use it
<code>-v / --view</code>	Print the current default	Quick status check
<code>-u / --unset</code>	Remove the default	Reset after finishing a project

Best practices

Prefer `--clone` over cloning separately. Both `gh repo create` and `gh repo fork` accept `--clone`. Using it saves a round trip and ensures the remote is wired correctly from the start.

Use `gh repo set-default` in multi-remote setups. When a directory has remotes pointing to different GitHub repositories, `gh` may pick the wrong one for `gh pr` and `gh issue` commands. Set the default once and forget it:

```
gh repo set-default owner/canonical-repo
```

Think hard before changing visibility. Changing a public repository to private has irreversible side-effects: stars and watchers are lost, public forks are detached from the network, and push rulesets are disabled. The `--accept-visibility-change-consequences` flag you must pass is a reminder to weigh those costs first.

Automate with `--json` and `--jq`. Most `gh repo` subcommands support JSON output. Pipe into `jq` for one-liners:

```
# List all your private repos by name
gh repo list --visibility private --json name --jq '[][.name]'

# Show star counts sorted descending
gh repo list --json name,stargazerCount \
  --jq 'sort_by(-.stargazerCount) | .[] | "\(.stargazerCount)\t\(.name)''
```

Add gitignore and license at creation time. It is far easier to set `--gitignore` and `--license` when creating than to bolt them on later. Run `gh repo license list` to see valid license keywords before creating.

Use `gh repo sync --force` only on forks you control. The flag performs a hard reset. Any commits in the destination branch that are not in the source will be permanently discarded.

Pitfalls & gotchas

`gh repo delete` requires the `delete_repo` scope. If you see a permissions error, add the scope and retry:

```
gh auth refresh -s delete_repo
gh repo delete owner/repo --yes
```

Deleting without an explicit argument is always interactive. Even with `--yes`, `gh` prompts for confirmation when no repository argument is supplied. Pass the full `OWNER/REPO` to make it non-interactive:

```
gh repo delete owner/repo --yes # non-interactive
gh repo delete --yes            # still prompts — --yes is ignored here
```

`gh repo sync` **only fast-forwards by default.** If your fork has diverged (for example you rebased a branch), the sync will fail unless you pass `--force`. That flag hard-resets the branch, discarding divergent commits.

Forking rewires origin. Running `gh repo fork` inside an existing clone sets `origin` to your fork and renames the original remote to `upstream`. If scripts or CI jobs reference a remote by name, audit them before forking.

Changing the default branch with `gh repo edit --default-branch` does not update local clones. After the rename, each collaborator must update their local tracking reference:

```
git fetch origin
git branch -u origin/<new-name> <new-name>
git remote set-head origin -a
```

Alternatively, delete and re-clone.

`gh repo list --fork` **does not cross ownership boundaries.** When listing forks in an organization, the output will not include forks owned by individual members — only forks owned by the org itself.

Worked examples

Publishing a new open-source project from scratch

You have code in `~/code/hello-world` and want to publish it:

```
cd ~/code/hello-world
git init
git add .
git commit -m "Initial commit"

gh repo create hello-world \
  --public \
  --description "A friendly Hello World" \
  --license mit \
  --source=. \
  --remote=origin \
  --push
```

- ✓ Created repository ada/hello-world on GitHub
<https://github.com/ada/hello-world>
- ✓ Added remote <https://github.com/ada/hello-world.git>
- ✓ Pushed commits to <https://github.com/ada/hello-world.git>

Browse the result immediately:

```
gh repo view --web
```

Contributing to an upstream project (fork-and-PR workflow)

Fork an upstream project, clone your fork, make a change, and open a PR:

```
# Fork and clone in one step
gh repo fork cli/cli --clone
cd cli

# Upstream is wired automatically
git remote -v
```

```
origin    https://github.com/ada/cli.git (fetch)
origin    https://github.com/ada/cli.git (push)
upstream  https://github.com/cli/cli.git (fetch)
upstream  https://github.com/cli/cli.git (push)
```

```
git switch -c fix/typo-in-readme
# ... edit files ...
git commit -am "Fix typo in README"
git push origin fix/typo-in-readme

# Open a PR against the upstream default branch
gh pr create --fill
```

Keep your fork current with upstream as work continues:

```
gh repo sync
```

Auditing and cleaning up stale repositories

List all archived repos sorted by last-updated date:

```
gh repo list --archived --json name,updatedAt \
--jq '.[ ] | "\(.updatedAt) \(.name)"' | sort
```

Delete a stale fork after confirming you no longer need it:

```
gh repo delete ada/old-fork --yes
```

Locking down a repository after a project ends

Archive it so the history is preserved but no new issues or PRs can be opened:

```
gh repo archive ada/finished-project --yes
```

If the project is revived:

```
gh repo unarchive ada/finished-project --yes
```

Collecting star counts across an organization

```
gh repo list my-org --limit 200 \
  --json name,stargazerCount,isPrivate \
  --jq '.[ ] | select(.isPrivate == false) | [.name, .stargazerCount] | @tsv' \
  | sort -t$'\t' -k2 -rn
```

Recovery

Accidentally deleted a repository. GitHub keeps deleted repositories recoverable for 90 days. Visit https://github.com/settings/deleted_repositories and restore from there. There is no `gh` CLI command for this — use the web UI.

Pushed to the wrong repository after a remote rename. Check where `origin` points, then correct it:

```
git remote -v
git remote set-url origin https://github.com/correct-owner/correct-repo.git
```

Cloned a fork without an upstream remote. Add it manually:

```
git remote add upstream https://github.com/original-owner/original-repo.git
git fetch upstream
```

Or re-clone with `gh repo clone`, which adds the upstream remote automatically for known forks.

Visibility change had unintended consequences. To reverse a private-to-public or public-to-private change:

```
gh repo edit --visibility public --accept-visibility-change-consequences
```

Stars, watchers, and fork-network connections lost during a private period are not restored.

See also

- *auth* — managing the credentials that `gh repo` uses for all API calls.
- *pr* — open and review pull requests against repositories cloned or forked here.
- *issue* — file and track issues in repositories managed here.
- *release* — publish versioned releases for repositories you create.
- *browse* — open any part of a repository in a browser with `gh browse`.

browse

Open the current repository — or any part of it — in your web browser from the terminal.

Mental model

`gh browse` constructs a GitHub URL and hands it to your system browser (or prints it with `--no-browser`). It does not clone anything or change local state. Think of it as a smart bookmark: it knows which repository the current directory belongs to, so you never have to copy and paste URLs by hand.

The command accepts three kinds of arguments — a number (issue or pull request), a file path (optionally with a line anchor), or a commit SHA — and routes each to the right page. Flags like `--settings` or `--releases` go directly to well-known repository sections without requiring you to memorise the URL shape.

Because the destination is just a URL, you can capture it for sharing or scripting with `--no-browser` rather than letting the browser open.

Synopsis

```
gh browse [<number> | <path> | <commit-sha>] [flags]
```

Everyday usage

Open the home page of the current repository:

```
gh browse
```

Open an issue or pull request by number:

```
gh browse 217
```

Open a file in the repository browser:

```
gh browse cmd/gh/main.go
```

Open a file at a specific line:

```
gh browse main.go:312
```

Open a directory:

```
gh browse script/
```

Open a specific commit:

```
gh browse 77507cd94ccafcf568f8560cfecde965fcfa63
```

Print the URL without opening a browser (useful for copying or scripting):

```
gh browse --no-browser
```

Open repository settings directly:

```
gh browse --settings
```

Open the Actions tab:

```
gh browse --actions
```

Open releases:

```
gh browse --releases
```

Open the wiki:

```
gh browse --wiki
```

Open projects:

```
gh browse --projects
```

Viewing a file on another branch or commit

```
# View a file at the tip of the bug-fix branch
gh browse main.go --branch bug-fix

# View a file as it was at a specific commit
gh browse main.go --commit=77507cd94cca9cf568f8560cfecde965fcfa63
```

Viewing blame

```
gh browse main.go:312 --blame
```

Targeting a different repository

If you are not inside the repository you want to open, pass `--repo` :

```
gh browse --repo cli/cli
gh browse --repo github.mycompany.com/platform/backend
```

Key options

Option	What it does	When to use it
<code>-n</code> / <code>--no-browser</code>	Print the URL instead of opening a browser	Scripting, headless environments, copying links
<code>-R</code> / <code>--repo</code>	Target a different repository	Working outside the repo directory
<code>-b</code> / <code>--branch</code>	View at the tip of a named branch	Reviewing another branch without checking it out
<code>-c</code> / <code>--commit</code>	View at a specific commit SHA (omit value for last commit)	Linking to a historical snapshot

<code>--blame</code>	Open the blame view for a file	Investigating who changed a line
<code>-a / --actions</code>	Open the Actions tab	Checking CI status quickly
<code>-p / --projects</code>	Open the Projects board	Checking project status
<code>-r / --releases</code>	Open the Releases page	Checking or sharing release notes
<code>-s / --settings</code>	Open repository settings	Admin tasks
<code>-w / --wiki</code>	Open the repository wiki	Reading or editing docs

Best practices

Use `--no-browser` to generate shareable links. When you want to point a colleague to a specific file, line, or commit, generate the URL and paste it into chat rather than opening the browser yourself:

```
gh browse src/auth.ts:42 --no-browser
```

Use `--commit` without a value to link to the last commit. Omitting the SHA defaults to the most recent commit on the current branch, which produces a permanent URL that will not drift as the branch advances:

```
gh browse README.md --commit
```

Combine `--repo` with `--no-browser` in scripts. This lets you build permalink generation helpers that do not depend on a git checkout being present:

```
# Print the Actions tab URL for any repo
gh browse --repo "$OWNER/$REPO" --actions --no-browser
```

Set `BROWSER` to control which browser opens. The `BROWSER` environment variable overrides the system default. Useful when you want to open links in a work profile while your default browser is personal:

```
BROWSER="/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" gh browse
```

Pitfalls & gotchas

`gh browse` requires a git repository (unless `--repo` is given). Running it outside a git working tree produces an error. Pass `--repo OWNER/REPO` to target a remote repository from any directory.

Number arguments open issues or pull requests, not commit SHAs. `gh browse 42` goes to issue or PR #42. To open a commit by a short SHA that happens to be all digits, you still need to spell out enough characters for GitHub to distinguish it from a number — in practice, use the full or a sufficiently long prefix of the commit hash.

`--commit` without a value means the last commit, not the current file state.

`gh browse README.md --commit` links to the last commit that touched any file on the branch, not specifically the last commit that changed `README.md`. The result is a permalink to a commit tree view, not a file-history view.

Line anchors require a file argument. `gh browse :42` is not valid; you must provide the path: `gh browse src/main.go:42`.

Branch names with slashes need no special quoting. `--branch feat/my-feature` works as-is; the slash is treated as part of the branch name, not a path separator.

Worked examples

Sharing a link to a specific line during code review

You spot a tricky function and want to point a reviewer at it without switching to the browser first:

```
gh browse src/payment/processor.go:88 --no-browser
```

```
https://github.com/acme/checkout/blob/main/src/payment/processor.go#L88
```

Paste that URL into your pull-request comment or Slack message.

Checking CI for a different repository without cloning it

```
gh browse --repo acme/infra --actions --no-browser
```

```
https://github.com/acme/infra/actions
```

Open it, or pass it to `open` / `xdg-open` manually.

Jumping to an issue or pull request from the terminal

While reading a `git log` you see a reference to PR #1337. Open it without leaving the terminal:

```
gh browse 1337
```

The browser navigates to the pull-request page directly. If the number corresponds to both an issue and a pull request (they share the same number space on GitHub), GitHub will route you to whichever exists.

Generating a permalink to last night's release commit

```
# Find the commit hash
git log --oneline -1 v2.4.0

# Open it as a permanent GitHub link
gh browse a3f9c12 --no-browser
```

```
https://github.com/acme/checkout/commit/a3f9c12...
```

Browsing a file on a feature branch without checking it out

```
gh browse pkg/api/routes.go --branch feat/new-router
```

GitHub opens the file as it exists on `feat/new-router`, leaving your local working tree on whatever branch you are currently on.

Recovery

`gh browse` is read-only and stateless — it opens a URL and exits. Nothing about your local repository changes, so there is nothing to undo.

If the browser fails to open, use `--no-browser` to retrieve the URL and open it manually:

```
gh browse --no-browser
```

If the command errors with “not a git repository”, either `cd` into a checkout or add `--repo OWNER/REPO`.

If you see “repository not found”, confirm that the active `gh` account has access to the repository (see *auth*).

See also

- *auth* — configure and switch the GitHub account `gh browse` uses.
- *repo* — `gh repo view --web` is an alias-style alternative for opening the repository home page.
- *pr* — `gh pr view --web` opens a specific pull request in the browser.
- *issue* — `gh issue view --web` opens a specific issue in the browser.

search

Search all of GitHub — repositories, issues, pull requests, commits, and code — directly from the terminal without opening a browser.

Mental model

GitHub's search is a single global index that covers every public resource on the platform (plus private resources you have access to). `gh search` exposes that index through five subcommands — `repos`, `issues`, `prs`, `commits`, `code` — each returning a specific resource type.

Every subcommand accepts two complementary ways to express a query:

- **Keyword arguments** — positional words (or a quoted phrase) that GitHub matches against the resource's primary text fields.
- **Qualifier flags** — typed, structured filters such as `--language=go`, `--state=open`, or `--stars=">100"` that map to GitHub search qualifiers under the hood.

You can combine both in a single command. Under the hood `gh` assembles the flags into a GitHub search query string and sends it to the Search API; the result is the same set of results you would get from `github.com/search`.

One important quirk: GitHub search qualifiers that begin with a hyphen (to *exclude* results) conflict with the shell's flag parsing. Use `--` before the qualifier to stop flag interpretation:

```
# Find issues that do NOT have the label "wontfix"
gh search issues -- "-label:wontfix"
```

Results default to 30 items. Every subcommand supports `--json` for machine-readable output and `-w` / `--web` to open the equivalent search on `github.com` in a browser.

Synopsis

```
gh search code      <query> [flags]
gh search commits   [<query>] [flags]
gh search issues    [<query>] [flags]
gh search prs       [<query>] [flags]
gh search repos     [<query>] [flags]
```

Everyday usage

Find repositories by keyword:

```
gh search repos "vim plugin"
```

Find open issues assigned to you across all of GitHub:

```
gh search issues --assignee=@me --state=open
```

Find pull requests where you have been asked to review:

```
gh search prs --review-requested=@me --state=open
```

Find Go files that contain a specific function call pattern:

```
gh search code "http.ListenAndServe" --language=go
```

Find all merged PRs you authored in the last month:

```
gh search prs --author=@me --merged --created=">2025-05-17"
```

Open the search results in a browser for further filtering:

```
gh search repos "oauth client" --language=python -w
```

search repos

Find repositories by keyword, language, topic, star count, and more.

```
# Public repos in a specific organisation
gh search repos --owner=cli --visibility=public

# Repos with a good-first-issue count suitable for contributors
gh search repos --language=rust --good-first-issues=">=5" --stars=">500"

# Repos tagged with multiple topics
gh search repos --topic=kubernetes,operator

# Exclude archived repos
gh search repos my-tool --archived=false
```

search issues

Find issues across all of GitHub with fine-grained filters.

```
# Issues you were mentioned in
gh search issues --mentions=@me --state=open

# Issues with more than 50 comments, sorted by most comments first
gh search issues performance --comments=">50" --sort=comments

# Issues with no assignee in a specific repo
gh search issues --repo=myorg/myrepo --no-assignee --state=open

# Issues updated in the last week
gh search issues --updated=">2026-06-10" --state=open
```

search prs

Find pull requests with PR-specific filters such as review status, base branch, and CI check state.

```
# Draft PRs in a repo
gh search prs --repo=myorg/myrepo --draft --state=open

# PRs waiting for review that target the main branch
gh search prs --base=main --review=required --state=open
```

```
# Merged PRs approved by a specific reviewer
gh search prs --reviewed-by=alice --merged

# PRs that failed CI
gh search prs --checks=failure --state=open
```

search commits

Search commit messages and metadata across public repositories or specific repos.

```
# Commits by a specific author containing a keyword
gh search commits "fix memory leak" --author=alice

# Commits committed before a date
gh search commits --committer-date="<2024-01-01" --repo=myorg/myrepo

# Merge commits only
gh search commits --merge --repo=cli/cli
```

search code

Search within file contents or file paths across public repositories.

```
# Find usages of a function across all Go files
gh search code "ctx.Deadline" --language=go

# Find a pattern in a specific repo
gh search code "TODO: remove" --repo=myorg/myrepo

# Search only file paths (not file contents)
gh search code "Makefile" --match=path

# Find config files by extension
gh search code database --extension=yaml --owner=myorg
```

Note: code search uses a legacy GitHub API. Results may differ from github.com/search and regex search is not yet available via this path.

Key options

Common flags (most subcommands)

Flag	What it does	When to use it
<code>-L / --limit int</code>	Maximum results to return (default 30)	Increase to fetch more than 30 matches
<code>--owner strings</code>	Filter by owner login	Scope to one or more orgs / users
<code>-R / --repo strings</code>	Filter on specific repository (not available for <code>search repos</code>)	Narrow to known repos

<code>--json fields</code>	Output JSON with named fields	Scripting and automation
<code>-q / --jq expression</code>	Filter JSON output with a jq expression	Extract specific fields inline
<code>-t / --template string</code>	Format JSON with a Go template	Custom tabular output
<code>-w / --web</code>	Open the query in a browser	Visual exploration / further filtering
<code>--visibility strings</code>	Filter by visibility: <code>public</code> , <code>private</code> , <code>internal</code> (not available for <code>search code</code>)	Limit to private repos you own

search repos flags

Flag	What it does	When to use it
<code>--language string</code>	Filter by primary coding language	Language-specific discovery
<code>--stars number</code>	Filter by star count (supports <code>></code> , <code><</code> , ranges)	Quality signal
<code>--forks number</code>	Filter by fork count	Activity signal
<code>--topic strings</code>	Filter by topic tags	Thematic discovery
<code>--license strings</code>	Filter by license type	License-compatible forks
<code>--archived</code>	Include or exclude archived repos (<code>true</code> / <code>false</code>)	Focus on active projects
<code>--include-forks string</code>	Include forks: <code>false</code> , <code>true</code> , <code>only</code>	Explore derivative work
<code>--sort string</code>	Sort by <code>forks</code> , <code>stars</code> , <code>updated</code> , <code>help-wanted-issues</code>	Change ranking
<code>--order string</code>	<code>asc</code> or <code>desc</code> (with <code>--sort</code>)	Reverse the ranking
<code>--good-first-issues number</code>	Filter by count of “good first issue” labels	Find welcoming projects
<code>--match strings</code>	Restrict match to <code>name</code> , <code>description</code> , or <code>readme</code>	Target specific fields
<code>--size string</code>	Filter on repo size in kilobytes	Avoid massive codebases

search issues flags

Flag	What it does	When to use it
<code>--state string</code>	<code>open</code> or <code>closed</code>	Focus on actionable items
<code>--assignee string</code>	Filter by assignee; <code>@me</code> for yourself	Your workload view
<code>--author string</code>	Filter by issue author	Track a user’s reports
<code>--label strings</code>	Filter by one or more labels	Triage workflows

<code>--no-label</code>	Only issues with no labels	Untriaged issues
<code>--milestone title</code>	Filter by milestone	Release planning
<code>--comments number</code>	Filter by comment count	Hot topics
<code>--created date</code>	Filter by creation date	Time-boxed retrospectives
<code>--updated date</code>	Filter by last-updated date	Recently active threads
<code>--involves user</code>	Any involvement (author, assignee, commenter, mention)	Full involvement view
<code>--match strings</code>	Restrict to <code>title</code> , <code>body</code> , or <code>comments</code>	Precision keyword matching
<code>--include-prs</code>	Include PRs in issue results	Combined view
<code>--sort string</code>	Sort by <code>comments</code> , <code>created</code> , <code>updated</code> , <code>reactions</code> , etc.	Surface high-activity items

search prs flags

Shares most flags with `search issues`, with the following additions:

Flag	What it does	When to use it
<code>--draft</code>	Filter on draft state	Exclude work-in-progress
<code>--merged</code>	Only merged PRs	Audit merged work
<code>--merged-at date</code>	Filter by merge date	Release retrospectives
<code>-B / --base string</code>	Filter by base branch name	PRs targeting a specific branch
<code>-H / --head string</code>	Filter by head branch name	Find a specific feature branch
<code>--checks string</code>	Filter by CI status: <code>pending</code> , <code>success</code> , <code>failure</code>	Find broken builds
<code>--review string</code>	Filter by review status: <code>none</code> , <code>required</code> , <code>approved</code> , <code>changes_requested</code>	Review queue
<code>--review-requested user</code>	Filter by reviewer or team requested	Your review queue
<code>--reviewed-by user</code>	Filter by reviewer who already reviewed	Audit reviewed PRs

search commits flags

Flag	What it does	When to use it
<code>--author string</code>	Filter by author login	A contributor's commits
<code>--author-name string</code>	Filter by author display name	When login is unknown
<code>--author-date date</code>	Filter by authored date	Time-bounded queries
<code>--committer string</code>	Filter by committer login	Bot or CI committer
<code>--committer-date date</code>	Filter by committed date	When merges shift dates
<code>--hash string</code>	Filter by commit hash	Look up a specific commit
<code>--merge</code>	Only merge commits	Merge history

<code>--sort string</code>	Sort by <code>author-date</code> or <code>committer-date</code>	Chronological ordering
----------------------------	---	------------------------

search code flags

Flag	What it does	When to use it
<code>--language string</code>	Filter by language	Precision code search
<code>--extension string</code>	Filter by file extension	Non-standard language files
<code>--filename string</code>	Filter by filename	Config or manifest lookup
<code>--match strings</code>	Restrict to <code>file</code> (contents) or <code>path</code>	Path-only or content-only
<code>--size string</code>	Filter by file size in kilobytes	Avoid generated/minified files

Best practices

Prefer qualifier flags over raw query strings where possible. Flags produce structured queries that GitHub's API can execute efficiently, and they are unambiguous. A raw keyword query is fine for initial exploration but a flag-only query is easier to automate and less prone to tokenisation surprises.

Use `--json` and `--jq` for scripting. The tabular default output is designed for reading, not parsing. When you need to feed results into another tool, use `--json fields --jq .` to get structured data:

```
# Extract repository full names as plain text
gh search repos "oauth client" --language=go --json fullName --jq '.[].fullName'
```

Use `@me` as a portable self-reference. The special token `@me` resolves to the active authenticated user, so scripts and aliases stay portable across accounts:

```
gh search prs --review-requested=@me --state=open
```

Set `--limit` intentionally. The default is 30. GitHub's API caps results at 1,000 per query. If you expect more than 30 relevant results, raise the limit or narrow your filters — do not rely on the default to show you everything.

Combine keyword arguments with qualifier flags for tighter results. GitHub applies both, which narrows the result set more precisely than either alone:

```
# "authentication" in the text, Go language, more than 200 stars
gh search repos authentication --language=go --stars=">200"
```

Open with `-w` to explore interactively. When building a complex query, start with `gh search ... -w` to see the full result count and GitHub's faceted sidebar before committing to flags.

Pitfalls & gotchas

Leading-hyphen qualifiers are eaten by the shell. Any search qualifier that starts with `-` (exclusion syntax) will be misinterpreted as a flag unless you use `--`:

```
# WRONG – shell sees -label as an unknown flag
gh search issues -label:bug

# CORRECT – everything after -- is treated as a query argument
gh search issues -- "-label:bug"
```

gh search code uses a legacy API. The code search endpoint that `gh` targets is powered by GitHub’s older search infrastructure. New features such as regex search and the exact result counts shown on `github.com` are not available. If your results look incomplete or differ from the website, use `-w` to fall back to the browser-based code search.

Date filter values require quoting when using comparison operators. Values that include `>` or `<` must be quoted to prevent the shell from treating them as redirects:

```
# WRONG – shell redirects stdin/stdout
gh search repos --stars=>500

# CORRECT
gh search repos --stars=">500"
```

The default `--limit` of 30 silently drops results. There is no warning when more results exist beyond the limit. If you expect a large result set, pass `-L 100` (or higher, up to 1,000) explicitly.

`--sort` and `--order` are linked. Specifying `--order` without `--sort` has no effect; the API ignores it. Always pair them:

```
gh search repos kubernetes --sort=stars --order=desc
```

Private repository searches require the right scope. Searching private repos requires that your token has the `repo` scope. If results are missing private content you expect to see, check your auth scopes with `gh auth status --show-token` (or run `gh auth refresh --scopes repo` see *auth*).

Worked examples

Finding a repository to contribute to

You want to find a well-maintained Go project related to CLI tooling that is welcoming to new contributors:

```
gh search repos cli terminal \
  --language=go \
  --stars=">100" \
  --good-first-issues=">=3" \
  --archived=false \
  --sort=stars \
  --order=desc \
  -L 10
```

NAME	STARS	FORKS	UPDATED
charmbracelet/bubbletea	27.8k	791	2026-06-16

urfave/cli	22.2k	1.6k	2026-06-15
spf13/cobra	38.1k	2.8k	2026-06-14
...			

Open the top result in a browser to read its contribution guide:

```
gh search repos cli terminal --language=go --good-first-issues=">=3" -w
```

Auditing open review requests

At the start of a work session, surface every open PR waiting for your review, sorted by most recently updated:

```
gh search prs \
  --review-requested=@me \
  --state=open \
  --sort=updated \
  --order=desc \
  --json number,title,repository,url \
  --jq '.[0] | "\(.repository.nameWithOwner)#\(.number) \(.title)'"
```

```
myorg/backend#312 Fix race condition in worker pool
myorg/frontend#98 Add dark mode toggle
myorg/infra#54 Bump Terraform provider to 5.x
```

Tracking down a regression

A bug was introduced sometime in April 2026 in the `myorg/payments` service. Search merged PRs in that window to identify candidates:

```
gh search prs \
  --repo=myorg/payments \
  --merged \
  --merged-at="2026-04-01..2026-04-30" \
  --sort=updated \
  --json number,title,author,mergedAt,url
```

Then narrow to PRs touching a specific area:

```
gh search prs "checkout flow" \
  --repo=myorg/payments \
  --merged \
  --merged-at="2026-04-01..2026-04-30"
```

Finding internal usage of a deprecated function

Before removing `legacyAuth()` from a shared library, confirm which repositories in your organisation still call it:

```
gh search code "legacyAuth(" \
  --owner=myorg \
  --language=go \
```

```
--json repository,path,url \  
--jq '.[] | "\(.repository.nameWithOwner) \(.path)'"
```

```
myorg/api-gateway internal/middleware/auth.go  
myorg/webhook-worker pkg/handler/verify.go
```

Recovery

If a `gh search` command returns no results when you expect some:

1. Verify authentication with `gh auth status`. If the token is expired, run `gh auth login` to re-authenticate (see *auth*).
2. Check that your token has the `repo` scope if you are searching private repositories:
`gh auth refresh --scopes repo`.
3. Simplify the query — remove flags one at a time until results appear, then add them back to identify the over-restrictive filter.
4. Use `-w` to run the same query on `github.com` and compare; if the website returns results the CLI does not, there may be an API lag or a code-search legacy-index limitation.

If a qualifier prefixed with `-` causes a “unknown flag” error, wrap the entire qualifier in quotes and prepend `--`:

```
gh search issues -- "-label:wontfix"
```

See also

- *auth* — manage authentication tokens and scopes required for private searches.
- *issue* — `gh issue list` for searching within a single repository without the global search API.
- *pr* — `gh pr list` for repository-scoped PR filtering.
- *repo* — `gh repo list` for listing repositories you own or are a member of.
- *browse* — `gh browse` to open a specific repository in the browser after finding it with `gh search repos`.

pr

Manage the full lifecycle of GitHub pull requests from the command line: create, inspect, review, merge, and close — without leaving your terminal.

Mental model

A pull request (PR) is GitHub’s unit of code review. It asks: “please merge the commits on my branch into your branch.” `gh pr` is a direct API client for that concept. Every subcommand targets a specific PR, identified by its number, URL, or head branch name — `gh pr` resolves them all to the same thing.

When you run a command against “the current branch” and omit an explicit PR identifier, `gh pr` looks up which open PR has your branch as its head. This means you can `gh pr view`, `gh pr checks`, and `gh pr merge` without ever typing a number — as long as your branch has a PR and you are on that branch.

The subcommands fall into two groups:

- **General commands** (`create`, `list`, `status`) — work across many PRs or create a new one.
- **Targeted commands** (`view`, `checkout`, `checks`, `review`, `merge`, `edit`, `diff`, `comment`, `ready`, `close`, `reopen`, `revert`, `update-branch`, `lock`, `unlock`) — operate on a single PR.

Synopsis

```
gh pr create      [flags]
gh pr list        [flags]
gh pr status      [flags]

gh pr view        [<number> | <url> | <branch>] [flags]
gh pr checkout    [<number> | <url> | <branch>] [flags]
gh pr checks      [<number> | <url> | <branch>] [flags]
gh pr diff         [<number> | <url> | <branch>] [flags]
gh pr review      [<number> | <url> | <branch>] [flags]
gh pr comment      [<number> | <url> | <branch>] [flags]
gh pr edit         [<number> | <url> | <branch>] [flags]
gh pr merge        [<number> | <url> | <branch>] [flags]
gh pr ready        [<number> | <url> | <branch>] [flags]
gh pr update-branch [<number> | <url> | <branch>] [flags]
gh pr revert       [<number> | <url> | <branch>] [flags]
gh pr close        [<number> | <url> | <branch>] [flags]
gh pr reopen       [<number> | <url> | <branch>] [flags]
gh pr lock         [<number> | <url> | <branch>] [flags]
gh pr unlock       [<number> | <url> | <branch>] [flags]
```

All targeted subcommands accept `-R OWNER/REPO` to operate on a different repository than the one detected from the current directory.

Everyday usage

Opening a new pull request

The fastest path when your branch commits already describe the change:

```
gh pr create --fill
```

`--fill` uses the commits' titles and bodies for the PR title and description. Omit it to get an interactive prompt, or go fully non-interactive:

```
gh pr create --title "Fix login redirect loop" --body "Closes #42"
```

Open as a draft when the work is not yet ready for review:

```
gh pr create --draft --title "WIP: refactor auth middleware"
```

Listing open pull requests

```
gh pr list
```

Narrow by author, label, or state:

```
gh pr list --author "@me"  
gh pr list --label "needs-review" --state open  
gh pr list --state merged --limit 10
```

Checking the status of your own PRs

```
gh pr status
```

This shows PRs you authored, PRs assigned to you, and PRs that mention you.

Viewing a PR

```
gh pr view          # PR for the current branch  
gh pr view 123  
gh pr view --web    # open in browser
```

Add `--comments` to see the discussion thread:

```
gh pr view 123 --comments
```

Checking CI status

```
gh pr checks        # for the current branch's PR  
gh pr checks 123
```

Watch checks live until they all finish:

```
gh pr checks --watch  
gh pr checks --watch --fail-fast # stop watching on the first failure
```

Reviewing a PR

Approve the PR for the current branch:

```
gh pr review --approve
```

Request changes with a message:

```
gh pr review 123 --request-changes --body "Please add tests for the edge case."
```

Leave a comment without approving or blocking:

```
gh pr review 123 --comment --body "Looks good, left a nit."
```

Merging

Merge with a merge commit:

```
gh pr merge 123 --merge
```

Squash all commits into one:

```
gh pr merge 123 --squash --delete-branch
```

Rebase the commits onto the base branch:

```
gh pr merge 123 --rebase
```

Enable auto-merge (merges automatically once all required checks pass):

```
gh pr merge 123 --auto --squash
```

Checking out a PR locally

```
gh pr checkout 123
```

Override the local branch name:

```
gh pr checkout 123 --branch fix-login-local
```

Commenting on a PR

```
gh pr comment 123 --body "LGTM"
```

Editing a PR after creation

Change the title and add a label:

```
gh pr edit 123 --title "Fix: login redirect loop" --add-label "bug"
```

Request an additional reviewer:

```
gh pr edit 123 --add-reviewer octocat
```

Marking a draft PR as ready

```
gh pr ready 123
```

Convert a ready PR back to draft:

```
gh pr ready 123 --undo
```

Updating a PR branch that has fallen behind

Merge the latest base branch into the PR branch:

```
gh pr update-branch 123
```

Rebase instead:

```
gh pr update-branch 123 --rebase
```

Closing and reopening

```
gh pr close 123 --comment "Superseded by #200"
gh pr reopen 123 --comment "Reviving after further discussion"
```

Reverting a merged PR

```
gh pr revert 123
```

This creates a new PR that undoes the changes introduced by PR 123.

Viewing the diff

```
gh pr diff          # diff for current branch's PR
gh pr diff 123
gh pr diff 123 --name-only
gh pr diff 123 --exclude '*.lock'
```

Key options

create

Flag	What it does	When to use it
<code>-f / --fill</code>	Populate title and body from commit messages	The commits already explain the change
<code>--fill-first</code>	Use only the first commit for title and body	A single-commit PR
<code>--fill-verbose</code>	Use full commit message bodies in the description	When commit bodies contain important context
<code>-d / --draft</code>	Open as a draft	Work in progress, not yet ready for review
<code>-t / --title</code>	Set the PR title	Non-interactive workflows

<code>-b / --body</code>	Set the PR body	Non-interactive workflows
<code>-F / --body-file</code>	Read the body from a file	Long descriptions prepared in an editor
<code>-B / --base</code>	Target branch to merge into	When the default branch is not the right target
<code>-r / --reviewer</code>	Request review from a handle or team	Assign reviewers at creation time
<code>-a / --assignee</code>	Assign people; @me for yourself	Assign at creation time
<code>-l / --label</code>	Add labels by name	Categorise the PR immediately
<code>--no-maintainer-edit</code>	Prevent maintainers from pushing to the head branch	Forks where you want full control
<code>-w / --web</code>	Open the browser to finish creating	Prefer the web form
<code>--dry-run</code>	Print details instead of creating the PR (may still push git changes)	Validate flags before running for real
<code>-T / --template</code>	Start from a body template file	Enforce a consistent PR format

list

Flag	What it does	When to use it
<code>-s / --state</code>	open , closed , merged , or all	Review historical PRs
<code>-A / --author</code>	Filter by author login; @me for yourself	Find your own PRs
<code>-a / --assignee</code>	Filter by assignee	Your review queue
<code>-l / --label</code>	Filter by one or more labels	Triage by category
<code>-B / --base</code>	Filter by target branch	PRs targeting a release branch
<code>-S / --search</code>	GitHub search query	Complex filters (e.g., review:required)
<code>-L / --limit</code>	Maximum results (default 30)	Wider history scans
<code>--json</code>	Output as JSON with named fields	Scripting and automation
<code>-q / --jq</code>	Filter JSON output with a jq expression	Extract specific fields
<code>-w / --web</code>	Open list in browser	Browse interactively

checks

Flag	What it does	When to use it
<code>--watch</code>	Poll until all checks complete	Block a script until CI finishes
<code>--fail-fast</code>	Stop watching on the first failure (requires --watch)	Fail scripts early
<code>-i / --interval</code>	Refresh interval in seconds (default 10)	Reduce API calls on slow CI

<code>--required</code>	Show only required checks	Cut noise when many optional checks exist
<code>-w / --web</code>	Open the checks page in a browser	Inspect individual job logs

merge

Flag	What it does	When to use it
<code>-m / --merge</code>	Merge commit	Preserve full history
<code>-s / --squash</code>	Squash into one commit	Clean up a noisy branch
<code>-r / --rebase</code>	Rebase onto base branch	Linear history without merge commits
<code>-d / --delete-branch</code>	Delete the local and remote branch after merge	Keep the repo tidy
<code>--auto</code>	Merge automatically once requirements are met	Merge queues and required checks
<code>--disable-auto</code>	Cancel a previously set auto-merge	Changed your mind
<code>--admin</code>	Bypass branch-protection rules	Emergency fix with administrator access
<code>--match-head-commit</code>	Only merge if the PR head matches a specific SHA	Guard against last-second pushes

review

Flag	What it does	When to use it
<code>-a / --approve</code>	Approve the PR	Standard approval
<code>-r / --request-changes</code>	Request changes	Block merge until addressed
<code>-c / --comment</code>	Comment without approving or blocking	Leaving notes
<code>-b / --body</code>	Inline review message	Required when requesting changes
<code>-F / --body-file</code>	Read the review body from a file	Long prepared reviews

edit

Flag	What it does	When to use it
<code>-t / --title</code>	Change the PR title	Correct a typo after creation
<code>-b / --body</code>	Replace the PR body	Update the description
<code>-B / --base</code>	Change the target branch	Retarget to a different branch
<code>--add-reviewer / --remove-reviewer</code>	Manage reviewers; <code>@copilot</code> for Copilot review	Re-request review after updates

<code>--add-assignee /</code> <code>--remove-assignee</code>	Manage assignees; <code>@me</code> and <code>@copilot</code> supported	Reassign during review
<code>--add-label /</code> <code>--remove-label</code>	Manage labels	Relabel without the web UI
<code>--add-project /</code> <code>--remove-project</code>	Manage project associations	Requires <code>project</code> scope; see <i>auth</i>

diff

Flag	What it does	When to use it
<code>--name-only</code>	List changed filenames only	Quick impact scan
<code>-e / --exclude</code>	Exclude files matching a glob pattern	Filter out generated files or lock files
<code>--patch</code>	Output in standard patch format	Piping into <code>git apply</code> or other tools
<code>--color</code>	<code>always</code> , <code>never</code> , or <code>auto</code>	Scripting without colour codes

Best practices

Use `--fill` for most PRs. Writing good commit messages pays off immediately:

`gh pr create --fill` turns them into the PR description with no extra effort.

Open drafts early. A draft PR (`--draft`) signals work in progress, triggers CI, and starts the conversation before the code is final. Convert it with `gh pr ready` when you are done.

Prefer `--auto` `--squash` (or `--rebase`) over manual timing. When a repository enforces required checks, `gh pr merge --auto` queues the merge and walks away. You do not need to poll and re-run the merge command after CI passes.

Delete the branch on merge. Pass `--delete-branch` to `gh pr merge` to clean up both the remote and local tracking branch in one step. A growing list of stale branches is one of the most common sources of confusion on shared repositories.

Use `--search` for non-trivial filtering. The `--author`, `--label`, and `--base` flags handle common cases, but `gh pr list --search` accepts the full GitHub search syntax:

```
gh pr list --search "review:required is:open" --state open
gh pr list --search "base:release/2.0 is:merged"
```

Let `gh pr checks --watch` gate automation. Instead of polling in a shell loop, let `gh` do it:

```
gh pr checks --watch && gh pr merge --squash --delete-branch
```

Exit code 0 means all checks passed; non-zero means at least one failed.

Request Copilot review when available. On supported plans, `--add-reviewer "@copilot"` requests an automated code review from GitHub Copilot — useful as a first pass before human reviewers see the change:

```
gh pr edit 123 --add-reviewer "@copilot"
```

Pitfalls & gotchas

gh pr create will offer to fork and push if your branch is local-only. If you are working on a fork already, double-check the prompt before confirming — you may end up with a fork of a fork. Use `--head OWNER:BRANCH` to be explicit about which remote branch is the head.

--fill is overridden by explicit --title / --body. If you specify both `--fill` and `--title`, the explicit title wins. This is intentional, but it surprises people who expect `--fill` to be a base that explicit flags extend rather than replace.

Auto-merge and merge queues interact differently. If the base branch has a merge queue enabled, `gh pr merge --auto` adds the PR to the queue rather than merging directly. To bypass the queue you need `--admin`, which requires administrator privileges.

gh pr checkout creates a local branch that tracks the remote. If the PR author later force-pushes their branch, your local branch will be behind. Use `--force` to reset it to the current state:

```
gh pr checkout 123 --force
```

gh pr review --request-changes without --body is rejected. GitHub requires a comment body when requesting changes. Always include `-b`:

```
gh pr review 123 --request-changes --body "Please add error handling."
```

edit --add-project requires the project scope. If you get a scope error, add it once and continue — no logout required:

```
gh auth refresh --scopes project
```

See *auth* for details on managing scopes.

close does not delete the branch by default. Unlike `merge`, closing a PR without `--delete-branch` leaves the remote branch intact. Add the flag explicitly when you want cleanup:

```
gh pr close 123 --delete-branch
```

--state merged in gh pr list is distinct from --state closed. Merged PRs are a subset of closed PRs. Using `--state closed` returns both merged and abandoned PRs. Use `--state merged` when you are searching for merged work specifically.

Worked examples

Opening a feature PR with reviewers, labels, and a milestone

You have finished a feature branch `feat/dark-mode`:

```
# Make sure the branch is pushed
git push -u origin feat/dark-mode

# Create the PR, requesting two reviewers and tagging the sprint milestone
gh pr create \
```

```
--fill \  
--reviewer octocat,myorg/frontend-team \  
--label "enhancement" \  
--milestone "Sprint 24"
```

<https://github.com/myorg/myapp/pull/87>

A colleague pushes follow-up commits, so CI re-runs. Watch from the terminal:

```
gh pr checks 87 --watch
```

```
All checks were successful  
0 failing, 0 pending, and 6 passing checks
```

Approve and merge when ready:

```
gh pr review 87 --approve  
gh pr merge 87 --squash --delete-branch
```

Reviewing a contributor's PR from a fork

A contributor opened PR 201 from their fork. Check out the branch to run it locally:

```
gh pr checkout 201
```

```
remote: Enumerating objects: 12, done.  
From https://github.com/myorg/myapp  
* [new ref]      refs/pull/201/head -> patch-1  
Switched to branch 'patch-1'
```

Read the diff for a quick overview before diving in:

```
gh pr diff 201 --name-only
```

```
src/auth/middleware.ts  
tests/auth/middleware.test.ts
```

Submit a structured review with a comment:

```
gh pr review 201 --comment \  
--body "Tested locally – the middleware behaves correctly. One style nit: prefer  
\`const\` over \`let\` on line 42."
```

If a nit is blocking, request changes:

```
gh pr review 201 --request-changes \  
--body "Please rename the variable to match the project convention (camelCase)."
```

After the contributor pushes a fix, re-request your review and approve:

```
gh pr edit 201 --add-reviewer "@me"  
gh pr review 201 --approve
```

Handling a stale PR branch

PR 150 was opened two weeks ago. The base branch has moved on and GitHub shows a conflict warning. Update the PR branch without leaving the terminal:

```
gh pr update-branch 150
```

If the team prefers a linear history:

```
gh pr update-branch 150 --rebase
```

Then re-check CI:

```
gh pr checks 150 --watch
```

Reverting a bad merge

PR 99 introduced a regression. Create a revert PR immediately:

```
gh pr revert 99 --title "Revert: fix regression from PR 99"
```

GitHub creates a new PR with an inverse diff of PR 99. Merge it through the normal review process.

Scripting: find all open PRs you need to review

Extract PR numbers, author logins, and titles using JSON output and jq:

```
gh pr list \  
  --search "review-requested:@me is:open" \  
  --json number,title,author \  
  --jq '.[ ] | "\(.number)\t\(.author.login)\t\(.title)'"
```

```
204 contributor-a Add dark mode  
198 contributor-b Fix login redirect
```

Recovery

PR created against the wrong base branch. Edit it without closing and reopening:

```
gh pr edit 123 --base correct-base-branch
```

Draft PR accidentally published. Convert back to draft:

```
gh pr ready 123 --undo
```

Merge failed due to conflicts. Update the branch, resolve locally, push, then retry:

```
gh pr update-branch 123          # or --rebase
# resolve conflicts in your editor, then:
git add .
git commit
git push
gh pr merge 123 --squash --delete-branch
```

Merged the wrong PR by mistake. Revert it:

```
gh pr revert 123
```

This opens a new PR with the inverse diff — review and merge that PR to undo the original change.

gh pr checkout **put you on a detached HEAD**. Run the command without `--detach`, or create a named branch from where you are:

```
gh pr checkout 123              # creates a named local branch by default
```

For broader undo patterns, see *Getting out of jams*.

See also

- *auth* — authenticate `gh` and switch between accounts before running PR commands; manage OAuth scopes required by `edit --add-project`.
- *repo* — `gh repo fork` and `gh repo clone` set up the remotes that `gh pr create` uses.
- *run* — inspect individual CI workflow runs surfaced by `gh pr checks`.
- *issue* — link issues to PRs via `Closes #N` in the PR body to auto-close them on merge.

issue

Create, view, list, and manage GitHub Issues without leaving the terminal.

Mental model

A GitHub Issue is a unit of tracked work: a bug report, a feature request, a task, or a question. Every issue lives in a repository, carries a number (e.g. #42), and moves between the `open` and `closed` states. Issues can be labelled, assigned to people, attached to milestones, added to Projects, pinned to the repository page, linked to related issues through blocking/blocked-by relationships, and organised into parent/sub-issue hierarchies.

`gh issue` is the command group that mirrors everything the GitHub web UI lets you do with issues. Each subcommand targets an issue either by its number or by its full URL — both forms work identically everywhere. When you run a subcommand inside a cloned repository `gh` infers the repository from the git remote; pass `-R OWNER/REPO` to target a different repository without changing directory.

Think of the subcommands in three tiers:

- **Discovery** — `list`, `status`, `view`: find and read issues.
- **Lifecycle** — `create`, `edit`, `close`, `reopen`, `delete`, `transfer`: create and move issues through their lifecycle.
- **Housekeeping** — `comment`, `develop`, `lock`, `unlock`, `pin`, `unpin`: add context or manage visibility.

Synopsis

```
gh issue list      [flags]
gh issue status    [flags]
gh issue view      {<number> | <url>} [flags]

gh issue create    [flags]
gh issue edit      {<numbers> | <urls>} [flags]
gh issue close     {<number> | <url>} [flags]
gh issue reopen    {<number> | <url>} [flags]
gh issue delete    {<number> | <url>} [flags]
gh issue transfer  {<number> | <url>} <destination-repo>

gh issue comment   {<number> | <url>} [flags]
gh issue develop   {<number> | <url>} [flags]
gh issue lock      {<number> | <url>} [flags]
gh issue unlock    {<number> | <url>}
gh issue pin       {<number> | <url>}
gh issue unpin     {<number> | <url>}
```

Everyday usage

Listing issues

Show open issues in the current repository (default: up to 30):

```
gh issue list
```

Filter to issues assigned to you:

```
gh issue list --assignee "@me"
```

Filter by label, state, or milestone:

```
gh issue list --label bug --label "help wanted"
gh issue list --state all
gh issue list --milestone "v2.0"
```

Use a full-text search query (same syntax as GitHub's search bar):

```
gh issue list --search "error no:assignee sort:created-asc"
```

Open the list in a browser instead of the terminal:

```
gh issue list --web
```

Checking your personal dashboard

Show issues in the current repository that are relevant to you — created by you, assigned to you, or mentioning you:

```
gh issue status
```

Viewing a single issue

Read an issue's title, body, and metadata:

```
gh issue view 42
```

Include comments:

```
gh issue view 42 --comments
```

Open in the browser:

```
gh issue view 42 --web
```

Creating an issue

Open an interactive prompt for title and body:

```
gh issue create
```

Supply everything from the command line (no prompts):

```
gh issue create --title "Button overlaps footer on mobile" \
  --body "Steps to reproduce: ..."
```

Apply labels, an assignee, and a milestone in one shot:

```
gh issue create --title "Cache miss under load" \  
               --label bug --label "help wanted" \  
               --assignee "@me" \  
               --milestone "v1.5"
```

Use a saved template:

```
gh issue create --template "Bug Report"
```

Open the browser form instead (useful for complex Markdown):

```
gh issue create --web
```

Create a sub-issue under a parent:

```
gh issue create --title "Write unit tests" --parent 100
```

Editing an issue

Change the title and replace all labels in one call:

```
gh issue edit 42 --title "Button overlaps footer (Safari only)" \  
                --add-label bug --remove-label "needs triage"
```

Edit multiple issues at once (same repository):

```
gh issue edit 42 43 44 --add-label "help wanted"
```

Set or clear a parent/sub-issue relationship:

```
gh issue edit 55 --parent 10  
gh issue edit 55 --remove-parent
```

Set blocking/blocked-by dependencies:

```
gh issue edit 42 --add-blocked-by 40 --add-blocking 50
```

Closing and reopening

Close with an optional reason and comment:

```
gh issue close 42  
gh issue close 42 --reason "not planned" --comment "Out of scope for v1."  
gh issue close 42 --duplicate-of 38
```

Reopen a closed issue:

```
gh issue reopen 42 --comment "Regression confirmed in v1.4.2."
```

Commenting

Add a comment non-interactively:

```
gh issue comment 42 --body "Reproduced on macOS 14.5."
```

Read from a file (or stdin):

```
gh issue comment 42 --body-file notes.md  
echo "LGTM" | gh issue comment 42 --body-file -
```

Edit your last comment:

```
gh issue comment 42 --edit-last
```

Delete your last comment (with confirmation bypass):

```
gh issue comment 42 --delete-last --yes
```

Creating a linked development branch

Create a branch tied to an issue and check it out immediately:

```
gh issue develop 42 --checkout
```

Specify a name and base branch:

```
gh issue develop 42 --name fix/button-overlap --base main
```

List branches already linked to an issue:

```
gh issue develop --list 42
```

Housekeeping

Lock a heated conversation:

```
gh issue lock 42 --reason too_heated
```

Unlock it later:

```
gh issue unlock 42
```

Pin an issue to the repository's issue list (GitHub supports up to three pinned issues):

```
gh issue pin 42
```

Unpin it:

```
gh issue unpin 42
```

Transfer to another repository:

```
gh issue transfer 42 myorg/other-repo
```

Permanently delete (irreversible — requires admin permission on the repository):

```
gh issue delete 42
gh issue delete 42 --yes # skip confirmation prompt
```

Key options

list

Flag	What it does	When to use it
<code>-s / --state</code>	Filter by <code>open</code> , <code>closed</code> , or <code>all</code>	Reviewing resolved issues
<code>-l / --label</code>	Filter by label (repeatable)	Narrowing to a category
<code>-a / --assignee</code>	Filter by assignee login; <code>@me</code> for yourself	Your own queue
<code>-A / --author</code>	Filter by the issue's creator	Auditing a contributor
<code>-m / --milestone</code>	Filter by milestone number or title	Sprint planning
<code>-S / --search</code>	Full GitHub search query	Complex ad-hoc queries
<code>--type</code>	Filter by issue type name	Typed issue workflows
<code>-L / --limit</code>	Maximum issues to fetch (default 30)	Larger result sets
<code>--json</code>	Output specific fields as JSON	Scripting / piping to <code>jq</code>
<code>-q / --jq</code>	Filter JSON output with a <code>jq</code> expression	Inline data extraction
<code>-w / --web</code>	Open results in the browser	Richer filtering UI

create

Flag	What it does	When to use it
<code>-t / --title</code>	Issue title	Non-interactive creation
<code>-b / --body</code>	Issue body text	Short bodies on the command line
<code>-F / --body-file</code>	Read body from a file or <code>-</code> for stdin	Long Markdown bodies
<code>-l / --label</code>	Apply a label (repeatable)	Categorise at creation time
<code>-a / --assignee</code>	Assign by login; <code>@me</code> to self-assign	Immediate ownership
<code>-m / --milestone</code>	Attach to a milestone	Sprint/release tracking
<code>-p / --project</code>	Add to a Project board by title	Project planning
<code>-T / --template</code>	Use a repository issue template	Consistent bug/feature reports
<code>--type</code>	Set the issue type	Typed issue workflows
<code>--parent</code>	Make this a sub-issue of another issue	Hierarchical task breakdown
<code>--blocked-by</code>	Mark as blocked by other issue numbers	Dependency tracking at creation
<code>--blocking</code>	Mark as blocking other issue numbers	Dependency tracking at creation

<code>-w / --web</code>	Open the browser form	Complex Markdown or file attachments
-------------------------	-----------------------	--------------------------------------

edit

Flag	What it does	When to use it
<code>-t / --title</code>	Replace the title	Correcting an unclear title
<code>-b / --body / -F / --body-file</code>	Replace the body	Updating the description
<code>--add-label / --remove-label</code>	Add or remove labels	Triaging in bulk
<code>--add-assignee / --remove-assignee</code>	Assign or unassign (supports @me)	Reassigning work
<code>--add-project / --remove-project</code>	Attach or detach from a Project	Moving between boards
<code>-m / --milestone</code>	Change milestone	Rescheduling
<code>--remove-milestone</code>	Remove the milestone association	Deferring indefinitely
<code>--parent / --remove-parent</code>	Set or clear the parent issue	Reorganising hierarchy
<code>--add-sub-issue / --remove-sub-issue</code>	Attach or detach sub-issues	Reorganising hierarchy
<code>--add-blocked-by / --remove-blocked-by</code>	Manage blocked-by relationships	Dependency graph updates
<code>--add-blocking / --remove-blocking</code>	Manage blocking relationships	Dependency graph updates
<code>--type / --remove-type</code>	Set or clear the issue type	Reclassification

close

Flag	What it does	When to use it
<code>-r / --reason</code>	<code>completed</code> , <code>not planned</code> , or <code>duplicate</code>	Accurate state reason
<code>--duplicate-of</code>	Reference the original issue number or URL	Deduplication
<code>-c / --comment</code>	Attach a closing comment	Leaving context for others

comment

Flag	What it does	When to use it
<code>-b / --body</code>	Comment text	Quick inline comments
<code>-F / --body-file</code>	Read comment from file or stdin	Longer or templated text
<code>--edit-last</code>	Edit your most recent comment	Correcting a typo
<code>--delete-last</code>	Delete your most recent comment	Retracting a comment
<code>--create-if-none</code>	Create a new comment when used with <code>--edit-last</code>	Idempotent scripting
<code>--yes</code>	Skip the delete confirmation	Scripting <code>--delete-last</code>
<code>-w / --web</code>	Open the browser comment form	Rich editing

develop

Flag	What it does	When to use it
<code>-n / --name</code>	Branch name to create	Custom branch naming
<code>-b / --base</code>	Remote branch to base the new branch on	Feature branches off non-default base
<code>-c / --checkout</code>	Check out the branch after creating it	Immediately start coding
<code>-l / --list</code>	List branches already linked to the issue	Auditing
<code>--branch-repo</code>	Repository where the branch should be created	Cross-repo development

Best practices

Write the title first, then the body. Use `--title` and `--body` (or `--body-file`) for every scripted or automated creation. Relying on the interactive prompt works fine ad hoc, but automation needs deterministic input.

Use `--body-file` - to pipe Markdown from a template generator. A script that assembles a report from JSON data can write the Markdown to stdout and pipe it straight into the issue body:

```
./generate-report.sh | gh issue create \  
  --title "Nightly report $(date +%Y-%m-%d)" \  
  --body-file -
```

Prefer `--add-label` / `--remove-label` over replacing all labels. The `edit` subcommand adds and removes labels individually, so you cannot accidentally wipe labels that were set by automation or other team members.

Close with a reason. GitHub records whether an issue was `completed`, `not planned`, or `duplicate`. This context is permanently visible in the issue timeline and in `list` output, helping future contributors understand why something was closed. Make it a habit:

```
gh issue close 42 --reason completed  
gh issue close 99 --reason "not planned" --comment "Deprioritised; reopen if  
needed."
```

Use `develop --checkout` to start feature work. Creating a linked branch through `gh issue develop` registers the branch–issue relationship in GitHub’s UI. This makes it easy for reviewers to navigate from a pull request back to the issue and understand the context.

Add `--json` and `--jq` to scripts instead of grepping terminal output. The JSON field list for `list` and `view` is stable across `gh` versions; terminal formatting is not:

```
# Get the number of every open bug  
gh issue list --label bug --state open --json number --jq '.[].number'
```

Batch edits save API round-trips. When you need to apply the same label or assignee to several issues, pass multiple numbers in one call:

```
gh issue edit 10 11 12 13 --add-label "sprint-3"
```

Pitfalls & gotchas

gh issue list only returns open issues by default. This surprises everyone at least once. Pass `--state all` or `--state closed` when you know the issue was closed:

```
gh issue list --state closed --search "cache bug"
```

delete is permanent and unrecoverable. GitHub does not soft-delete issues: once gone, the issue number is retired and the content is gone. Prefer `close --reason "not planned"` unless you genuinely need to erase the issue. The `--yes` flag skips the confirmation prompt entirely — do not use it in ad-hoc interactive sessions.

develop creates the branch on the remote, not locally (unless you pass `--checkout`). Running `gh issue develop 42` without `--checkout` creates the branch on GitHub but leaves your local repository unchanged. You still need to `git fetch` and check out the branch manually if you omit `--checkout`.

Adding an issue to a Project requires the project scope. If `create` or `edit` fails with a scope error when you pass `--project`, grant the scope and retry:

```
gh auth refresh --scopes project
gh issue create --project "Roadmap" --title "..."
```

--search uses GitHub's issue search syntax, not shell glob syntax. Quotes inside the query refer to GitHub search qualifiers, not shell quoting. Always wrap the whole query in double quotes and escape inner quotes if needed:

```
gh issue list --search "label:bug label:\"help wanted\""
```

status shows issues relevant to you in the current repository (or the one specified with `-R`), not across all repositories. If you want to filter further, use `list` with `--assignee "@me"` or `--author "@me"`.

lock accepts only four reason values. Anything outside `off_topic`, `resolved`, `spam`, `too_heated` causes an error.

Worked examples

Triaging a bug report end-to-end

A user files a vague issue. You take ownership, label it, ask for reproduction steps, and eventually resolve it.

```
# Assign yourself and apply triage labels
gh issue edit 88 --add-assignee "@me" \
    --add-label bug \
    --add-label "needs info"

# Ask the reporter for more detail
gh issue comment 88 --body \
```

```

    "Thanks for the report! Could you share the exact error message and OS version?"

# Once the reporter replies, remove needs-info and add reproduced
gh issue edit 88 --remove-label "needs info" --add-label "reproduced"

# Create a linked development branch and start working
gh issue develop 88 --checkout

# After the fix is merged, close as completed with a note
gh issue close 88 --reason completed \
    --comment "Fixed in #91. Will be available in the next release."

```

Bulk labelling issues for a sprint

At sprint planning, the team decides which open issues belong to sprint 4.

```

# List candidates interactively in the browser first
gh issue list --state open --label "backlog" --web

# Apply the sprint label to the chosen issues
gh issue edit 20 21 25 30 33 --add-label "sprint-4" --remove-label "backlog"

# Confirm
gh issue list --label "sprint-4" --json number,title \
    --jq '.[] | "#\(.number) \(.title)"'

```

```

#20 Improve cache eviction algorithm
#21 Add dark mode toggle
#25 Fix mobile layout on issue list page
#30 Write migration guide for v2 API
#33 Reduce cold-start time below 200 ms

```

Scripting a nightly issue report

A CI job collects overnight error counts and files a dated issue with the results.

```

#!/usr/bin/env bash
# nightly-report.sh
REPORT=$(./collect-errors.sh) # produces Markdown

gh issue create \
    --repo myorg/myapp \
    --title "Nightly error report $(date +%Y-%m-%d)" \
    --body "$REPORT" \
    --label "automated" \
    --assignee ops-oncall

```

Because the body comes from a variable rather than an interactive prompt, the script runs unattended in CI.

Closing duplicates in bulk

After a popular library releases a breaking change, dozens of duplicate issues appear. Close them all as duplicates of the canonical issue (#200):

```
# Find all the duplicates by searching for a shared error string
gh issue list --search "\"cannot read property of undefined\"" \
--state open --json number --jq '[][.number]' \
| xargs -I{} gh issue close {} \
--duplicate-of 200 \
--comment "Duplicate of #200. Please subscribe there for updates."
```

Recovery

Accidentally closed an issue? Reopen it and leave a note:

```
gh issue reopen 42 --comment "Closed by mistake."
```

Accidentally deleted an issue? Deletion is permanent. There is no recovery path. Contact GitHub Support only if the deletion was very recent — they may be able to restore it in exceptional circumstances, but this is not guaranteed.

Wrong labels applied in bulk? Reverse the edit with the inverse flags:

```
gh issue edit 20 21 25 --remove-label "sprint-4" --add-label "backlog"
```

Branch created in the wrong repo? Delete the branch on GitHub and re-run

```
gh issue develop with --branch-repo pointing at the correct repository.
```

See *Getting out of jams* for broader guidance on recovering from accidental operations.

See also

- *auth* — grant the `project` scope when `--project` flags fail.
- *pr* — `gh pr create` links a pull request to an issue; use `develop` to create the branch first.
- *label* — manage the label definitions referenced by `--label` flags.
- *project* — manage Project boards that issues are added to via `--project`.
- *repo* — `gh repo view` shows pinned issues and overall repository context.

label

Create, edit, delete, list, and clone issue and pull-request labels in a GitHub repository.

Mental model

Labels in GitHub are repository-scoped tags attached to issues and pull requests to signal type, priority, status, or any other dimension your team cares about. They have three properties: a **name** (the slug that appears in search filters), a **color** (a six-character hex value that drives the colored chip in the UI), and an optional **description** (a short tooltip).

`gh label` manipulates those labels directly through the GitHub API without opening a browser. Because labels are per-repository, every subcommand operates on a specific repo — the current repository by default, or any other via `--repo`. The `clone` subcommand is the one exception that reads from a *source* repository and writes to a destination.

A common team pattern is to designate one “canonical” repository that holds your full, curated label set and use `gh label clone` to stamp that set onto every new repo. The `--force` flag on both `create` and `clone` makes idempotent re-runs safe.

Synopsis

```
gh label list    [--search <query>] [--sort <field>] [--order <asc|desc>]
                  [--limit <n>] [--json <fields>] [--web] [-R <repo>]

gh label create <name> [--color <hex>] [--description <text>] [--force]
                  [-R <repo>]

gh label edit    <name> [--name <new-name>] [--color <hex>]
                  [--description <text>] [-R <repo>]

gh label delete <name> [--yes] [-R <repo>]

gh label clone   <source-repository> [--force] [-R <repo>]
```

Everyday usage

List all labels in the current repository:

```
gh label list
```

List labels sorted alphabetically:

```
gh label list --sort name
```

Search for labels related to bugs:

```
gh label list --search bug
```

Create a new label with a color and description:

```
gh label create enhancement \
  --color 84b6eb \
  --description "New feature or request"
```

Edit an existing label's color:

```
gh label edit bug --color FF0000
```

Rename a label and update its description at the same time:

```
gh label edit wontfix \  
  --name "won't fix" \  
  --description "Intentionally not addressed"
```

Delete a label (prompts for confirmation):

```
gh label delete obsolete-label
```

Delete without a confirmation prompt (useful in scripts):

```
gh label delete obsolete-label --yes
```

Copy all labels from a template repository into the current repo:

```
gh label clone my-org/label-template
```

Overwrite any conflicting labels in the destination:

```
gh label clone my-org/label-template --force
```

Key options

list

Flag	What it does	When to use it
<code>-S / --search</code>	Filter by name or description	Find a specific label quickly
<code>--sort</code>	Sort by <code>created</code> or <code>name</code> (default: <code>created</code>)	Browse labels in alphabetical order
<code>--order</code>	Sort direction: <code>asc</code> or <code>desc</code> (default: <code>asc</code>)	Show newest labels first with <code>--order desc</code>
<code>-L / --limit</code>	Max labels to fetch (default: 30)	Repos with large label sets
<code>--json</code>	Output JSON with specified fields	Scripting; available fields: <code>color</code> , <code>createdAt</code> , <code>description</code> , <code>id</code> , <code>isDefault</code> , <code>name</code> , <code>updatedAt</code> , <code>url</code>
<code>-q / --jq</code>	Filter JSON output with a jq expression	Extract just the names or colors
<code>-t / --template</code>	Format JSON with a Go template	Custom tabular output
<code>-w / --web</code>	Open the labels page in the browser	Visual browsing
<code>-R / --repo</code>	Target a different repository	Any <code>label</code> subcommand

create

Flag	What it does	When to use it
<code>-c / --color</code>	Set the label color as a 6-char hex value	Give the label a specific color; a random color is chosen if omitted
<code>-d / --description</code>	Short description shown as a tooltip	Document the label's intended use
<code>-f / --force</code>	Update color and description if the label already exists	Idempotent provisioning scripts

edit

Flag	What it does	When to use it
<code>-n / --name</code>	Rename the label	Standardizing naming across repos
<code>-c / --color</code>	Change the label color	Rebranding or accessibility improvements
<code>-d / --description</code>	Change the description	Clarifying a label's scope

delete

Flag	What it does	When to use it
<code>--yes</code>	Skip the interactive confirmation prompt	Scripted cleanup

clone

Flag	What it does	When to use it
<code>-f / --force</code>	Overwrite labels that already exist in the destination	Re-syncing a repo to the canonical template

Best practices

Maintain a canonical label repository. Keep one repository (e.g. `my-org/labels`) whose sole purpose is to hold the agreed label set. Seed new repositories from it with `gh label clone`. This guarantees consistent names and colors across every repo, which matters for cross-repo issue searches and automation that filters by label.

Use `--force` when re-syncing, not just when first cloning. On a first clone the destination is empty, so `--force` is harmless but unnecessary. Reserve it for explicit re-sync runs where you want the canonical set to win over any local drift.

Always supply a color. GitHub assigns a random color when you omit `--color`, producing an inconsistent palette. Choose a palette upfront and encode it in your `create` commands or canonical repository.

Use `--json` and `--jq` for scripting. The text output of `gh label list` is for humans. When a script needs label names, pipe JSON instead:

```
gh label list --json name --jq '.[].name'
```

Use `--force on create` for idempotent provisioning. Automation scripts that run on every deploy can call `gh label create --force` and never need to check whether the label already exists first.

Prefer `edit` over delete-and-recreate when renaming. GitHub preserves the association between a label and its issues when you rename via the API. Deleting and recreating the label severs that link; `gh label edit --name` keeps it.

Pitfalls & gotchas

`--search` disables `--sort` and `--order`. When you pass `--search`, results are ranked by best match. The `--sort` and `--order` flags are silently ignored in that mode.

Color must be exactly 6 hex characters with no leading `#`. Both of these will fail:

```
gh label create triage --color "#FF6600"    # WRONG – do not include the hash
gh label create triage --color F60          # WRONG – must be 6 characters
```

Correct form:

```
gh label create triage --color FF6600
```

`clone` does not delete labels absent from the source. Labels present in the destination but missing from the source are left untouched. If you want the destination to be an exact mirror you must delete the extra labels manually with `gh label delete`.

`clone` skips existing labels unless you pass `--force`. If the destination already has a label with the same name but a different color or description, it is silently skipped. Add `--force` to overwrite.

`--limit` defaults to 30. A repository with more than 30 labels will appear incomplete in the default `list` output. Pass a higher value when you need the full set:

```
gh label list --limit 200
```

Deleting a label does not remove it from closed issues' history. The label is stripped from the issue's current state but the deletion event is recorded in the issue timeline.

Worked examples

Bootstrapping a new repository from a shared label set

A team keeps their canonical labels in `my-org/standards`. When they create a new repository they run:

```
# Create the new repo
gh repo create my-org/new-service --private

# Copy all labels from the canonical repo into the new one
gh label clone my-org/standards --repo my-org/new-service
```

```
✓ Cloned 18 labels to my-org/new-service
```

Six months later the canonical set is updated. Re-sync and overwrite any drifted labels:

```
gh label clone my-org/standards --repo my-org/new-service --force
```

Auditing a label set with JSON output

List every label as JSON and print name, color, and description side by side:

```
gh label list --limit 100 --json name,color,description \
--jq '.[ ] | "\(.color) \(.name) - \(.description)'"
```

```
e4e669 duplicate - This issue or pull request already exists
d73a4a bug       - Something isn't working
0075ca feature   - New feature or request
```

Find labels that have no description (empty string):

```
gh label list --limit 100 --json name,description \
--jq '.[ ] | select(.description == "") | .name'
```

Delete each undescribed label after reviewing the list:

```
gh label delete "undescribed-label" --yes
```

Renaming a label without losing issue associations

Use `edit` rather than delete-and-recreate to preserve the link between a label and its issues:

```
# Old name: "in-progress" - team wants "status: in progress"
gh label edit "in-progress" \
--name "status: in progress" \
--color 0E8A16 \
--description "Work is actively underway"
```

All issues that carried `in-progress` now show `status: in progress` with no manual re-tagging required.

Recovery

If you delete a label by mistake, recreate it with the original name, color, and description, then re-apply it to affected issues:

```
# Recreate the deleted label
gh label create accidentally-deleted \
--color CC0000 \
--description "Original description here"

# Re-apply to a specific issue
gh issue edit 42 --add-label "accidentally-deleted"
```

If a `clone --force` run overwrote labels you wanted to keep, the only path back is to `edit` each label to its previous color and description. Check your team's canonical source or the GitHub UI's label history for the original values.

For broader repository mistakes, see *Getting out of jams*.

See also

- *issue* — attach and remove labels on issues with `gh issue edit --add-label` and `gh issue edit --remove-label`.
- *pr* — the same `--add-label` / `--remove-label` flags apply to pull requests via `gh pr edit`.
- *repo* — `gh repo create` creates the repository that labels live in.

project

Create, view, and manage GitHub Projects (v2) boards and their items from the command line.

Mental model

GitHub Projects v2 is a spreadsheet-like planning tool that lives at the organization or user level, not inside a single repository. A project holds **items** (issues, pull requests, and draft issues) that can come from any repository. Each item has **fields** — built-in ones like Title, Status, and Assignees, plus custom fields you define (text, number, date, or single-select).

The `gh project` command group is a full CRUD interface for that data model:

```
owner (org or user)
├─ project (numbered 1, 2, 3...)
│   ├─ fields (Title, Status, custom...)
│   └─ items (issues, PRs, draft issues)
│       └─ field values (per-item metadata)
```

Because projects exist outside any single repository, most subcommands require `--owner` (a GitHub login or organization name). Use `@me` to target your own account without typing your username.

The `project` scope is not granted by default. Before running any `gh project` command, confirm your token has it — and add it if not:

```
gh auth status          # look for "project" in the listed scopes
gh auth refresh -s project # add the scope
```

Synopsis

```
gh project list          [--owner <login>] [--closed] [-L limit] [--format json]
[-w]
gh project view          [<number>] [--owner <login>] [--format json] [-w]
gh project create        --owner <login> --title <title> [--format json]
gh project edit          [<number>] --owner <login> [--title t] [--description d]
                        [--readme r] [--visibility PUBLIC|PRIVATE]
gh project close         [<number>] --owner <login> [--undo]
gh project delete        [<number>] --owner <login>
gh project copy          [<number>] --source-owner <login> --target-owner <login>
                        --title <title> [--drafts]

gh project field-list    [<number>] --owner <login> [-L limit]
gh project field-create  [<number>] --owner <login> --name <name>
                        --data-type TEXT|SINGLE_SELECT|DATE|NUMBER
                        [--single-select-options a,b,c]
gh project field-delete  --id <field-id>

gh project item-list     [<number>] --owner <login> [--query <filter>] [-L limit]
gh project item-add      [<number>] --owner <login> --url <issue-or-pr-url>
gh project item-create   [<number>] --owner <login> --title <title> [--body
<body>]
gh project item-edit     --id <item-id> [--field-id <id>] [--project-id <id>]
                        [--text t] [--number n] [--date YYYY-MM-DD]
```

```

                                [--single-select-option-id id] [--iteration-id id]
                                [--title t] [--body b] [--clear]
gh project item-archive    [<number>] --owner <login> --id <item-id> [--undo]
gh project item-delete    [<number>] --owner <login> --id <item-id>

gh project link            [<number>] --owner <login> [-R repo] [-T team]
gh project unlink         [<number>] --owner <login> [-R repo] [-T team]
gh project mark-template  [<number>] --owner <org> [--undo]

```

Everyday usage

Browsing projects

List all projects you own:

```
gh project list
```

Include closed projects:

```
gh project list --closed
```

List an organization's projects:

```
gh project list --owner my-org
```

Open a project in the browser instead of printing it:

```
gh project view 3 --owner my-org --web
```

Creating and editing

Create a new project:

```
gh project create --owner "@me" --title "Q3 Roadmap"
```

Rename a project and set it to public:

```
gh project edit 3 --owner "@me" --title "Q3 Roadmap (public)" --visibility PUBLIC
```

Add a description and a readme:

```
gh project edit 3 --owner "@me" \
  --description "Tracks Q3 deliverables" \
  --readme "## How we work\nUpdate status every Friday."
```

Closing and deleting

Close a finished project (it stays visible but is marked closed):

```
gh project close 3 --owner "@me"
```

Reopen it later:

```
gh project close 3 --owner "@me" --undo
```

Delete a project permanently (no confirmation prompt — see Pitfalls):

```
gh project delete 3 --owner "@me"
```

Copying a project

Duplicate a project's structure to a new owner (useful for team templates):

```
gh project copy 3 --source-owner monalisa --target-owner my-org --title "Fork of Roadmap"
```

Include draft issues in the copy:

```
gh project copy 3 --source-owner monalisa --target-owner my-org \
  --title "Fork of Roadmap" --drafts
```

Managing fields

List the fields defined in project 1:

```
gh project field-list 1 --owner "@me"
```

Add a plain-text field:

```
gh project field-create 1 --owner "@me" --name "Notes" --data-type TEXT
```

Add a single-select field (like a status column):

```
gh project field-create 1 --owner "@me" --name "Priority" \
  --data-type SINGLE_SELECT \
  --single-select-options "P0,P1,P2,P3"
```

Delete a field by its ID (obtain the ID from `field-list --format json`):

```
gh project field-delete --id PVTf_abc123
```

Managing items

List items in project 1:

```
gh project item-list 1 --owner "@me"
```

Filter items using the Projects query syntax (github.com and GHES 3.20+):

```
gh project item-list 1 --owner "@me" --query "assignee:@me is:issue is:open"
gh project item-list 1 --owner "@me" --query "label:bug -status:Done"
```

Add an existing issue or pull request to a project:

```
gh project item-add 1 --owner "@me" \  
--url https://github.com/myorg/myrepo/issues/42
```

Create a draft issue (a standalone card, not linked to any repository):

```
gh project item-create 1 --owner "@me" --title "Research caching layer" \  
--body "Investigate Redis vs Memcached for the API tier."
```

Edit a field value on an item (all three IDs are required for non-draft items):

```
# Set a text field  
gh project item-edit \  
--id PVTI_abc123 \  
--field-id PVTF_def456 \  
--project-id PVT_ghi789 \  
--text "Blocked on design review"
```

Clear a field value:

```
gh project item-edit \  
--id PVTI_abc123 \  
--field-id PVTF_def456 \  
--project-id PVT_ghi789 \  
--clear
```

Archive an item (hides it from the default view but keeps it in the project):

```
gh project item-archive 1 --owner "@me" --id PVTI_abc123
```

Unarchive:

```
gh project item-archive 1 --owner "@me" --id PVTI_abc123 --undo
```

Delete an item permanently:

```
gh project item-delete 1 --owner "@me" --id PVTI_abc123
```

Linking to repositories and teams

Link project 1 to the current repository (run from inside the repo):

```
gh project link 1
```

Link to a specific repository:

```
gh project link 1 --owner monalisa --repo my_repo
```

Link to a team (organization projects only):

```
gh project link 1 --owner my-org --team frontend-team
```

Unlink:

```
gh project unlink 1 --owner monalisa --repo my_repo
```

Marking a project as a template

Mark an organization project as a template so teams can copy it from the web UI:

```
gh project mark-template 1 --owner "my-org"
```

Remove the template mark:

```
gh project mark-template 1 --owner "my-org" --undo
```

Note: `mark-template` requires an org login, not a personal account.

Key options

Shared across most subcommands

Flag	What it does	When to use it
<code>--owner <login></code>	Target a specific user or org	Required for most subcommands
<code>--format json</code>	Emit JSON instead of a table	Scripting; piping to <code>jq</code>
<code>-q / --jq <expr></code>	Filter JSON output with a jq expression	Extract a single field inline
<code>-t / --template <tmpl></code>	Format JSON with a Go template	Custom tabular output
<code>-L / --limit <n></code>	Maximum records to fetch (default 30)	Large projects with many items/fields
<code>-w / --web</code>	Open in browser	Quick visual inspection

list

Flag	What it does	When to use it
<code>--closed</code>	Include closed projects	Auditing historical projects

edit

Flag	What it does	When to use it
<code>--title <string></code>	Rename the project	After a rename decision
<code>-d / --description <string></code>	Short description shown on the project card	Onboarding new contributors
<code>--readme <string></code>	Long-form README for the project	Documenting workflow conventions
<code>--visibility PUBLIC PRIVATE</code>	Control who can see the project	Sharing with external stakeholders

copy

Flag	What it does	When to use it
<code>--source-owner</code>	Owner of the project being copied	Required
<code>--target-owner</code>	Owner of the new copy	Required
<code>--drafts</code>	Also copy draft issue items	When drafts carry planning value

field-create

Flag	What it does	When to use it
<code>--data-type</code>	TEXT , SINGLE_SELECT , DATE , or NUMBER	Required; determines value type
<code>--single-select-options</code>	Comma-separated option names	Required when <code>--data-type</code> is SINGLE_SELECT

item-list

Flag	What it does	When to use it
<code>--query <filter></code>	Projects filter syntax (assignee, label, status, etc.)	Narrowing large backlogs

item-edit

Flag	What it does	When to use it
<code>--id</code>	Node ID of the item	Required
<code>--field-id</code>	Node ID of the field	Required for non-draft items
<code>--project-id</code>	Node ID of the project	Required for non-draft items
<code>--text</code>	New text value	TEXT fields
<code>--number</code>	New numeric value	NUMBER fields
<code>--date</code>	New date value (YYYY-MM-DD)	DATE fields
<code>--single-select-option-id</code>	Option node ID	SINGLE_SELECT fields
<code>--iteration-id</code>	Iteration node ID	Iteration fields
<code>--title / --body</code>	Update draft issue metadata	Draft items only
<code>--clear</code>	Remove the field value	Clearing stale data

link / unlink

Flag	What it does	When to use it
<code>-R / --repo <name></code>	Repository to link/unlink	Repository association
<code>-T / --team <name></code>	Team to link/unlink (org projects)	Team-level visibility

Best practices

Capture IDs with `--format json` and `jq`. Most write operations (`item-edit` , `field-delete` , `item-archive` , `item-delete`) require node IDs that are never shown in the default table view. Retrieve them once and store them in shell variables:

```
PROJECT_ID=$(gh project list --owner "@me" --format json \
| jq -r '.projects[] | select(.title=="Q3 Roadmap") | .id')

ITEM_ID=$(gh project item-list 1 --owner "@me" --format json \
| jq -r '.items[] | select(.title=="Research caching layer") | .id')
```

Use `--query` to filter before paginating. `item-list` without a query fetches up to `--limit` items in project order. For projects with hundreds of items, a targeted query is faster and avoids hitting the default limit:

```
gh project item-list 1 --owner "@me" \
--query "assignee:@me status:\\"In Progress\\""
```

Keep the project scope narrow. The `project` OAuth scope grants read/write access to all your projects. Do not add it to tokens that only need repository access.

Prefer `item-archive` over `item-delete` for completed work. Archived items remain searchable and restore with `--undo`. Deleted items are gone immediately with no undo path from the CLI.

Use `copy` to seed new projects from a standard template. For teams that run the same type of project repeatedly (sprints, quarterly roadmaps), build a canonical project, mark it as a template with `mark-template`, then copy it at the start of each cycle.

Script bulk operations with a JSON pipeline. The combination of `--format json`, `jq`, and a shell loop handles bulk updates cleanly — for example, moving all “Backlog” items to a new single-select value after renaming a status option.

Pitfalls & gotchas

`gh project delete` has no confirmation prompt. Unlike the web UI, the CLI deletes immediately upon execution. There is no undo. Double-check the project number and owner before running it.

Node IDs vs. project numbers. The `<number>` argument in most subcommands is the human-readable project number (1, 2, 3...). However, `--id`, `--field-id`, and `--project-id` flags all require GraphQL node IDs (strings starting with `PVT_`, `PVTF_`, `PVTI_`). These are not interchangeable — passing a project number where an ID is expected silently fails or targets the wrong object. Retrieve node IDs via `--format json`.

`item-edit` requires three IDs for non-draft items. You must supply `--id` (item), `--field-id` (field), and `--project-id` (project) together. Omitting any one produces an error. For draft issues, only `--id` is needed for title and body edits.

`mark-template` only works on organization-owned projects. Attempting to mark a personal project as a template returns an error. The `--owner` value must be an organization login.

The `project` scope is not included in the default `gh auth login` scopes. Every `gh project` command fails with an authorization error until you add the scope. Run `gh auth refresh -s project` once per account.

`--single-select-options` must be supplied at creation time. You cannot add options to an existing `SINGLE_SELECT` field via `field-create`. Plan your option set up front; modifying it

later requires deleting and recreating the field (losing stored values) or editing through the web UI.

copy does not preserve field values on items. It copies the project structure (fields, views) and optionally the items themselves, but field values on those items are not transferred. Use `copy` for structural templates, not snapshots.

Worked examples

Setting up a sprint board from scratch

Create a project and note the project number from the URL it prints:

```
gh project create --owner "@me" --title "Sprint 42" --format json \
| jq -r '.url'
```

```
https://github.com/users/monalisa/projects/7
```

Add a SINGLE_SELECT Priority field to project 7:

```
gh project field-create 7 --owner "@me" \
--name "Priority" \
--data-type SINGLE_SELECT \
--single-select-options "P0,P1,P2,P3"
```

List the fields to capture their node IDs:

```
gh project field-list 7 --owner "@me" --format json \
| jq '.fields[] | {name,id}'
```

```
{"name": "Title",    "id": "PVTf_aaa"}
{"name": "Status",  "id": "PVTf_bbb"}
{"name": "Priority", "id": "PVTf_ccc"}
```

Add an existing issue from a repository:

```
gh project item-add 7 --owner "@me" \
--url https://github.com/monalisa/api/issues/99
```

Get the new item's node ID and the P1 option ID:

```
ITEM_ID=$(gh project item-list 7 --owner "@me" --format json \
| jq -r '.items[] | select(.content.number==99) | .id')

OPTION_ID=$(gh project field-list 7 --owner "@me" --format json \
| jq -r '.fields[] | select(.name=="Priority") | .options[] |
select(.name=="P1") | .id')

PROJECT_ID=$(gh project list --owner "@me" --format json \
| jq -r '.projects[] | select(.number==7) | .id')
```

Set the item's Priority to P1:

```
gh project item-edit \
  --id "$ITEM_ID" \
  --field-id PVTf_ccc \
  --project-id "$PROJECT_ID" \
  --single-select-option-id "$OPTION_ID"
```

Archiving all Done items at the end of a sprint

Retrieve the IDs of every item whose Status is “Done” and archive each:

```
gh project item-list 7 --owner "@me" \
  --query "status:Done" \
  --limit 100 \
  --format json \
  | jq -r '.items[].id' \
  | while read -r item_id; do
    gh project item-archive 7 --owner "@me" --id "$item_id"
    echo "Archived $item_id"
  done
```

To restore them, swap `item-archive` for `item-archive --undo` in the same loop, keeping a list of the IDs you archived (archived items no longer appear in the default `item-list` output).

Creating a reusable project template for an organization

Build the canonical project structure, add all desired fields and views, then mark it as a template:

```
gh project mark-template 5 --owner "my-org"
```

Team members can copy it from the GitHub web UI, or via the CLI:

```
gh project copy 5 --source-owner my-org --target-owner my-org \
  --title "Sprint $(date +%Y-%W)"
```

Recovery

Accidentally deleted a project. There is no CLI recovery path. Before deleting any project, export its items to JSON as a safety net:

```
gh project item-list <number> --owner <owner> --limit 1000 --format json \
  > project-backup.json
```

If the organization has admin access, check with a GitHub organization owner whether a server-side recovery option is available.

Accidentally deleted an item. Item deletion is permanent from the CLI. If the item was an issue or pull request it still exists in its repository — add it back with `item-add --url`. Draft issues that were deleted cannot be recovered.

Wrong field value set on an item. Run `item-edit --clear` to remove the value, then re-run `item-edit` with the correct value.

Token missing `project` scope. Run `gh auth refresh -s project` and complete the browser flow. See *auth* for detailed scope management.

See also

- *auth* — adding the `project` scope to your token with `gh auth refresh`.
- *issue* — `gh issue` for creating and managing the issues you then add to projects.
- *pr* — `gh pr` for pull requests tracked inside a project board.
- *repo* — `gh project link` connects projects to repositories managed with `gh repo`.

run

List, view, watch, rerun, cancel, delete, and download artifacts from GitHub Actions workflow runs.

Mental model

Every time a GitHub Actions workflow triggers — on a push, a pull-request event, a schedule, or a manual dispatch — GitHub creates a **workflow run**. A run belongs to a specific workflow file and a specific commit. It contains one or more **jobs**, and each job contains **steps**.

`gh run` is a command group that gives you a terminal interface to those runs. You can:

- **Observe:** list recent runs, watch a run stream its progress live, or view a run's full log after the fact.
- **Act:** rerun a failed run (or just its failed jobs), cancel a run that is still in progress, or force-cancel one that is stuck.
- **Collect:** download the build artifacts a run produced.
- **Clean up:** delete run records you no longer need.

Runs are identified by a numeric run ID. Most subcommands accept the ID as a positional argument; omit it and `gh` presents an interactive selector.

`gh run` works within the repository of your current directory by default. Pass `-R owner/repo` to target any repository on GitHub.

Synopsis

```
gh run list      [flags]
gh run view     [<run-id>] [flags]
gh run watch    <run-id> [flags]
gh run rerun    [<run-id>] [flags]
gh run cancel  [<run-id>] [flags]
gh run delete   [<run-id>] [flags]
gh run download [<run-id>] [flags]
```

Everyday usage

List the most recent runs for the current repository:

```
gh run list
```

Watch a run stream live as it executes (prompts for a run if none is given):

```
gh run watch
```

View a summary of a finished run:

```
gh run view 12345678
```

View the full log — useful after a failure:

```
gh run view 12345678 --log-failed
```

Rerun only the failed jobs in a run:

```
gh run rerun 12345678 --failed
```

Cancel a run that is still in progress:

```
gh run cancel 12345678
```

Download all artifacts produced by a run:

```
gh run download 12345678
```

list

`gh run list` (alias `gh run ls`) shows the most recent runs, newest first. Filter by branch, event, status, user, or workflow to narrow the list.

```
# Show runs on the main branch
gh run list --branch main

# Show only failed runs
gh run list --status failure

# Show runs triggered by a specific commit
gh run list --commit abc1234

# Show runs for one workflow file
gh run list --workflow ci.yml

# Output JSON for scripting
gh run list --json name,status,conclusion,url
```

view

`gh run view` shows a run's status, jobs, and optional log output.

```
# Interactively select a run to view
gh run view

# View a specific run, expanded to show each job step
gh run view 12345678 --verbose

# View only failed step logs
gh run view 12345678 --log-failed

# View the complete raw log
gh run view 12345678 --log

# View a specific job by its database ID
gh run view --job 987654321

# Open the run in the browser
gh run view 12345678 --web
```

```
# Exit non-zero if the run failed (useful in scripts)
gh run view 12345678 --exit-status
```

watch

`gh run watch` polls a running workflow and prints job and step progress until the run completes. By default it refreshes every 3 seconds and shows all steps; `--compact` narrows output to only relevant or failed steps.

```
# Watch the most recently triggered run (interactive selector)
gh run watch

# Watch a specific run in compact mode
gh run watch 12345678 --compact

# Notify when done (exit-status propagates failure to the shell)
gh run watch 12345678 --exit-status && terminal-notifier -message "Build passed"
```

Note: `gh run watch` does not support fine-grained personal access tokens because GitHub's API does not expose a `checks:read` permission for them.

rerun

`gh run rerun` triggers a new attempt of an existing run. You can rerun the whole run, only its failed jobs (plus their dependencies), or a single job.

The job ID accepted by `--job` is the **database ID**, not the integer visible in the browser URL. Retrieve it with:

```
gh run view 12345678 --json jobs --jq '.jobs[] | {name, databaseId}'
```

```
# Rerun the entire run
gh run rerun 12345678

# Rerun only failed jobs
gh run rerun 12345678 --failed

# Rerun a specific job
gh run rerun 12345678 --job 987654321

# Rerun with debug logging enabled
gh run rerun 12345678 --debug
```

cancel

`gh run cancel` cancels a run that is queued or in progress. If the run is stuck and a normal cancel does not take effect, use `--force`.

```
# Cancel a run
gh run cancel 12345678

# Force-cancel a run that is not responding
gh run cancel 12345678 --force
```

delete

`gh run delete` removes a run's record from GitHub. This does not affect the underlying code or workflow; it only removes the run entry from the Actions history. Deletion is permanent.

```
# Interactively select a run to delete
gh run delete

# Delete a specific run
gh run delete 12345678
```

download

`gh run download` fetches the artifacts a run produced. Each artifact is extracted into its own subdirectory named after the artifact, unless you are downloading exactly one artifact (in which case it is extracted into the current directory).

```
# Download all artifacts from a specific run
gh run download 12345678

# Download a specific artifact by name
gh run download 12345678 --name dist

# Download multiple named artifacts
gh run download 12345678 --name dist --name coverage

# Download artifacts matching a glob pattern
gh run download 12345678 --pattern "coverage-*"

# Download to a specific directory
gh run download 12345678 --dir ./artifacts

# Download the latest artifact across all runs (no run-id required)
gh run download --name dist
```

Key options

Shared flag

Flag	What it does	When to use it
<code>-R / --repo</code>	Target a different repository	Working outside the current repo

list

Flag	What it does	When to use it
<code>-b / --branch</code>	Filter by branch name	Reviewing CI health on a feature branch
<code>-c / --commit</code>	Filter by commit SHA	Auditing a specific commit's CI results
<code>-e / --event</code>	Filter by trigger event	Isolating push vs. PR vs. schedule runs

<code>-s / --status</code>	Filter by run status	Finding all failures or pending runs
<code>-w / --workflow</code>	Filter by workflow name or file	Focusing on one workflow
<code>-u / --user</code>	Filter by triggering user	Auditing a team member's runs
<code>-a / --all</code>	Include disabled workflows	Seeing runs from workflows that have been disabled
<code>-L / --limit</code>	Maximum runs to fetch (default 20)	Getting a longer history
<code>--json fields</code>	Output JSON fields	Scripting and piping
<code>-q / --jq</code>	Filter JSON with a jq expression	Extracting specific fields inline

view

Flag	What it does	When to use it
<code>-v / --verbose</code>	Show each job's steps	Getting a step-by-step breakdown
<code>--log</code>	Print the full raw log	Investigating a run in detail
<code>--log-failed</code>	Print only failed step logs	Quickly finding what broke
<code>-j / --job</code>	Focus on a specific job by database ID	Drilling into one job's log
<code>-a / --attempt</code>	View a specific retry attempt	Comparing attempts of a flaky run
<code>--exit-status</code>	Exit non-zero if run failed	Scripting pass/fail gates
<code>-w / --web</code>	Open the run in a browser	Viewing detailed annotations
<code>--json fields</code>	Output JSON fields	Scripting
<code>-q / --jq</code>	Filter JSON with a jq expression	Extracting specific fields inline

watch

Flag	What it does	When to use it
<code>--compact</code>	Show only relevant or failed steps	Reducing noise for large workflows
<code>--exit-status</code>	Exit non-zero if run fails	Chaining with shell commands
<code>-i / --interval</code>	Refresh interval in seconds (default 3)	Reducing API calls on slow connections

rerun

Flag	What it does	When to use it
<code>--failed</code>	Rerun only failed jobs and their dependencies	Saving time after a partial failure
<code>-j / --job</code>	Rerun a specific job by database ID	Retrying one flaky job
<code>-d / --debug</code>	Enable debug logging for the rerun	Diagnosing hard-to-reproduce failures

cancel

Flag	What it does	When to use it
<code>--force</code>	Force-cancel the run	When a normal cancel is not taking effect

download

Flag	What it does	When to use it
<code>-n / --name</code>	Download artifacts matching this name (repeatable)	Fetching specific build outputs
<code>-p / --pattern</code>	Download artifacts matching a glob pattern	Fetching groups of related artifacts
<code>-D / --dir</code>	Destination directory (default <code>.</code>)	Keeping artifacts out of the project root

Best practices

Use `--exit-status` in scripts and CI gates. When you call `gh run view` or `gh run watch` in a script, add `--exit-status` so the script fails if the run failed. Without it, the command always exits 0, silently hiding failures:

```
gh run watch 12345678 --exit-status || { echo "CI failed"; exit 1; }
```

Rerun with `--failed` before rerunning the whole workflow. GitHub bills Actions minutes per job. Rerunning only the failed jobs is cheaper and faster:

```
gh run rerun 12345678 --failed
```

Use `--debug` when a job fails inconsistently. The flag enables step-level debug output (`ACTIONS_STEP_DEBUG=true`) for the next attempt. Turn it off by rerunning again without the flag once you have your diagnosis.

Download artifacts to a named directory. By default artifacts land in `.`, which can clutter your working tree. Use `--dir ./artifacts` (or a path that is listed in `.gitignore`) to keep things tidy:

```
gh run download 12345678 --dir ./artifacts
```

Filter run list before acting. Combine `--status`, `--branch`, and `--workflow` flags to narrow to exactly the runs you mean to rerun or cancel. Accidentally canceling the wrong run in a shared repository is disruptive.

Use JSON output for scripting. The `--json` flag with a list of fields and `--jq` for inline filtering lets you build reliable pipelines without parsing human-readable output:

```
gh run list --status failure --json databaseId,headBranch --jq '.[].databaseId'
```

Pitfalls & gotchas

`--job` takes a database ID, not the URL number. The integer at the end of a job's browser URL (`/runs/<run-id>/jobs/<number>`) is not the database ID. Passing the URL number to `gh run rerun --job` or `gh run view --job` results in a 404. Get the correct ID first:

```
gh run view 12345678 --json jobs --jq '.jobs[] | {name, databaseId}'
```

`gh run watch` does not work with fine-grained PATs. GitHub does not expose a `checks:read` scope for fine-grained personal access tokens. Authenticated with such a token, `gh run watch` will fail. Use a classic PAT or the `GITHUB_TOKEN` provided by Actions.

`run list --workflow` skips disabled workflows unless you add `--all`. If you filter by workflow name and get no results, the workflow may be disabled. Add `-a` / `--all` to include it.

Runs from organization or enterprise ruleset workflows show no workflow name. This is an API limitation. The `workflowName` JSON field will be empty for those runs.

`gh run download` without a run ID fetches the latest artifact of that name, which may not be what you expect. If a later run overwrote or deleted an artifact, you will get the most recent surviving version. Always supply the run ID when you need a specific artifact:

```
gh run download 12345678 --name dist # specific run
```

Deleting a run is permanent. There is no undo. The underlying code and workflow file are unaffected, but the run record is gone. Confirm the run ID with `gh run view` before deleting.

Worked examples

Monitoring a deployment pipeline end-to-end

You push a commit and want to tail the deployment workflow until it succeeds or fails, then know the result immediately.

```
# Find the run triggered by your push
gh run list --branch my-feature --workflow deploy.yml --limit 5
```

STATUS	TITLE	WORKFLOW	BRANCH	EVENT	ID
in_progress	Add caching to build step	deploy.yml	my-feature	push	98765432
success	Fix typo in README	deploy.yml	my-feature	push	98765000

```
# Tail the live run
gh run watch 98765432 --compact --exit-status
```

```
* deploy.yml #98765432 (my-feature)
  * build (in_progress)
    ✓ Checkout
    ✓ Set up Node
    * Run tests
    ...
  ✓ deploy.yml completed with 'success'
```

The shell exits 0, so you can chain a notification:

```
gh run watch 98765432 --exit-status && say "Deploy passed"
```

Diagnosing and rerunning a flaky test job

A run failed in one job. View the failed log, understand what broke, then rerun with debug logging.

```
# View only the failed steps
gh run view 12345678 --log-failed
```

```
► test (ubuntu-latest)
  ✗ Run integration tests
    Error: connection refused on port 5432
```

The log shows a transient database connection error. Rerun with debug to capture more detail:

```
gh run rerun 12345678 --failed --debug
```

The rerun passes. Compare the two attempts to confirm:

```
gh run view 12345678 --attempt 2 --verbose
```

Collecting build artifacts in CI

A workflow builds platform-specific binaries and uploads them as artifacts named `dist-linux`, `dist-macos`, and `dist-windows`. After the run succeeds, download all three into a release staging directory:

```
RUNID=$(gh run list --workflow build.yml --status success --limit 1 \
  --jq 'databaseId' --jq '[0].databaseId')

gh run download "$RUNID" --pattern "dist-*" --dir ./release/staging
```

```
✓ Downloaded dist-linux to ./release/staging/dist-linux
✓ Downloaded dist-macos to ./release/staging/dist-macos
✓ Downloaded dist-windows to ./release/staging/dist-windows
```

Bulk-cancelling queued runs on a stale branch

A branch had a flurry of pushes and left many queued runs you no longer need. List their IDs in JSON, then cancel each one:

```
gh run list --branch old-feature --status queued --jq 'databaseId' \
  --jq '[0].databaseId' \
  | xargs -I{} gh run cancel {}
```

Recovery

A run is stuck in progress and will not cancel. Use `--force`:

```
gh run cancel 12345678 --force
```

You deleted the wrong run. Deletion is permanent. The underlying code and workflow file are unaffected; only the run record is gone. If the commit still exists you can trigger a new run by re-pushing or using a manual dispatch (`gh workflow run`).

A rerun produced a different failure. View the specific attempt to compare logs side by side:

```
gh run view 12345678 --attempt 1 --log-failed
gh run view 12345678 --attempt 2 --log-failed
```

An artifact directory appears empty after download. Confirm the artifact name is exact, then re-download:

```
gh run view 12345678 --json jobs \
  --jq '.jobs[].steps[].name'
gh run download 12345678 --name exact-artifact-name --dir ./artifacts
```

See also

- *auth* — authenticate `gh` before running any `gh run` commands.
- *pr* — `gh pr checks` shows CI status associated with a pull request.
- *workflow* — `gh workflow run`, `gh workflow enable`, and `gh workflow disable` control the workflow definitions that generate runs.
- *repo* — `gh repo view` links to the Actions tab for a repository.

workflow

List, view, enable, disable, and manually trigger GitHub Actions workflow files directly from the command line.

Mental model

A GitHub Actions workflow is a YAML file stored under `.github/workflows/` in a repository. GitHub Actions reads those files and turns them into pipelines that run in response to events — pushes, pull requests, schedules, or manual dispatches.

`gh workflow` is the command-line interface to that layer. It does not interact with individual runs (that is the job of `gh run`); it works at the level of the *workflow definition file itself* — listing all files the repository knows about, showing their YAML source, toggling them on and off, and firing a manual `workflow_dispatch` event to start a run.

The key distinction: a workflow is a file, a run is an execution of that file. `gh workflow list` shows you the files; `gh run list` shows you the executions. Use both together when diagnosing a CI problem.

Workflows can be in one of two states:

- **active** — eligible to be triggered by events and visible in `gh workflow list` by default.
- **disabled** — will not be triggered, and is hidden from the default listing unless you pass `--all`.

GitHub identifies each workflow by an integer ID as well as by its filename. `gh workflow` accepts either form in most subcommands.

Synopsis

```
gh workflow list    [flags]
gh workflow view    [<workflow-id> | <workflow-name> | <filename>] [flags]
gh workflow run     [<workflow-id> | <workflow-name>] [flags]
gh workflow enable  [<workflow-id> | <workflow-name>] [flags]
gh workflow disable [<workflow-id> | <workflow-name>] [flags]
```

All subcommands accept `-R / --repo [HOST/]OWNER/REPO` to target a repository other than the one inferred from the current directory.

Everyday usage

List all active workflows in the current repository:

```
gh workflow list
```

NAME	STATE	ID
CI	active	12345678
Release	active	23456789
Nightly tests	active	34567890

Include disabled workflows in the output:

```
gh workflow list --all
```

View the YAML source of a workflow interactively (picks from a menu when called with no argument):

```
gh workflow view --yaml
```

View a specific workflow by filename:

```
gh workflow view ci.yml --yaml
```

Open a workflow's summary page in the browser:

```
gh workflow view ci.yml --web
```

Trigger a workflow that has a `workflow_dispatch` trigger:

```
gh workflow run ci.yml
```

Trigger the same workflow on a specific branch:

```
gh workflow run ci.yml --ref my-feature-branch
```

Pass named inputs to a parameterised workflow:

```
gh workflow run deploy.yml -f environment=staging -f version=1.4.2
```

Disable a workflow so it stops running:

```
gh workflow disable stale.yml
```

Re-enable it:

```
gh workflow enable stale.yml
```

Key options

list

Flag	What it does	When to use it
<code>-a / --all</code>	Include disabled workflows in the output	Auditing all workflow files, not just active ones
<code>-L / --limit int</code>	Cap the number of results returned (default 50)	Repositories with many workflows
<code>--json fields</code>	Emit JSON with selected fields (<code>id</code> , <code>name</code> , <code>path</code> , <code>state</code>)	Scripting and piping to <code>jq</code>
<code>-q / --jq expression</code>	Filter the JSON output with a <code>jq</code> expression	Inline extraction without a separate <code>jq</code> call

<code>-t / --template string</code>	Format JSON output using a Go template	Custom tabular views
-------------------------------------	--	----------------------

view

Flag	What it does	When to use it
<code>-y / --yaml</code>	Print the raw YAML of the workflow file	Auditing what Actions will actually execute
<code>-w / --web</code>	Open the workflow summary in the browser	Seeing recent run history at a glance
<code>-r / --ref string</code>	Show the workflow file from a specific branch or tag	Comparing versions across branches

run

Flag	What it does	When to use it
<code>-r / --ref string</code>	Run the workflow file at a specific branch or tag	Testing a workflow change before it merges
<code>-f / --raw-field key=value</code>	Supply a workflow input as a plain string	Passing simple scalar parameters
<code>-F / --field key=value</code>	Supply a workflow input, respecting @ file-read syntax	Passing file contents as input values
<code>--json</code>	Read workflow inputs as JSON via stdin	Scripting complex or dynamic input sets

enable / disable

These subcommands take no flags beyond `--repo`. Supply the workflow by its integer ID, display name, or filename.

Best practices

Prefer filenames over numeric IDs in scripts. IDs are repository-stable but opaque in code review. A filename like `ci.yml` communicates intent; `12345678` does not. Both are accepted by every subcommand.

Pair `gh workflow run` with `gh run watch` to close the loop. Triggering a dispatch event fires the run asynchronously. Immediately following with `gh run watch` (from the `gh run` command group) lets you tail the logs without opening a browser:

```
gh workflow run ci.yml --ref main
gh run watch    # interactively pick the just-triggered run
```

Use `--json` with `gh workflow list` for automation. The four JSON fields — `id`, `name`, `path`, `state` — are enough to build inventory scripts, dashboards, or audit reports without screen-scraping:

```
gh workflow list --all --json id,name,state \
| jq '.[ ] | select(.state == "disabled") | .name'
```

Gate `workflow_dispatch` triggers with inputs for repeatability. A workflow that accepts named inputs can be triggered with consistent parameters from the CLI, from another workflow with `workflow_call`, or by a human through the GitHub UI. Defining inputs in the YAML and passing them with `-f` in `gh workflow run` gives you a single, auditable invocation pattern.

Use `--ref` to test a modified workflow before merging. Push your `.github/workflows/ci.yml` change to a feature branch, then run:

```
gh workflow run ci.yml --ref fix/update-node-version
```

This exercises the updated file without touching the default branch.

Pitfalls & gotchas

`gh workflow run` only works on workflows with `workflow_dispatch`. Attempting to run a workflow that lacks an `on: workflow_dispatch` trigger returns an error. Check with `gh workflow view <file> --yaml` to confirm the trigger is present before trying to dispatch.

Disabled workflows are invisible to `gh workflow list` by default. If a workflow seems to have disappeared, run `gh workflow list --all`. A well-intentioned colleague may have disabled it. Re-enable with `gh workflow enable <name>`.

Workflow inputs are strings — no type coercion. The `-f` / `--raw-field` flag always passes a plain string. If your workflow YAML declares an input of type `boolean` or `number`, GitHub Actions performs the coercion on its end, but pass `"true"` or `"42"` rather than bare values to avoid shell interpretation issues.

The `--ref` flag on `run` must contain the workflow file. If the branch you name does not yet have the workflow file at `.github/workflows/<name>`, the run fails with a “workflow file not found” error. Make sure the branch has been pushed before dispatching.

`gh workflow view` without `--yaml` shows a summary, not the source. The default output is a human-readable table of recent runs and the workflow’s state. Pass `--yaml` explicitly when you want to inspect or copy the actual YAML definition.

Worked examples

Auditing and re-enabling a disabled workflow

A nightly test workflow stopped appearing in run history. Investigate:

```
gh workflow list --all
```

NAME	STATE	ID
CI	active	12345678
Release	active	23456789
Nightly tests	disabled	34567890

Inspect the file to confirm it still has the right trigger:

```
gh workflow view 34567890 --yaml
```

```
name: Nightly tests
on:
  schedule:
    - cron: '0 2 * * *'
...
```

Re-enable it:

```
gh workflow enable 34567890
```

```
✓ Enabled nightly.yml
```

Verify the state has changed:

```
gh workflow list
```

NAME	STATE	ID
CI	active	12345678
Release	active	23456789
Nightly tests	active	34567890

Triggering a parameterised deployment workflow

A repository has a `deploy.yml` workflow with two declared inputs: `environment` (values: `staging`, `production`) and `version`. Trigger a staging deployment from the command line:

```
gh workflow run deploy.yml -f environment=staging -f version=2.3.1
```

```
✓ Created workflow_dispatch event for deploy.yml at main
```

To see runs for this workflow, try: `gh run list --workflow=deploy.yml`

Watch the run as it executes (see *run* for full details):

```
gh run list --workflow=deploy.yml --limit 1
```

STATUS	TITLE	WORKFLOW	BRANCH	EVENT	ID
*	Manual run	deploy	main	workflow_dispatch	9876543210

Pass a complex input set from a JSON file:

```
cat inputs.json
```

```
{"environment": "staging", "version": "2.3.1"}
```

```
gh workflow run deploy.yml --json < inputs.json
```

Inspecting a workflow on a feature branch before merging

You are reviewing a pull request that modifies `.github/workflows/ci.yml`. Before approving, run the updated workflow against the PR branch:

```
gh workflow run ci.yml --ref feature/upgrade-actions
```

Open the resulting run in the browser to review logs:

```
gh run list --workflow=ci.yml --limit 1
# note the run ID from the output, then:
gh run view <run-id> --web
```

Recovery

If a workflow run was triggered accidentally or with wrong inputs, it can be cancelled via `gh run cancel <run-id>` (see *run*). There is no undo for `gh workflow disable` or `gh workflow enable`, but re-running the inverse command restores the previous state immediately.

If a dispatch triggers no run — common when the target branch does not contain the workflow file — push the branch first, then re-run the command. If runs continue not to appear, check `gh workflow list --all` to confirm the workflow is not disabled, and verify the YAML has `on: workflow_dispatch` with `gh workflow view <file> --yaml`.

For authentication or permission errors when calling `gh workflow run`, see *auth* — the active credential must have the `workflow` OAuth scope. Add it without logging out:

```
gh auth refresh --scopes workflow
```

See also

- *run* — list, view, watch, cancel, and rerun individual workflow executions.
- *auth* — managing credentials and OAuth scopes, including the `workflow` scope required to trigger runs.
- *secret* — managing the encrypted secrets that workflows consume at runtime.
- *variable* — managing plaintext configuration variables available to workflow steps.
- *cache* — inspecting and deleting the dependency caches that workflows create.

cache

Inspect and delete GitHub Actions caches stored for a repository.

Mental model

Every time a GitHub Actions workflow runs, it can save files — dependency downloads, compiled outputs, tool installations — to a cache keyed by a string you choose. On the next run the workflow restores that cache, skipping the expensive re-download or re-build.

GitHub stores those caches server-side, scoped to a *(repository, ref, key)* triple. The cache is not shared between repositories, and by default a cache created on a branch cannot be read from a different branch (except that branches can always read their base branch's caches). GitHub automatically evicts caches after 7 days of no access, and caps total cache storage at 10 GB per repository.

`gh cache` gives you two operations on that server-side store:

- **list** — see what is cached, when it was last used, and how large it is.
- **delete** — remove one cache by ID or key, or wipe everything at once.

The `list` and `delete` commands target whichever repository `gh` resolves from the current directory, or the one you name with `--repo`. Both require a `repo` scope token, which the normal `gh auth login` flow grants.

Synopsis

```
gh cache list  [--key <prefix>] [--ref <ref>] [--sort <field>] [--order asc|desc]
               [--limit <n>] [--json <fields>] [--jq <expr>] [--template <tmpl>]
               [-R [HOST/]OWNER/REPO]

gh cache delete [<cache-id> | <cache-key> | --all] [--ref <ref>]
               [--succeed-on-no-caches]
               [-R [HOST/]OWNER/REPO]
```

Everyday usage

List all caches for the current repository (sorted by most recently accessed, newest first):

```
gh cache list
```

List caches whose key starts with a known prefix:

```
gh cache list --key node-modules-
```

List caches for a specific branch:

```
gh cache list --ref refs/heads/main
```

Delete a single cache by its numeric ID:

```
gh cache delete 1234
```

Delete a single cache by its exact key:

```
gh cache delete "node-modules-linux-abc123"
```

Delete every cache in the repository (useful after a large-scale dependency change):

```
gh cache delete --all
```

Delete all caches for a specific pull request:

```
gh cache delete --all --ref refs/pull/42/merge
```

Key options

list

Flag	What it does	When to use it
<code>-k / --key</code>	Filter to caches whose key starts with this prefix	Narrow results when you know the key naming convention
<code>-r / --ref</code>	Filter to caches for a specific ref (refs/heads/<branch> or refs/pull/<n>/merge)	Check a single branch or PR
<code>-S / --sort</code>	Sort by <code>created_at</code> , <code>last_accessed_at</code> , or <code>size_in_bytes</code> (default <code>last_accessed_at</code>)	Identify stale or oversized caches
<code>-O / --order</code>	<code>asc</code> or <code>desc</code> (default <code>desc</code>)	Reverse the sort to find the oldest or smallest entries
<code>-L / --limit</code>	Maximum number of caches to return (default 30)	Repos with many caches; increase if you need the full list
<code>--json</code>	Output raw JSON with the named fields	Scripting; available fields: <code>createdAt</code> , <code>id</code> , <code>key</code> , <code>lastAccessedAt</code> , <code>ref</code> , <code>sizeInBytes</code> , <code>version</code>
<code>-q / --jq</code>	Filter JSON output with a jq expression	Extract a single field from machine-readable output
<code>-t / --template</code>	Format JSON with a Go template	Custom human-readable output
<code>-R / --repo</code>	Target a different repository as [HOST/]OWNER/REPO	Running outside the repo's directory

delete

Flag	What it does	When to use it
<code>-a / --all</code>	Delete every cache in the repo, or every cache for the given <code>--ref</code>	Mass cleanup after a cache-key rename or dependency overhaul

<code>-r / --ref</code>	Scope deletion to a specific ref (refs/heads/<branch> or refs/pull/<n>/merge)	Combined with <code>--all</code> to target one branch or PR only
<code>--succeed-on-no-caches</code>	Exit with code 0 even when no matching caches exist (requires <code>--all</code>)	CI pipelines where the cache may or may not be present
<code>-R / --repo</code>	Target a different repository as [HOST/]OWNER/REPO	Running outside the repo's directory

Best practices

List before you delete. Run `gh cache list` first to confirm what you are about to remove. A typo in a key name passed to `gh cache delete` will silently do nothing (unless the key matches something unexpected), but `--all` is immediate and irreversible.

Use `--ref` to limit blast radius. When you want to reset caches for a single PR without touching main's caches, always combine `--all` with `--ref refs/pull/<n>/merge`. Omitting `--ref` with `--all` wipes every cache in the repository.

Use `--succeed-on-no-caches` in CI scripts. A cleanup step in a workflow that runs `gh cache delete --all` will fail with exit code 1 if no caches exist (for example on a first run). Add `--succeed-on-no-caches` to make the step idempotent:

```
gh cache delete --all --succeed-on-no-caches
```

Sort by size to spot bloat. Cache entries can grow large if they include compiled artefacts. Find your heaviest cache:

```
gh cache list --sort size_in_bytes --order desc --limit 5
```

Use JSON output for automation. Pair `--json` with `--jq` to extract exactly the field you need without parsing tabular text:

```
gh cache list --json id,key,sizeInBytes --jq '.[0] | select(.sizeInBytes > 500000000) | .key'
```

This prints keys for caches larger than 500 MB.

Pitfalls & gotchas

Cache keys are exact on delete, prefix on list. `gh cache list --key foo` matches any key that *begins with* `foo`. `gh cache delete foo` matches only an entry whose key is exactly `foo`. If you have caches with keys `foo-linux` and `foo-macos`, listing with `--key foo` shows both; deleting with `gh cache delete foo` matches neither.

Numeric IDs and string keys are both valid delete targets, but they look different. A numeric argument like `1234` is treated as a cache ID. A non-numeric string is treated as a cache key. To delete a cache whose key happens to be a number (unlikely but possible), list first and use the ID.

`--all` requires no positional argument. Passing a key or ID alongside `--all` is an error. Choose one mode.

`--succeed-on-no-caches` **must be combined with** `--all`. Using it without `--all` is an error. The flag only affects the “no caches found” exit code in the bulk-delete path.

Ref format is rigid. The `--ref` flag requires the fully-qualified ref string — `refs/heads/main`, not just `main`; `refs/pull/42/merge`, not `42` or `PR-42`. Passing a short ref returns no results without an error, which can be confusing.

The default limit is 30. A busy repository can have hundreds of caches. If `gh cache list` seems to be missing entries, raise the limit:

```
gh cache list --limit 200
```

Deletion requires the `repo` scope. If your token was created with reduced scopes (e.g. a fine-grained token scoped to read-only), `gh cache delete` will fail with a permission error. Refresh your token with the full `repo` scope, or use a classic personal access token. See [auth](#) for how to add scopes.

Worked examples

Auditing cache growth on a long-lived branch

A CI run complains that the repository is nearing its 10 GB cache limit. Find what is taking up the most space:

```
gh cache list --sort size_in_bytes --order desc --limit 10
```

ID	KEY	REF	SIZE	LAST
ACCESSED				
9871	gradle-deps-linux-abc12345	refs/heads/main	1.8 GB	about 1 hour ago
9870	gradle-deps-macos-def67890	refs/heads/main	1.7 GB	about 2 hours ago
9869	gradle-deps-windows-ghi11111	refs/heads/main	1.6 GB	about 3 hours ago
9840	node-modules-linux-jkl22222	refs/heads/main	420 MB	about 1 day ago
...				

The Gradle dependency caches are 5 GB combined. After updating the dependency lock file, the old caches will never be hit again. Delete them by key:

```
gh cache delete gradle-deps-linux-abc12345
gh cache delete gradle-deps-macos-def67890
gh cache delete gradle-deps-windows-ghi11111
```

Or wipe every cache on main so the next workflow run rebuilds them fresh:

```
gh cache delete --all --ref refs/heads/main
```

Cleaning up caches for a merged pull request

After merging PR 99, its associated caches are stranded. They will be evicted automatically after 7 days, but if you are tight on quota you can reclaim the space immediately:

```
# Verify what is there
gh cache list --ref refs/pull/99/merge
```

ID	KEY	REF	SIZE	LAST ACCESSED
8812	pip-linux-abc123	refs/pull/99/merge	310 MB	3 days ago
8811	pip-linux-abc123	refs/pull/99/merge	308 MB	4 days ago

```
gh cache delete --all --ref refs/pull/99/merge
```

```
✓ Deleted 2 caches from myorg/myrepo
```

Scripting a cache cleanup in a GitHub Actions workflow

Add a final job that clears PR caches when a pull request is closed:

```
on:
  pull_request:
    types: [closed]

jobs:
  cleanup-caches:
    runs-on: ubuntu-latest
    steps:
      - name: Delete PR caches
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          gh cache delete --all \
            --ref refs/pull/${ github.event.pull_request.number }/merge \
            --succeed-on-no-caches \
            --repo ${ github.repository }
```

`--succeed-on-no-caches` prevents the step from failing when the PR had no caches (e.g., if every run was a cache miss).

Recovery

`gh cache delete` has no undo. A deleted cache is gone; the next workflow run will rebuild it from scratch. This is usually harmless — the cache was a performance optimisation, not source data — but it does mean your next CI run will be slower.

If you accidentally deleted caches across the entire repository with `--all`, simply trigger a new workflow run on each affected branch. The runners will repopulate the caches on the first pass. There is nothing to recover.

If `gh cache list` or `gh cache delete` returns a permissions error, your token lacks the `repo` scope. See [auth](#) for how to refresh scopes:

```
gh auth refresh --scopes repo
```

See also

- [auth](#) — manage tokens and scopes; `gh cache delete` requires the `repo` scope.

- *run* — `gh run list` and `gh run view` show the workflow runs that created caches.
- *workflow* — trigger or re-run workflows after clearing caches to rebuild them.

release

Create, inspect, and manage GitHub Releases — versioned snapshots of your project paired with release notes and downloadable assets.

Mental model

A GitHub Release wraps three things together: a **git tag** (the exact commit being released), **release notes** (human-readable changelog), and zero or more **binary assets** (compiled binaries, checksums, archives).

Releases live in the GitHub UI at the `Releases` tab and are surfaced by package managers, CI scripts, and users who want a stable download without cloning the full repository.

The lifecycle has three phases:

1. **Draft** — created but invisible to the public. Useful for staging notes and assets before the announcement.
2. **Published** — visible. May be flagged as `prerelease` (alpha/beta) or as `latest` (the default download users see).
3. **Immutable** — once a repository enables release immutability, a published release's tag and assets are locked. Draft releases remain editable until published.

`gh release` mirrors this lifecycle exactly. You can create a draft, upload assets, edit notes, then flip it to published — or do it all in one command. Tags are created automatically from the default branch if they do not already exist; use `--verify-tag` to enforce that the tag must already exist first.

Synopsis

```
gh release create [<tag>] [<files>...]
gh release list   [flags]
gh release view   [<tag>]
gh release edit   <tag>
gh release delete <tag>
gh release download [<tag>] [flags]
gh release upload <tag> <files>...
gh release delete-asset <tag> <asset-name>
gh release verify  [<tag>]
gh release verify-asset [<tag>] <file-path>
```

All subcommands accept `-R / --repo [HOST/]OWNER/REPO` to target a repository other than the one inferred from the current directory.

Everyday usage

Listing releases

See what has been shipped:

```
gh release list
```

TITLE	TYPE	TAG NAME	PUBLISHED
v2.1.0	Latest	v2.1.0	about 2 days ago
v2.0.1		v2.0.1	about 3 weeks ago

v2.0.0		v2.0.0	about 2 months ago
v1.9.0-rc1	Pre-	v1.9.0-rc1	about 3 months ago

Exclude drafts and pre-releases to see only stable releases:

```
gh release list --exclude-drafts --exclude-pre-releases
```

Viewing a release

Inspect the latest release:

```
gh release view
```

Inspect a specific tag:

```
gh release view v2.1.0
```

Open it in a browser:

```
gh release view v2.1.0 --web
```

Creating a release interactively

Run without arguments and `gh` walks you through a prompt sequence:

```
gh release create
```

Creating a release non-interactively

The most common one-liner for a new stable release:

```
gh release create v2.2.0 --title "v2.2.0" --notes "Bug fixes and performance improvements."
```

Let GitHub generate the changelog automatically from merged pull requests:

```
gh release create v2.2.0 --generate-notes
```

Use notes from an annotated git tag:

```
gh release create v2.2.0 --notes-from-tag
```

Create a draft first, review it in the browser, then publish later:

```
gh release create v2.2.0 --draft --title "v2.2.0" --notes-file CHANGELOG.md
```

Mark a release as a pre-release:

```
gh release create v2.2.0-rc1 --prerelease --title "v2.2.0 Release Candidate 1" --generate-notes
```

Attaching assets at creation time

Pass file paths after the tag. To give an asset a custom display label, append `#Label text` to the path:

```
gh release create v2.2.0 \  
  ./dist/myapp-linux-amd64.tar.gz \  
  './dist/myapp-darwin-arm64.tar.gz#macOS (Apple Silicon)' \  
  ./dist/checksums.txt
```

Glob patterns work too:

```
gh release create v2.2.0 ./dist/*.tar.gz ./dist/checksums.txt
```

Editing an existing release

Publish a draft:

```
gh release edit v2.2.0 --draft=false
```

Update the release notes from a file:

```
gh release edit v2.2.0 --notes-file /path/to/release_notes.md
```

Rename a tag after the fact:

```
gh release edit v2.2.0 --tag v2.2.0-final
```

Uploading assets to an existing release

Add an asset that was missed at creation:

```
gh release upload v2.2.0 ./dist/myapp-windows-amd64.zip
```

Replace an asset that was uploaded with a mistake:

```
gh release upload v2.2.0 ./dist/myapp-windows-amd64.zip --clobber
```

Deleting an asset

```
gh release delete-asset v2.2.0 myapp-windows-amd64.zip
```

Skip the confirmation prompt in scripts:

```
gh release delete-asset v2.2.0 myapp-windows-amd64.zip --yes
```

Downloading release assets

Download all assets from a specific release:

```
gh release download v2.2.0
```

Download only matching files from the latest release (`--pattern` is required when no tag is given):

```
gh release download --pattern '*.tar.gz'
```

Multiple patterns:

```
gh release download v2.2.0 -p '*.tar.gz' -p 'checksums.txt'
```

Download source archive instead of release assets:

```
gh release download v2.2.0 --archive=tar.gz
```

Save to a specific directory:

```
gh release download v2.2.0 --dir ./downloads
```

Verifying release attestations

Verify the latest release has a valid cryptographically signed attestation:

```
gh release verify
```

Verify a specific release:

```
gh release verify v2.2.0
```

Verify that a locally downloaded asset matches the attestation:

```
gh release verify-asset v2.2.0 ./dist/myapp-linux-amd64.tar.gz
```

Deleting a release

Delete a release while keeping the git tag:

```
gh release delete v2.2.0
```

Delete both the release and the tag:

```
gh release delete v2.2.0 --cleanup-tag
```

Key options

create

Flag	What it does	When to use it
<code>-t / --title</code>	Set the release title	Non-interactive creation
<code>-n / --notes</code>	Release notes as a string	Short changelogs inline
<code>-F / --notes-file</code>	Read notes from a file (or <code>-</code> for stdin)	Longer changelogs in a file

<code>--generate-notes</code>	Auto-generate notes via GitHub API	Projects using pull requests with good titles
<code>--notes-from-tag</code>	Use annotated tag message as notes	Tag-centric workflows
<code>--notes-start-tag</code>	Starting tag for generated notes range	Skip older history in auto-generated notes
<code>-d / --draft</code>	Save as draft, not published	Stage the release before announcing
<code>-p / --prerelease</code>	Mark as pre-release	Alpha/beta/RC builds
<code>--latest</code>	Control whether this is the “Latest” release	Backfill a patch without displacing a newer latest
<code>--target</code>	Branch or commit SHA for tag auto-creation	Releasing from a branch other than main
<code>--verify-tag</code>	Abort if the tag does not already exist remotely	Enforce tag-first discipline
<code>--fail-on-no-commits</code>	Fail if no new commits since last release	CI guard against accidental re-releases
<code>--discussion-category</code>	Open a GitHub Discussion alongside the release	Community announcement releases

list

Flag	What it does	When to use it
<code>-L / --limit</code>	Maximum number to fetch (default 30)	Paginate a long release history
<code>-O / --order</code>	<code>asc</code> or <code>desc</code> (default <code>desc</code>)	Chronological ascending for scripting
<code>--exclude-drafts</code>	Omit drafts	Show only public releases
<code>--exclude-pre-releases</code>	Omit pre-releases	Show only stable releases
<code>--json fields</code>	Output selected fields as JSON	Scripting and automation
<code>-q / --jq</code>	Filter JSON with a jq expression	Extract a specific field

download

Flag	What it does	When to use it
<code>-p / --pattern</code>	Glob filter for asset filenames	Download only specific files
<code>-A / --archive</code>	Download source archive (<code>zip</code> or <code>tar.gz</code>)	Source-only fetches
<code>-D / --dir</code>	Target directory (default <code>.</code>)	Organise downloads
<code>-O / --output</code>	Write a single asset to a specific path (use <code>-</code> for stdout)	Scripting
<code>--clobber</code>	Overwrite existing files	Re-download updated assets

<code>--skip-existing</code>	Skip files that already exist locally	Idempotent download scripts
------------------------------	---------------------------------------	-----------------------------

edit

Flag	What it does	When to use it
<code>--draft</code>	Toggle draft state (<code>--draft=false</code> to publish)	Publish a previously staged draft
<code>--prerelease</code>	Mark the release as a prerelease	Promote RC to stable
<code>--latest</code>	Explicitly mark as latest	Set after backfilling a patch
<code>-t / --title</code>	Update the title	Correct a typo
<code>-n / --notes</code>	Replace notes with a string	Quick inline correction
<code>-F / --notes-file</code>	Replace notes from a file	Bulk notes update
<code>--tag</code>	Rename the tag	Fix a mis-tagged release
<code>--target</code>	Update target branch or SHA	Correction of release point
<code>--verify-tag</code>	Abort if new tag does not exist remotely	Safety check when renaming
<code>--discussion-category</code>	Start a discussion when publishing a draft	Community releases

delete

Flag	What it does	When to use it
<code>--cleanup-tag</code>	Also delete the git tag	Full rollback of a mis-release
<code>-y / --yes</code>	Skip confirmation prompt	Scripting

Best practices

Create a draft first for major releases. Use `--draft` to prepare the release offline, upload and verify assets, preview the notes in the GitHub UI, then publish with `gh release edit v1.0.0 --draft=false`. Drafts are invisible to users and to any CI that watches for new release events.

Use `--generate-notes` for consistent changelogs. GitHub's Release Notes API groups merged pull requests by label (bug, enhancement, etc.) according to your `.github/release.yml` policy. The result is a structured, reviewable changelog that requires no manual writing. Pair it with `--notes-start-tag` to scope the range when releases are close together.

Tag before you release in strict environments. Create the git tag locally and push it before running `gh release create`:

```
git tag -a v2.2.0 -m "Release v2.2.0"
git push origin v2.2.0
gh release create v2.2.0 --verify-tag --generate-notes
```

The `--verify-tag` flag aborts the command if the push failed — safer than letting `gh` auto-create a tag from whatever the remote default branch happens to be at that moment.

Attach checksums alongside binaries. Include a `checksums.txt` (SHA-256 of every binary) as a release asset. Users can verify their download without needing GitHub Attestations, and you can verify with both:

```
shasum -a 256 dist/*.tar.gz > dist/checksums.txt
gh release create v2.2.0 dist/*.tar.gz dist/checksums.txt
```

Automate releases in CI with `GH_TOKEN`. In GitHub Actions the workflow-scoped `GITHUB_TOKEN` has `contents: write` permission and is sufficient for `gh release create`. Set it as `GH_TOKEN` rather than calling `gh auth login`:

```
- name: Create release
  env:
    GH_TOKEN: ${github.token}
  run: |
    gh release create "$TAG" --generate-notes ./dist/*.tar.gz
```

See *auth* for the broader token and scope discussion.

Use `--fail-on-no-commits` in automated pipelines. If a release pipeline runs on a schedule, this flag prevents a no-op second release when no code has changed since the previous release.

Pitfalls & gotchas

Auto-created tags point to the remote default branch tip, not your local HEAD. If you have local commits that you have not pushed, running `gh release create v1.0.0` without `--verify-tag` creates a tag on whatever the default branch is on GitHub at that moment — possibly not your intended commit. Always push commits before creating a release.

`--generate-notes` also sets the title if you omit `--title`. This is usually convenient, but if your merge commit titles are noisy, the auto-generated title may be unexpected. Pass `--title` explicitly to stay in control.

`--clobber` on upload is destructive if the upload fails. The command deletes the existing asset before uploading the replacement. If the network drops mid-upload, the original asset is gone with no automatic recovery. Have the source file confirmed ready before using this flag.

Downloading without a tag requires `--pattern` or `--archive`. Running `gh release download` with no tag and no filter flag errors out. Always specify at least one filter flag when omitting the tag.

`--cleanup-tag` on delete removes the git tag from the remote. Deleting a release without `--cleanup-tag` leaves the tag in place — other automation that watches tags may still trigger on it. Delete both together unless you intentionally want to preserve the tag for re-release.

Release immutability locks assets after publishing. Once a repository enables immutability and a release is published, `delete-asset` and `upload --clobber` will fail. Draft releases remain editable. Plan uploads carefully before publishing.

Worked examples

Full release workflow with auto-generated notes

Build and test locally, then create a tagged release with generated notes and built binaries in one step:

```
# Build the binaries
make dist

# Create checksums
shasum -a 256 dist/*.tar.gz > dist/checksums.txt

# Push the tag first
git tag -a v3.0.0 -m "Release v3.0.0"
git push origin v3.0.0

# Create the release
gh release create v3.0.0 \
  --title "v3.0.0" \
  --generate-notes \
  dist/*.tar.gz \
  dist/checksums.txt
```

<https://github.com/acme/myapp/releases/tag/v3.0.0>

Staged draft release with team review

Prepare the release while the team reviews the notes:

```
# Create as draft with release notes from a prepared file
gh release create v3.1.0 \
  --draft \
  --title "v3.1.0" \
  --notes-file CHANGELOG.md \
  dist/*.tar.gz

# Open the draft in a browser to share with the team
gh release view v3.1.0 --web

# After approval, publish
gh release edit v3.1.0 --draft=false
```

Backfilling a patch release without displacing “Latest”

You release v2.1.1 as a critical security fix for users on the v2.x line, but v3.0.0 is already the latest stable release. Publish v2.1.1 without moving the “Latest” badge:

```
git tag -a v2.1.1 -m "Security patch for v2.x line"
git push origin v2.1.1

gh release create v2.1.1 \
  --title "v2.1.1 (security patch)" \
  --notes "Fixes CVE-2026-12345. Upgrade recommended for all v2.x users." \
  --latest=false
```

Downloading binaries in a CI install script

```
# Download only the Linux tarball from the latest release into /tmp
gh release download \
  --repo acme/myapp \
  --pattern '*linux-amd64*.tar.gz' \
  --dir /tmp/myapp

tar -xf /tmp/myapp/*.tar.gz -C /usr/local/bin/
```

Verifying a downloaded asset with attestation

After downloading, confirm the asset is authentic before executing it:

```
gh release download v3.0.0 --pattern 'myapp-linux-amd64.tar.gz'
gh release verify-asset v3.0.0 myapp-linux-amd64.tar.gz
```

```
✓ Attestation verified for myapp-linux-amd64.tar.gz
```

Cleaning up a botched release

You released v3.1.0 with the wrong binaries before anyone downloaded them. Roll it back completely:

```
gh release delete v3.1.0 --cleanup-tag --yes
```

Then rebuild, push the corrected tag, and recreate the release as shown in the first worked example above.

Recovery

Released the wrong commit? Delete the release and tag, correct the tag locally, force-push it, and recreate:

```
gh release delete v3.1.0 --cleanup-tag --yes
git tag -d v3.1.0
git tag -a v3.1.0 <correct-sha> -m "Release v3.1.0"
git push origin v3.1.0 --force
gh release create v3.1.0 --title "v3.1.0" --generate-notes
```

Wrong asset uploaded and release not yet published? Replace the asset while it is still a draft:

```
gh release upload v3.1.0 dist/corrected-asset.tar.gz --clobber
```

Notes contain a typo after publishing? Edit the release in place; this does not require re-publishing:

```
gh release edit v3.1.0 --notes-file corrected-notes.md
```

Release is published and the repository is immutable? Assets cannot be replaced. Document the errata in the release notes via `gh release edit`, and consider creating a new patch release.

See *Getting out of jams* for broader recovery patterns including force-pushing tags and resetting git history.

See also

- *auth* — `gh release create` requires a token with `contents: write` scope; see *auth* for how to grant it.
- *repo* — repository settings control release immutability and the default branch used when auto-creating tags.
- *attestation* — `gh attestation` provides lower-level cryptographic attestation verification independent of a specific release.
- *run* — GitHub Actions workflows commonly trigger on `release` events; use `gh run` to inspect those triggered workflow runs.

gist

Create, view, edit, and manage GitHub Gists — lightweight, shareable snippets of code or text hosted at gist.github.com.

Mental model

A Gist is a minimal GitHub repository with exactly one commit history and one or more files. Unlike a full repository, it has no issues, pull requests, or branches — it is designed for sharing a self-contained snippet, a config file, or a quick script.

Every Gist lives at a stable URL and has a 32-character hex ID. That ID (or the full URL) is how `gh gist` subcommands address a specific gist:

```
https://gist.github.com/ada-lovelace/5b0e0062eb8e9654adad7bb1d81cc75f
                        | owner | ID
```

Gists are either **secret** (the default) or **public**. Secret means the URL is unlisted — anyone with the link can still view it. Public means the gist appears in your profile and in GitHub search results.

Because a gist is a git repository, you can clone it, push local changes back, and work with it in any git client. `gh gist clone` handles the URL bookkeeping for you.

Synopsis

```
gh gist list      [flags]
gh gist create    [<filename>... | <pattern>... | -] [flags]
gh gist view      [<id> | <url>] [flags]
gh gist edit      {<id> | <url>} [<filename>] [flags]
gh gist rename    {<id> | <url>} <old-filename> <new-filename> [flags]
gh gist clone     <gist> [<directory>] [-- <gitflags>...]
gh gist delete    {<id> | <url>} [flags]
```

A gist argument is accepted as a bare 32-character hex ID or as a full

`https://gist.github.com/OWNER/ID` URL.

Everyday usage

Listing your gists

```
gh gist list
```

5b0e0062eb8e9654adad7bb1d81cc75f	deploy.sh	1 file	secret
2024-11-03T09:12:00Z			
a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4	notes.md	2 files	public
2024-10-18T14:05:00Z			

Fetch more than the default ten entries:

```
gh gist list --limit 50
```

Filter by a regular expression across descriptions and filenames (add `--include-content` to also search inside file content):

```
gh gist list --filter deploy --include-content
```

Creating a gist

Share a single file publicly:

```
gh gist create --public deploy.sh
```

Create a secret gist with a description from multiple files:

```
gh gist create -d "advent of code 2024 solutions" day01.py day02.py
```

Pipe command output directly into a new gist and open it in the browser:

```
kubectl get pods -A | gh gist create - --filename k8s-pods.txt -w
```

Use a glob pattern:

```
gh gist create *.md *.txt
```

Viewing a gist

```
gh gist view 5b0e0062eb8e9654adad7bb1d81cc75f
```

List which files a gist contains without printing their content:

```
gh gist view 5b0e0062eb8e9654adad7bb1d81cc75f --files
```

Show a single file from a multi-file gist in raw form:

```
gh gist view 5b0e0062eb8e9654adad7bb1d81cc75f --filename deploy.sh --raw
```

Open a gist in the browser:

```
gh gist view 5b0e0062eb8e9654adad7bb1d81cc75f --web
```

Editing a gist

Open the gist in your `$EDITOR` interactively (prompts when no ID is given):

```
gh gist edit
```

Edit a specific file in an existing gist:

```
gh gist edit 5b0e0062eb8e9654adad7bb1d81cc75f --filename deploy.sh
```

Replace a gist file with the content of a local file (pass the local file as a positional argument after the ID):

```
gh gist edit 5b0e0062eb8e9654adad7bb1d81cc75f --filename deploy.sh deploy.sh
```

Add a new file to an existing gist:

```
gh gist edit 5b0e0062eb8e9654adad7bb1d81cc75f --add README.md
```

Remove a file from a gist:

```
gh gist edit 5b0e0062eb8e9654adad7bb1d81cc75f --remove old-file.py
```

Change the gist description:

```
gh gist edit 5b0e0062eb8e9654adad7bb1d81cc75f --desc "Updated deploy script"
```

Renaming a file inside a gist

```
gh gist rename 5b0e0062eb8e9654adad7bb1d81cc75f deploy.sh deploy-prod.sh
```

Cloning a gist locally

```
gh gist clone 5b0e0062eb8e9654adad7bb1d81cc75f
```

Clone into a specific directory:

```
gh gist clone 5b0e0062eb8e9654adad7bb1d81cc75f ~/scripts/deploy
```

Pass extra git flags after `--`:

```
gh gist clone 5b0e0062eb8e9654adad7bb1d81cc75f -- --depth 1
```

Deleting a gist

Interactive (prompts for selection and confirmation):

```
gh gist delete
```

Non-interactive, skip the confirmation prompt:

```
gh gist delete 5b0e0062eb8e9654adad7bb1d81cc75f --yes
```

Key options

create

Flag	What it does	When to use it
<code>-d</code> / <code>--desc</code>	Set a description	Always — makes <code>list</code> output useful
<code>-p</code> / <code>--public</code>	List the gist publicly (default is secret)	Sharing broadly; note it appears in search
<code>-f</code> / <code>--filename</code>	Override the filename when reading from stdin	stdin input has no natural filename

<code>-w / --web</code>	Open the created gist in a browser	Immediately share or copy the URL
-------------------------	------------------------------------	-----------------------------------

list

Flag	What it does	When to use it
<code>-L / --limit</code>	Maximum number of gists to fetch (default 10)	When you have more than ten gists
<code>--public</code>	Show only public gists	Auditing what you have made discoverable
<code>--secret</code>	Show only secret gists	Finding unlisted gists
<code>--filter</code>	Filter by regex on description and filenames (content requires <code>--include-content</code>)	Searching across many gists
<code>--include-content</code>	Include file content in regex filtering	Needed when the search term is inside a file

view

Flag	What it does	When to use it
<code>-f / --filename</code>	Show only one file from a multi-file gist	Focused reading
<code>--files</code>	List filenames only, no content	Seeing what files a gist contains
<code>-r / --raw</code>	Print raw content instead of rendered markdown	Piping output to other tools
<code>-w / --web</code>	Open in browser	Sharing or copying the URL

edit

Flag	What it does	When to use it
<code>-f / --filename</code>	Select which file to edit	Multi-file gists
<code>-d / --desc</code>	Change the gist description	After repurposing a gist
<code>-a / --add</code>	Add a new file to the gist	Growing a multi-file gist
<code>-r / --remove</code>	Remove a file from the gist	Cleaning up old files

delete

Flag	What it does	When to use it
<code>--yes</code>	Skip the confirmation prompt	Scripting or batch deletion

Best practices

Always supply `--desc`. The `gh gist list` table shows descriptions alongside IDs. A gist without a description shows a blank field, making it nearly impossible to find later among a list of hex IDs.

Default to secret; promote to public deliberately. GitHub offers no unpublish mechanism — once a gist is public, you cannot make it secret again short of deleting and recreating it. Start secret while drafting and create a new public gist when the content is ready to share.

Set a meaningful filename when piping stdin. GitHub renders syntax highlighting based on file extension. Without `--filename`, the file is stored as `gistfile1.txt` and gets no highlighting:

```
# Good: syntax highlighting, meaningful name in the list
git log --oneline | gh gist create - --filename recent-commits.txt
```

Clone for sustained editing. When you need to make multiple edits over time, `gh gist clone` gives you a local working copy with full git history. Edit locally, then `git push` to sync changes back:

```
gh gist clone 5b0e0062eb8e9654adad7bb1d81cc75f ~/gists/deploy
# edit files...
git add -A && git commit -m "Fix region variable" && git push
```

Use `--include-content` sparingly. Fetching file content for every gist is significantly slower and consumes more API rate-limit quota than the default metadata-only listing. Use it when you genuinely need to search inside files, not as a routine default.

Script deletions with `--yes` cautiously. There is no recycle bin. Deleting a gist is permanent. Review the list output before piping it into a delete command.

Pitfalls & gotchas

Secret gists are not private — they are unlisted. Anyone who has the URL can view a secret gist without authentication. Do not store credentials, API keys, or other sensitive data in any gist, secret or public.

You cannot make a public gist secret after creation. GitHub does not expose an endpoint for this. Your only option is to delete the original and create a new secret gist with the same content. Forks and embeds of the original persist on GitHub; you cannot fully retract a public gist once it has been visible.

`gh gist edit` without an ID opens `$EDITOR` interactively. If `$EDITOR` is not set or the terminal is non-interactive, the command may behave unexpectedly. Set `EDITOR` (or `VISUAL`) in your shell profile before relying on this flow.

Glob patterns expand on the client side. The shell expands `*.py` before `gh` sees the arguments. On a system where no `.py` files exist in the current directory, the command errors with “no matches found”. Verify your working directory before using patterns.

`gh gist list` shows only your own gists. There is no flag to list another user’s public gists. To inspect someone else’s gists, use `gh api /users/<username>/gists` (see *api*) or visit their GitHub profile.

Rate limits with `--include-content`. Each gist’s file content is fetched individually. Scanning a large collection with `--include-content` can quickly exhaust your API rate-limit quota. Reduce `--limit` or wait for the quota to reset (typically one hour) if you hit the limit.

Worked examples

Capturing a command's output as a shareable snippet

You ran a verbose build and want to share the log with a colleague without pasting walls of text into chat:

```
npm run build 2>&1 | gh gist create - \
  --filename build-log.txt \
  --desc "npm build output 2024-11-05 – failing on chunk size" \
  -w
```

```
- Creating gist build-log.txt...
✓ Created secret gist build-log.txt
Opening github.com/gist/ada-lovelace/a1b2c3d4... in your browser.
```

The `-w` flag opens the gist immediately so you can copy the URL from the address bar and paste it into chat.

Maintaining a living dotfile snippet

You keep a snippet of your `.zshrc` aliases as a public gist so you can quickly bootstrap new machines.

Initial creation:

```
gh gist create --public ~/.zshrc \
  --desc "Personal zsh aliases and functions"
```

Later, after updating your aliases locally, replace the gist file in one command:

```
gh gist edit <ID> --filename .zshrc ~/.zshrc
```

Or clone the gist once and use `git push` from then on:

```
gh gist clone <ID> ~/gists/zshrc
cd ~/gists/zshrc
# edit .zshrc here
git add .zshrc && git commit -m "Add git aliases" && git push
```

Finding a forgotten gist by content

You remember writing a bash retry function but cannot recall which gist it was in:

```
gh gist list --limit 100 --filter retry --include-content
```

```
5b0e0062eb8e9654adad7bb1d81cc75f  utils.sh
  shell utilities for CI pipelines
    function retry() {
```

Copy the ID from the first column, then view or clone the gist.

Batch-deleting gists in a script

Delete every secret gist whose description matches a pattern:

```
# Review first
gh gist list --secret --limit 200 --filter "temp"

# Then delete after confirming the list looks right
gh gist list --secret --limit 200 --filter "temp" \
| awk '{print $1}' \
| xargs -I{} gh gist delete {} --yes
```

Always run the listing step first and read its output before proceeding.

Recovery

Deleted gists cannot be restored through `gh` or the GitHub web interface. If the gist was cloned locally, the content is still in that clone's git history. Retrieve a previous version with:

```
cd ~/gists/mygist
git log --oneline
git checkout <hash> -- <filename>
git add <filename> && git commit -m "Restore previous version" && git push
```

If you need the URL to remain stable and want branching or access control, consider migrating the content to a full repository (see *repo*).

If gist commands fail with permission errors, the `gist` scope may not be granted on your token. Add it without logging out:

```
gh auth refresh --scopes gist
```

See *auth* for the full scope management workflow.

See also

- *auth* — the `gist` OAuth scope is required; use `gh auth refresh --scopes gist` if gist commands fail with permission errors.
- *repo* — use a full repository instead of a gist when you need branches, issues, pull requests, or fine-grained access control.
- *api* — use `gh api /users/<username>/gists` to list another user's public gists, or `/gists/public` for the global public feed.

attestation

Verify the integrity and provenance of artifacts, and download or prepare attestation bundles for offline use.

Mental model

An attestation is a cryptographically signed statement — a provenance claim — that ties an artifact (a binary, container image, or archive) to the GitHub Actions workflow that built it. The signature is anchored in Sigstore’s transparency log and in GitHub’s own Sigstore instance, so anyone with the public trust root can confirm that the artifact was produced by exactly the workflow, on exactly the repository or runner, at exactly the commit you expect.

Three concepts underpin every `attestation` command:

- **Subject.** The artifact being attested: a local file path or an OCI image URI (`oci://<image-uri>`).
- **Predicate.** The nature of the claim. The default predicate type is `https://slsa.dev/provenance/v1` (SLSA Provenance v1). Other predicate types can be verified with `--predicate-type`.
- **Actor identity.** The workflow and repository that signed the attestation. At minimum you must name the owning organization (`--owner`) or repository (`--repo`); for stronger guarantees you pin the exact workflow path with `--signer-workflow`.

Online verification (`verify`) reaches out to the GitHub API to fetch attestations automatically. Offline verification requires you to first `download` the attestation bundle and obtain a trusted root file via `trusted-root`, then pass both to `verify --bundle` and `verify --custom-trusted-root`.

Synopsis

```
gh attestation verify    [<file-path> | oci://<image-uri>] [--owner | --repo]
[flags]
gh attestation download [<file-path> | oci://<image-uri>] [--owner | --repo]
[flags]
gh attestation trusted-root [--tuf-url <url> --tuf-root <file-path>] [--verify-
only] [flags]
```

`gh` also accepts the alias `gh at` for `gh attestation`.

Everyday usage

Verify a local binary against its owning repository

```
gh attestation verify myapp-linux-amd64 --repo my-org/myapp
```

Verify using the owning organization (looser, still useful)

```
gh attestation verify myapp-linux-amd64 --owner my-org
```

Verify a container image

```
# You must be authenticated with the registry before running this.
gh attestation verify oci://ghcr.io/my-org/myapp:v1.2.3 --owner my-org
```

Download attestations for offline storage

```
gh attestation download myapp-linux-amd64 --repo my-org/myapp
# Writes: sha256:<digest>.jsonl (sha256-<digest>.jsonl on Windows)
```

Verify offline using a previously downloaded bundle

```
# Fetch the trusted root first (one-off, or refresh periodically)
gh attestation trusted-root > trusted_root.jsonl

# Then verify without network access
gh attestation verify myapp-linux-amd64 \
  --repo my-org/myapp \
  --bundle sha256:abc123.jsonl \
  --custom-trusted-root trusted_root.jsonl
```

Emit full JSON for policy evaluation

```
gh attestation verify myapp-linux-amd64 --owner my-org --format json
```

Key options

verify

Flag	What it does	When to use it
<code>-R / --repo</code>	Scope lookup to a specific <code><owner>/<repo></code>	Strongest identity check; use when you know the exact repo
<code>-o / --owner</code>	Scope lookup to a GitHub organization	When the repo is uncertain but the org is trusted
<code>--signer-workflow</code>	Require the signing workflow to match <code>[host/]<owner>/<repo>/<path>/<to>/<workflow></code>	Pin the exact build definition; recommended for production
<code>--signer-repo</code>	Require the signing workflow's repo to match <code><owner>/<repo></code>	Needed when the signer is a reusable workflow in a separate repo
<code>-b / --bundle</code>	Path to a local bundle file (JSON or JSON lines)	Offline verification
<code>--bundle-from-oci</code>	Fetch the bundle from the OCI registry instead of GitHub	When attestations are stored in-registry
<code>--custom-trusted-root</code>	Path to a <code>trusted_root.jsonl</code> file	Offline or air-gapped verification
<code>--cert-identity</code>	Require the certificate SAN to match this value exactly	Exact signer identity enforcement
<code>-i / --cert-identity-regex</code>	Require the certificate SAN to match this regex	Flexible signer identity enforcement

<code>--predicate-type</code>	Require attestations to use this predicate type	Verifying non-SLSA attestations (default: <code>https://slsa.dev/provenance/v1</code>)
<code>--deny-self-hosted-runners</code>	Fail if any attestation was produced on a self-hosted runner	Enforce GitHub-hosted runner policy
<code>-d / --digest-alg</code>	Digest algorithm: <code>sha256</code> or <code>sha512</code> (default <code>sha256</code>)	SHA-512 artifact digests
<code>-L / --limit</code>	Maximum number of attestations to fetch (default 30)	Large repositories with many attestations
<code>--format</code>	Output format: <code>json</code>	Downstream policy enforcement
<code>-q / --jq</code>	Filter JSON output with a jq expression	Quick field extraction
<code>--no-public-good</code>	Do not verify attestations signed with the Sigstore public good instance	Custom trust roots only
<code>--hostname</code>	Target a specific GitHub host	GitHub Enterprise Server

download

Flag	What it does	When to use it
<code>-R / --repo</code>	Scope lookup to <code><owner>/<repo></code>	Retrieve attestations for a specific repo
<code>-o / --owner</code>	Scope lookup to a GitHub organization	Retrieve all org-level attestations
<code>--predicate-type</code>	Filter downloaded bundles by predicate type	When only one attestation type is needed
<code>-L / --limit</code>	Maximum number of attestations to fetch (default 30)	Limit bundle file size
<code>-d / --digest-alg</code>	Digest algorithm: <code>sha256</code> or <code>sha512</code> (default <code>sha256</code>)	SHA-512 artifact digests
<code>--hostname</code>	Target a specific GitHub host	GitHub Enterprise Server

trusted-root

Flag	What it does	When to use it
<code>--tuf-url</code>	URL of a custom TUF repository mirror	Custom or enterprise Sigstore deployments
<code>--tuf-root</code>	Path to the TUF <code>root.json</code> obtained out-of-band	Required alongside <code>--tuf-url</code>
<code>--verify-only</code>	Check local TUF repository integrity without printing a bundle	Periodic health-check without output

<code>--hostname</code>	Target a specific GitHub host	GitHub Enterprise Server
-------------------------	-------------------------------	--------------------------

Best practices

Always pin the signer workflow for production gates. Specifying `--owner` alone trusts any workflow in any repository belonging to that organization. Specifying `--repo` narrows it to one repository. Adding `--signer-workflow` locks down the exact `.yaml` file. For a supply-chain security gate in CI, use all three in combination:

```
gh attestation verify dist/myapp \
  --repo my-org/myapp \
  --signer-workflow my-org/myapp/.github/workflows/release.yaml
```

Verify before deploying, not after. Run `gh attestation verify` as an explicit step in your deployment pipeline. A passing attestation check means the binary was produced by your controlled workflow from your controlled source — not a tampered artifact injected after the build.

Use `--deny-self-hosted-runners` in high-security environments. Self-hosted runners are harder to audit. GitHub-hosted runners run in ephemeral VMs with a known baseline. If your security policy requires ephemeral, GitHub-managed build environments, add this flag to your verification commands.

Store trusted roots alongside attestation bundles for air-gapped use. Run

`gh attestation trusted-root > trusted_root.jsonl` on a machine with internet access, then copy both the `.jsonl` bundle and `trusted_root.jsonl` into the air-gapped environment. The trusted root is tied to a specific TUF snapshot, so refresh it periodically (e.g., weekly) as the TUF repository rotates keys.

Pipe `--format json` output into a policy engine for custom enforcement. The JSON array returned by a successful verification contains `signature.certificate` and `verifiedTimestamps` fields whose contents cannot be forged by the signing workflow. Build your allow/deny logic on those fields. Be cautious when reading `statement.predicate` — that content is user-controlled and could be manipulated if the workflow were compromised.

Pitfalls & gotchas

`--owner` does not mean “any repo in the org is fine forever.” It means the attestation’s certificate must show an owner that matches the flag value. This is a good floor but a weak ceiling. Pair it with `--signer-workflow` to avoid accidentally accepting an attestation from a fork or a workflow you did not author.

Reusable workflows require `--signer-repo` or `--signer-workflow` pointed at the reusable workflow, not the caller. When a release is signed inside a reusable workflow hosted at `actions/slsa-goreleaser`, the certificate SAN names that reusable workflow, not your repo’s caller workflow. Using `--repo my-org/myapp` alone will fail. You need:

```
gh attestation verify dist/myapp \
  --owner my-org \
  --signer-repo actions/slsa-goreleaser
```

OCI verification requires prior registry authentication. `gh` does not handle registry login. Run `docker login ghcr.io` (or the equivalent for your registry) before passing `oci://` URIs to `verify` or `download`.

Bundle file names contain colons on non-Windows systems. The file written by `download` is named `sha256:<digest>.jsonl` on Linux/macOS. On Windows the colon is illegal and the file becomes `sha256-<digest>.jsonl`. Write scripts to handle both forms if they run cross-platform.

`download` is still in public preview and may change. The `--help` output notes this explicitly. Avoid hard-coding the output file format or naming convention in production pipelines until the subcommand reaches GA.

Attestations are fetched from the GitHub API by default — not from the artifact itself.

The artifact and its attestation are independent objects. If an artifact is copied to a new registry without its attestation being republished, `verify` will fail. Use `--bundle-from-oci` when the registry stores the attestation alongside the image.

Worked examples

End-to-end: build, attest, verify in a GitHub Actions release workflow

A typical release workflow produces a binary, generates an attestation via `actions/attest-build-provenance`, and then a deployment job verifies before rolling out.

```
jobs:
  release:
    permissions:
      id-token: write      # Required to mint the OIDC token for signing
      contents: write     # Required to upload release assets
      attestations: write # Required to create attestations
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: make dist/myapp
      - name: Attest build provenance
        uses: actions/attest-build-provenance@v1
        with:
          subject-path: dist/myapp

  deploy:
    needs: release
    steps:
      - name: Download release asset
        run: gh release download v1.2.3 --pattern "myapp" --repo my-org/myapp
      - name: Verify attestation
        env:
          GH_TOKEN: ${GITHUB_TOKEN}
        run: |
          gh attestation verify myapp \
            --repo my-org/myapp \
            --signer-workflow my-org/myapp/.github/workflows/release.yml
```

Offline verification in an air-gapped environment

On a machine with internet access, prepare the verification bundle:

```
# Download the attestation bundle
gh attestation download dist/myapp --repo my-org/myapp
# -> writes sha256:abc123def456.jsonl

# Fetch the current trusted root
gh attestation trusted-root > trusted_root.jsonl
```

Transfer `sha256:abc123def456.jsonl`, `trusted_root.jsonl`, and `dist/myapp` to the air-gapped machine. Then verify:

```
gh attestation verify dist/myapp \
  --repo my-org/myapp \
  --bundle sha256:abc123def456.jsonl \
  --custom-trusted-root trusted_root.jsonl
```

```
Loaded digest sha256:abc123def456... for file:///dist/myapp
Loaded 1 attestation from the bundle
✓ Verification succeeded!

sha256:abc123def456...
  - Attestation verified using a bundle with 1 attestation(s).
```

Extracting certificate fields for policy decisions

After verifying, extract the source repository and ref from the certificate to enforce a “tagged release only” policy:

```
gh attestation verify dist/myapp --owner my-org --format json | \
jq '.[].verificationResult.signature.certificate | {
  sourceRepository: .extensions.sourceRepository,
  sourceRef:       .extensions.sourceRepositoryRef
}''
```

```
{
  "sourceRepository": "https://github.com/my-org/myapp",
  "sourceRef":       "refs/tags/v1.2.3"
}
```

Fail the deployment if the ref is not a tag:

```
ref=$(gh attestation verify dist/myapp --owner my-org --format json \
| jq -r '.[0].verificationResult.signature.certificate.extensions.sourceRepositoryRef')

if [[ "$ref" != refs/tags/* ]]; then
  echo "ERROR: artifact was not built from a tagged release (got: $ref)" >&2
  exit 1
fi
```

Recovery

If verification fails with “no attestations found”, the artifact may not have been attested, or you may have specified the wrong `--owner` / `--repo`. Check that the repository name matches exactly, then try broadening to `--owner` to rule out a repository naming mismatch.

If verification fails with a certificate identity error, a reusable workflow is likely the actual signer. Inspect the raw attestation to find the true SAN:

```
gh attestation download dist/myapp --repo my-org/myapp  
jq '[0].bundle.verificationMaterial.certificate.rawBytes' sha256:*.jsonl
```

Decode the base64-encoded certificate and look at the SAN extension to determine the correct value for `--signer-workflow` or `--signer-repo`.

If the trusted root becomes stale and offline verification fails with a timestamp or certificate error, refresh the trusted root on a connected machine and re-transfer it:

```
gh attestation trusted-root > trusted_root.jsonl
```

See also

- *auth* — `gh auth login` is required before calling the GitHub API; set `GH_TOKEN` for CI use.
- *release* — `gh release download` retrieves the artifacts you then pass to `gh attestation verify`.
- *run* — `gh run view` lets you trace which Actions run produced a given artifact before verifying its attestation.

secret

Create, update, list, and delete encrypted secrets for use in GitHub Actions, Dependabot, Codespaces, and Agents — at the repository, environment, organization, or user level.

Mental model

A GitHub secret is a named, encrypted value that GitHub injects into a workflow run (or Codespaces environment) at execution time. You never see the raw value again after storing it; GitHub stores only the encrypted form, and the runner decrypts it in memory just before the step that needs it.

Secrets live at four levels:

- **Repository** — available to Actions runs, Agents sessions, and Dependabot inside one repository. This is the default level for `gh secret set`.
- **Environment** — scoped to a named deployment environment inside a repository (e.g., `production`, `staging`). Useful for environment-specific credentials that should only be visible to jobs targeting that environment.
- **Organization** — shared across repositories in an organization. Visibility can be `all` (every repo), `private` (private repos only), or `selected` (an explicit allowlist).
- **User** — available to Codespaces running under your personal account.

`gh secret set` encrypts the value locally using the repository's (or org's) public key before sending it over the wire. GitHub never receives the plaintext.

Synopsis

```
gh secret set    <secret-name> [flags]
gh secret list   [flags]
gh secret delete <secret-name> [flags]
```

`gh secret ls` is an alias for `gh secret list`. `gh secret remove` is an alias for `gh secret delete`.

Everyday usage

Setting a secret interactively

Omit `--body` and `gh` prompts you to paste the value:

```
gh secret set DATABASE_URL
```

Setting a secret from a shell variable

```
gh secret set API_KEY --body "$MY_API_KEY"
```

The value never touches your shell history as long as it is already in the variable.

Reading the secret value from a file

```
gh secret set TLS_CERT < cert.pem
```

Bulk-loading secrets from a `.env` file

```
gh secret set -f .env
```

Each `KEY=value` line in the file becomes a separate secret. Lines starting with `#` are ignored. Pass `-f -` to read from stdin instead.

Setting an environment-scoped secret

```
gh secret set DB_PASS --env production
```

Only jobs that target the `production` deployment environment can read this secret.

Setting an organization secret

```
# Visible to all repositories in the org
gh secret set SHARED_TOKEN --org myOrg --visibility all

# Visible only to specific repositories
gh secret set DEPLOY_KEY --org myOrg --repos api,frontend,infra

# Stored but accessible to no repository (useful for reserving a name)
gh secret set PLACEHOLDER --org myOrg --no-repos-selected
```

Setting a secret for a specific application

```
gh secret set NPM_TOKEN --app dependabot
gh secret set CODESPACE_TOKEN --user
```

Listing secrets in the current repository

```
gh secret list
```

NAME	UPDATED
API_KEY	about 2 hours ago
DATABASE_URL	about 1 day ago
TLS_CERT	about 3 days ago

Secret values are never shown; only names and timestamps are returned.

Listing secrets in JSON format

```
gh secret list --json name,updatedAt,visibility
```

Listing secrets for a deployment environment

```
gh secret list --env production
```

Listing organization secrets

```
gh secret list --org myOrg
```

Deleting a secret

```
gh secret delete OLD_API_KEY
```

Deleting an environment or org secret

```
gh secret delete STAGING_DB --env staging
gh secret delete LEGACY_TOKEN --org myOrg
```

Key options

set

Flag	What it does	When to use it
<code>-b / --body string</code>	Supply the value directly	Scripting; avoids the interactive prompt
<code>-f / --env-file file</code>	Load from a dotenv file	Bulk-importing secrets
<code>-e / --env environment</code>	Target a deployment environment	Environment-specific credentials
<code>-o / --org organization</code>	Target an organization	Shared secrets across repos
<code>-u / --user</code>	Target your user account	Personal Codespaces secrets
<code>-a / --app string</code>	Target a specific app (actions, agents, codespaces, dependabot)	App-scoped secrets
<code>-r / --repos repositories</code>	List of repositories that can access an organization or user secret	Explicit allowlist for org or user secrets
<code>-v / --visibility string</code>	all, private, or selected (default private)	Org-level visibility control
<code>--no-repos-selected</code>	Block all repos from accessing the org secret	Parking a name without exposure
<code>--no-store</code>	Print the encrypted base64 value instead of storing	Debugging the encryption path
<code>-R / --repo</code>	Target a different repository	Cross-repo secret management

list

Flag	What it does	When to use it
<code>-e / --env string</code>	List secrets for an environment	Environment-scoped audit
<code>-o / --org string</code>	List org secrets	Organization-level audit
<code>-u / --user</code>	List user secrets	Personal Codespaces audit
<code>-a / --app string</code>	Filter by application	App-specific listing
<code>--json fields</code>	Output JSON with chosen fields	Scripting and automation
<code>-q / --jq expression</code>	Filter JSON with a jq expression	Inline JSON processing

<code>-t / --template string</code>	Format JSON with a Go template	Custom output
-------------------------------------	--------------------------------	---------------

delete

Flag	What it does	When to use it
<code>-e / --env string</code>	Delete from a deployment environment	Environment cleanup
<code>-o / --org string</code>	Delete from an organization	Org-level secret removal
<code>-u / --user</code>	Delete from your user account	Personal Codespaces cleanup
<code>-a / --app string</code>	Target a specific application	App-scoped delete

Best practices

Never pass secret values as positional arguments or in command history. Use

`--body "$VAR"` (from a variable already in memory), a redirected file (`< secret.txt`), or the interactive prompt. Commands written directly into shell history can be recovered by teammates or attackers with access to that machine.

Use environment-scoped secrets for deployment credentials. Putting production database passwords in repository secrets means any Actions job in the repo can read them, even on feature branches. Environment secrets require a job to explicitly target `environment: production`, and you can add required reviewers or branch filters to the environment.

Prefer organization secrets for credentials shared across many repos. Updating one org secret propagates to every authorized repository automatically. Maintaining the same secret name in thirty individual repositories guarantees they will drift out of sync.

Rotate secrets through `gh secret set` without downtime. `set` is an upsert — it overwrites the existing value in place. Queue a new value with `gh secret set` before the old credential expires; workflows already in progress use the version that was loaded when the run started.

Audit secrets regularly with `gh secret list --json name,updatedAt`. A secret that has not been rotated in months is a liability. Pipe the output through `jq` to find stale entries:

```
gh secret list --json name,updatedAt \
| jq '.[ ] | select(.updatedAt < "2024-01-01") | .name'
```

Use `--app dependabot` for Dependabot-specific secrets. Actions secrets and Dependabot secrets are separate namespaces because Dependabot runs in a different execution context. A secret set without `--app` is not automatically available to Dependabot.

Pitfalls & gotchas

Values are write-only — you can never read them back. `gh secret list` returns only names and timestamps. There is no `gh secret get`. Store the plaintext value in a password manager before setting it; if you lose it, you must generate a new credential and overwrite the secret.

--body in your shell history. If you type `gh secret set F00 --body supersecret` directly into a terminal, the value lands in `.bash_history` or `.zsh_history`. Use a variable or the interactive prompt instead:

```
# Risky – value ends up in shell history
gh secret set F00 --body "supersecret"

# Safe – value comes from a variable already set outside of history
gh secret set F00 --body "$SECRET_VALUE"
```

.env file syntax is strict. `gh secret set -f .env` expects lines in the form `KEY=value`. Quoted values, `export KEY=value` prefixes, and multi-line values may not parse as expected. Test with a small file and verify immediately with `gh secret list`.

Environment secrets require the environment to exist first. If the deployment environment `staging` does not exist in the repository settings, `gh secret set --env staging` will fail. Create the environment in the GitHub UI or via `gh api` before setting secrets against it.

Organization secret visibility defaults to private. Running `gh secret set --org myOrg` without `--visibility` restricts the secret to private repositories. If your target repo is public, add `--visibility all` explicitly.

--repos implies selected visibility for org secrets. Passing `--repos` automatically targets only those repositories. If you later want to widen access, re-run `gh secret set` with a new `--repos` list or a different `--visibility` value.

Deleting a secret that is in active use silently breaks workflows. There is no confirmation prompt before delete. Confirm with `gh secret list` and search the repository's workflow files before removing a secret that workflows depend on.

Worked examples

Rotating a production database password

The old password is expiring. Generate a new one, store it, then verify:

```
# Generate a new password
NEW_PASS=$(openssl rand -base64 32)

# Overwrite the existing secret in place
gh secret set DB_PASSWORD --env production --body "$NEW_PASS"

# Confirm the timestamp updated
gh secret list --env production
```

NAME	UPDATED
DB_PASSWORD	less than a minute ago

Store `$NEW_PASS` in your team password manager, then update the database to accept the new credential before the old one expires.

Importing a full set of secrets from a file into a new repository

You have a `.env.ci` file with all the secrets a workflow needs. Import them in one step:

```
gh secret set -f .env.ci --repo myOrg/new-repo
```

Then confirm everything arrived:

```
gh secret list --repo myOrg/new-repo
```

Sharing a deployment key across several repositories in an org

```
# Allow exactly three repos to read DEPLOY_KEY
gh secret set DEPLOY_KEY \
  --org myOrg \
  --repos api,frontend,infra \
  --body "$DEPLOY_KEY_VALUE"

# Later, extend the allowlist by re-running set with the full updated list
gh secret set DEPLOY_KEY \
  --org myOrg \
  --repos api,frontend,infra,docs \
  --body "$DEPLOY_KEY_VALUE"
```

Auditing and cleaning up stale secrets

List all repo secrets with timestamps and find ones that have not been rotated recently:

```
gh secret list --json name,updatedAt \
  | jq -r '.[[] | [.name, .updatedAt] | @tsv'
```

Remove a secret that is no longer referenced in any workflow:

```
gh secret delete LEGACY_DEPLOY_TOKEN
```

Dry-run: inspect the encrypted payload without storing

```
gh secret set TEST_SECRET --body "hello" --no-store
```

```
QIAJAFq8...base64-encoded-ciphertext...==
```

This is useful when debugging a custom tool that handles secret encryption before submitting via the API.

Recovery

Secrets cannot be retrieved after they are set. If a secret value is lost:

1. Generate a new credential from the upstream service (API provider, cloud console, etc.).
2. Overwrite the old secret: `gh secret set <NAME> --body "$NEW_VALUE"`.
3. Update any dependent systems (databases, external services) to accept the new credential.

If a secret was deleted by mistake and workflows are failing, re-create it immediately:

```
gh secret set DELETED_SECRET --body "$RECOVERED_VALUE"
```

Workflows that failed due to the missing secret can be re-run from the GitHub Actions UI once the secret is restored.

If credentials may have been exposed (for example, accidentally echoed in a workflow log or committed to a branch), treat it as a leak: revoke the credential at the source immediately, generate a replacement, and rotate via `gh secret set` before any further workflow runs.

See also

- *auth* — authenticate `gh` before managing secrets.
- *variable* — `gh variable` manages non-secret, plaintext configuration values in Actions; use secrets for anything sensitive.
- *run* — `gh run` triggers and inspects the workflows that consume secrets.
- *workflow* — `gh workflow` lists and enables the workflows that reference secrets.

variable

Manage plaintext configuration variables for GitHub Actions and Dependabot at the repository, environment, or organization level.

Mental model

Variables are named key-value pairs stored on GitHub that Actions workflows and Dependabot jobs can read at runtime via `${{ vars.MY_VAR }}`. They differ from secrets in one important way: their values are visible in the GitHub UI and in API responses. Use variables for configuration that is not sensitive — build flags, version pins, feature toggle names, deployment region strings — and keep passwords, tokens, and keys in secrets (see *secret*).

Variables exist at three levels, each with its own scope:

- **Repository** — visible to all Actions runs and Dependabot jobs inside that specific repository.
- **Environment** — visible only to runs that target a named deployment environment (e.g., `production`, `staging`) in a specific repository.
- **Organization** — visible to runs inside member repositories; an organization variable can be restricted to selected repositories or opened to all repositories in the org.

When a variable name exists at multiple levels, the most specific level wins: environment overrides repository, repository overrides organization.

Synopsis

```
gh variable list    [-e env] [-o org] [-R repo] [--json fields]
gh variable get     <variable-name> [-e env] [-o org] [-R repo] [--json fields]
gh variable set     <variable-name> [-b body] [-e env] [-f env-file]
                   [-o org] [-r repos] [-v visibility] [-R repo]
gh variable delete  <variable-name> [-e env] [-o org] [-R repo]
```

Everyday usage

List all variables on the current repository:

```
gh variable list
```

List variables for a deployment environment:

```
gh variable list --env production
```

List variables for an organization:

```
gh variable list --org my-org
```

Get the value of a single variable:

```
gh variable get DEPLOY_REGION
```

Set a variable interactively (gh prompts for the value):

```
gh variable set APP_VERSION
```

Set a variable non-interactively by passing the value directly:

```
gh variable set APP_VERSION --body "2.4.1"
```

Read the value from a shell variable to avoid it appearing in shell history:

```
gh variable set APP_VERSION --body "$APP_VERSION"
```

Read the value from a file:

```
gh variable set SOME_CONFIG < config.txt
```

Bulk-import variables from a dotenv file:

```
gh variable set -f .env
```

Set a variable scoped to a deployment environment:

```
gh variable set DEPLOY_REGION --env staging
```

Set an organization-level variable visible to all repositories:

```
gh variable set NODE_VERSION --org my-org --visibility all
```

Restrict an organization variable to specific repositories:

```
gh variable set NODE_VERSION --org my-org --repos repo-a,repo-b,repo-c
```

Delete a variable:

```
gh variable delete APP_VERSION
```

Key options

list / get

Flag	What it does	When to use it
-e / --env	Target a deployment environment	Inspect per-environment config
-o / --org	Target an organization	Inspect org-level variables
-R / --repo	Target a different repository	When not inside the target repo
--json fields	Output JSON with named fields	Scripting and automation
-q / --jq	Filter JSON with a jq expression	Extract specific values inline
-t / --template	Format JSON with a Go template	Custom tabular output

Available JSON fields for both commands: `createdAt`, `name`, `numSelectedRepos`, `selectedReposURL`, `updatedAt`, `value`, `visibility`.

set

Flag	What it does	When to use it
<code>-b / --body</code>	Supply the variable value as a string	Non-interactive scripts
<code>-f / --env-file</code>	Load names and values from a dotenv file	Bulk import
<code>-e / --env</code>	Set a deployment environment variable	Per-environment config
<code>-o / --org</code>	Set an organization variable	Shared config across repos
<code>-r / --repos</code>	Repositories that can access an org variable	Scoped org variables
<code>-v / --visibility</code>	Org variable visibility: <code>all</code> , <code>private</code> , <code>selected</code> (default: <code>private</code>)	Control org-wide exposure
<code>-R / --repo</code>	Target a different repository	When not inside the target repo

delete

Flag	What it does	When to use it
<code>-e / --env</code>	Delete from a deployment environment	Remove env-scoped variable
<code>-o / --org</code>	Delete from an organization	Remove org-level variable
<code>-R / --repo</code>	Target a different repository	When not inside the target repo

Best practices

Keep sensitive values out of variables. Because variable values are visible in the UI and in JSON API responses, never store tokens, passwords, certificates, or private keys as variables. Use *secret* for those instead.

Use `--body "$SHELL_VAR"` in automation. Running `gh variable set F00` without `--body` opens an interactive prompt, which hangs in CI. Always supply the value explicitly in scripts. Expanding from a shell variable keeps the literal value out of the command string that appears in process listings and build logs:

```
gh variable set BUILD_FLAGS --body "$CI_BUILD_FLAGS"
```

Bulk-import from `.env` with `--env-file`. For repositories that already maintain a dotenv file for local development, you can promote the non-secret values to GitHub in one step:

```
gh variable set -f .env.production
```

Lines starting with `#` are treated as comments and skipped. Quoted values are unquoted automatically.

Use `--visibility selected` for organization variables. Granting access to all repositories in an org (`--visibility all`) can expose build config to repositories whose maintainers do not expect it. Start with `selected` and name the repositories explicitly, then broaden only when needed.

Check the active scope before setting. A repository variable with the same name as an org variable silently shadows the org variable for that repo. Run `gh variable list` at both levels to spot collisions before they cause subtle workflow failures:

```
gh variable list           # repository level
gh variable list --org my-org # organization level
```

Pitfalls & gotchas

Variables are not secrets. Values appear in plain text in `gh variable list` output and in the GitHub web UI. Anyone with read access to the repository (or to the org's settings) can see them. If you accidentally set a secret as a variable, delete it immediately with `gh variable delete` and add it as a secret instead.

--body is required in non-interactive environments. Without `--body` (and without stdin being a pipe), `gh variable set` opens an interactive prompt and the command hangs in CI. Always use `--body` in scripts.

Visibility flag only applies to organization variables. Passing `--visibility` without `--org` is an error. The flag has no meaning at the repository or environment level.

The --repos flag implicitly sets visibility to selected. When you supply `--repos`, the GitHub API requires `visibility=selected` the `gh` CLI handles this automatically, so you do not need to pass `--visibility selected` explicitly. You can still pass it explicitly for clarity, but omitting it is not an error:

```
gh variable set NODE_VERSION \
  --org my-org \
  --repos api-service,web-frontend
```

Environment variables set with --env are scoped to one repository. There is no org-level concept of environments; `--env` and `--org` are mutually exclusive.

`gh variable list alias` is `gh variable ls`. Both work identically. Similarly, `gh variable delete` can be invoked as `gh variable remove`.

Worked examples

Standardizing the Node.js version across a monorepo

Your organization runs several repositories that all need to stay on the same Node.js version. Store it as an org variable so workflows can reference `${{ vars.NODE_VERSION }}` without each repo pinning its own copy.

```
# Set it once at the org level, restricted to backend repositories
gh variable set NODE_VERSION \
  --body "20.14.0" \
  --org my-org \
  --visibility selected \
  --repos api-service,worker,jobs
```

Verify:

```
gh variable list --org my-org
```

NAME	VALUE	UPDATED
NODE_VERSION	20.14.0	about 1 minute ago

When the LTS version bumps, update in one place:

```
gh variable set NODE_VERSION --body "22.3.0" --org my-org
```

Per-environment deployment targets

A repository deploys to different AWS regions depending on the environment. Store the region as an environment variable so each environment points to the right place.

```
gh variable set AWS_REGION --body "eu-west-1" --env staging
gh variable set AWS_REGION --body "eu-north-1" --env production
```

Inspect both:

```
gh variable list --env staging
gh variable list --env production
```

```
# staging
NAME      VALUE      UPDATED
AWS_REGION eu-west-1  about 1 minute ago

# production
NAME      VALUE      UPDATED
AWS_REGION eu-north-1 about 1 minute ago
```

In the workflow, the correct region is selected automatically based on which environment the job targets:

```
jobs:
  deploy:
    environment: production
    steps:
      - run: echo "Deploying to ${vars.AWS_REGION}"
```

Bulk-importing configuration from a dotenv file

A repository keeps non-secret build configuration in `.env.ci`:

```
# .env.ci
BUILD_TARGET=linux-amd64
CACHE_BUCKET=my-build-cache
LOG_LEVEL=info
```

Import all three variables at once:

```
gh variable set -f .env.ci
```

Then verify:

```
gh variable list
```

NAME	VALUE	UPDATED
BUILD_TARGET	linux-amd64	about 1 minute ago
CACHE_BUCKET	my-build-cache	about 1 minute ago
LOG_LEVEL	info	about 1 minute ago

Extracting a variable value in a script

Use `--json` and `--jq` to pull a single value into a shell variable without parsing table output:

```
region=$(gh variable get AWS_REGION --env production --json value --jq .value)
echo "Deploying to $region"
```

```
Deploying to eu-north-1
```

Recovery

Accidentally set a sensitive value as a variable — delete it immediately, then create it as a secret:

```
gh variable delete MY_API_KEY
echo "$MY_API_KEY" | gh secret set MY_API_KEY
```

Wrong value set — overwrite it by running `gh variable set` again with the correct value. There is no separate “update” subcommand; `set` creates or replaces:

```
gh variable set APP_VERSION --body "2.5.0"
```

Variable not visible to a workflow — check that the variable exists at the right level (`--env`, `--org`, or repository) and that no variable at a more specific level is shadowing it. Use `gh variable list` at each level to compare.

Organization variable not accessible to a repository — if visibility is `selected`, the repository must be explicitly listed. Add it with:

```
gh variable set NODE_VERSION \
  --org my-org \
  --visibility selected \
  --repos existing-repo,new-repo
```

See also

- *secret* — encrypted secrets for sensitive values; the companion to variables.
- *auth* — managing the credentials that allow `gh variable` to call the GitHub API.
- *run* — `gh run` and `gh workflow` are where variables surfaced via `vars.*` context are consumed.
- *workflow* — managing the Actions workflow files that reference variables.

ruleset

Inspect and audit the repository protection rules that GitHub enforces on branches and tags.

Mental model

A GitHub **ruleset** is a named policy attached to a repository or organisation that tells GitHub which operations are allowed or forbidden on matching refs. Unlike the older branch-protection settings, rulesets can target multiple branches at once (using `fnmatch` patterns), can be layered — a repository inherits rulesets from its organisation and enterprise — and are version-controlled inside GitHub itself.

Rules can require pull requests before merging, mandate status checks, restrict who may push or delete a branch, enforce signed commits, and more. The `gh ruleset` command group lets you **read** that configuration: list the rulesets that exist, view the details of any single ruleset, and check which rules would fire against a specific branch name before you push.

`gh ruleset` is a read-only surface. Creating, editing, or deleting rulesets is done through the GitHub web UI or the REST API (see *api*).

Synopsis

```
gh ruleset list    [flags]
gh ruleset view    [<ruleset-id>] [flags]
gh ruleset check   [<branch>] [flags]
```

`gh rs` is an alias for `gh ruleset`.

Everyday usage

List rulesets for the current repository

```
gh ruleset list
```

ID	NAME	SOURCE	STATUS	RULES
12	require-pr-and-ci	my-org/backend	active	6
34	protect-release-branches	my-org/backend	active	3
78	org-wide-signing-policy	my-org	active	1

The `SOURCE` column shows whether a ruleset is defined directly on the repository or inherited from an organisation (or enterprise).

List only rulesets defined on the repository itself

Pass `--parents=false` to suppress inherited rulesets:

```
gh ruleset list --parents=false
```

List organisation-wide rulesets

```
gh ruleset list --org my-org
```

Your token needs the `admin:org` scope for this. Grant it with `gh auth refresh -s admin:org` (see *auth*).

View a single ruleset

```
gh ruleset view 34
```

If you omit the ID, `gh` shows an interactive prompt to pick from the rulesets that apply to the current repository.

Open the ruleset in the browser to read its full configuration:

```
gh ruleset view 34 --web
```

Check which rules apply to a branch

```
gh ruleset check my-feature-branch
```

No argument uses the current branch. Pass `--default` to check the repository's default branch regardless of which branch is checked out:

```
gh ruleset check --default
```

Open the equivalent rules page in the browser:

```
gh ruleset check --web
```

Key options

list

Flag	What it does	When to use it
<code>-L / --limit int</code>	Maximum rulesets to return (default 30)	Large organisations with many rulesets
<code>-o / --org string</code>	List organisation-wide rulesets	Auditing policies at the org level
<code>-p / --parents</code>	Include inherited rulesets (default true)	Pass <code>false</code> to see only repo-local rules
<code>-w / --web</code>	Open the rulesets list in the browser	Visual inspection
<code>-R / --repo</code>	Target a different repository	Cross-repo auditing

view

Flag	What it does	When to use it
<code>-o / --org string</code>	Indicate the ruleset ID belongs to an org	Viewing organisation-level rulesets by ID
<code>-p / --parents</code>	Include rulesets configured at higher levels that also apply (default true)	Pass <code>--parents=false</code> to see only repo-local rules

<code>-w / --web</code>	Open the ruleset in the browser	Reading full rule details
<code>-R / --repo</code>	Target a different repository	Cross-repo inspection

check

Flag	What it does	When to use it
<code>--default</code>	Check rules on the default branch	CI scripts that should not depend on the checked-out branch
<code>-w / --web</code>	Open the branch rules page in the browser	Visual walkthrough of what applies
<code>-R / --repo</code>	Target a different repository	Checking rules before opening a PR into another repo

Best practices

Run `gh ruleset check` before pushing to an unfamiliar branch. Rules are evaluated server-side when you push, so a failed push is the first sign something is blocked. Running the check locally first tells you exactly which requirements must be satisfied — required status checks, signed commits, a pull request — before the push can succeed.

```
gh ruleset check # check the branch you are about to push to
```

Use `--parents=false` when auditing repository-local policy. The default includes inherited organisation and enterprise rulesets, which can make the list noisy. When you are reviewing only what your team owns, suppress the parents:

```
gh ruleset list --parents=false
```

Audit org-wide rulesets periodically. Policies defined at the organisation level apply silently to every repository in the org. Running `gh ruleset list --org <org>` across your organisation gives you a single view of what every repository is subject to, independent of per-repository settings.

Open in `--web` when you need the full rule payload. The terminal output of `gh ruleset view` shows the ruleset summary. The browser view exposes the complete rule list, enforcement level, bypass actors, and the target branch or tag patterns — information not fully surfaced in the CLI output.

Pitfalls & gotchas

The branch name passed to `check` does not have to exist. The command evaluates pattern matching against whatever string you supply, which is useful for planning: you can verify that a branch named `release/2.0` would be protected before you create it. Conversely, if you spell the branch name wrong you will get back the rules for the typo'd name, not for your real branch — no warning is issued.

`--org` on `list` requires elevated token scope. Without `admin:org` your request is rejected. The error message references permission denied but does not always name the missing scope. Fix it with:

```
gh auth refresh -s admin:org
```

Inherited rulesets have IDs that are org-scoped, not repo-scoped. When you see a ruleset in `gh ruleset list` that comes from an organisation, its ID belongs to the organisation. Passing that ID to `gh ruleset view` without `--org <org>` may return “not found” because `gh` searches the repository’s own ruleset namespace. Use `--org` to resolve it:

```
gh ruleset view 78 --org my-org
```

ruleset is read-only. There is no `gh ruleset create` or `gh ruleset edit`. Attempting to manage ruleset configuration with `gh ruleset` alone is not possible; use the web UI or `gh api` with the rulesets REST endpoints.

gh ruleset check returns all matching rules, not just blocking ones. A ruleset can be in `evaluate` mode (logging only, not enforcing). The check output does not distinguish enforced rules from evaluate-mode rules, so a rule appearing in the output does not guarantee it will block a push.

Worked examples

Pre-push rules audit

You are about to push commits to `main` on a repository you have not contributed to before. Check what rules apply before touching the remote:

```
gh ruleset check main --repo owner/backend
```

```
Total rules: 4
- Branch is not deletable.
- Cannot force push.
- Requires linear history.
- Must pass status checks: ci/build, ci/test
```

The output tells you that a linear history is required. You squash or rebase your commits before pushing:

```
git rebase origin/main
git push origin main
```

Auditing organisation policy inheritance

You are the platform engineer responsible for a GitHub organisation with dozens of repositories. You want to confirm that the org-wide signing requirement is in place:

```
gh ruleset list --org my-org
```

ID	NAME	SOURCE	STATUS	RULES
78	org-wide-signing-policy	my-org	active	1
91	require-pr-all-repos	my-org	active	3

View the signing ruleset in detail:

```
gh ruleset view 78 --org my-org
```

```
Name: org-wide-signing-policy
ID: 78
Source: my-org (Organization)
Enforcement: active
Target: ~DEFAULT_BRANCH

Rules
- Require signed commits
```

Now verify that a specific repository inherits it:

```
gh ruleset list --repo my-org/payments-service
```

The `78` entry appears in the list with `SOURCE` set to `my-org`, confirming the repository is covered.

Checking rules before opening a PR into a protected branch

Your team uses a `release/*` glob to protect release branches. Before creating `release/3.1`, confirm what rules that name would trigger:

```
gh ruleset check release/3.1 --repo my-org/backend
```

If required status checks appear in the output, you know any PR targeting this branch must have those checks green before it can merge.

Recovery

If a push is rejected by a ruleset with a message like

`GH013: Repository rule violations found`, run `gh ruleset check` on the target branch to see exactly which rules fired:

```
gh ruleset check <branch>
```

Address each requirement — add a PR, get required checks to pass, rewrite history to be linear, sign commits — then retry the push.

If you believe a ruleset is misconfigured (wrong target pattern, wrong enforcement level), open it in the browser for editing:

```
gh ruleset view <id> --web
```

Changes to rulesets take effect immediately with no cache delay; the next push attempt will reflect the updated policy.

See also

- `auth` — grant the `admin:org` scope needed by `gh ruleset list --org`.

- *api* — create, update, or delete rulesets via the REST API when the read-only CLI is insufficient.
- *pr* — pull requests are one of the most common requirements enforced by rulesets.
- *run* — required status checks referenced by rulesets correspond to workflow runs managed with `gh run`.

ssh-key

List, add, and delete the SSH keys registered with your GitHub account.

Mental model

GitHub stores public SSH keys against your account so that `git push` and `git clone` over SSH can prove who you are without a password. Think of it as a lock (GitHub holds the lock, you hold the private key). `gh ssh-key` is the toolbelt for managing those locks: you can see which keys are registered, add a new one, or remove one that is no longer in use.

Two key types exist on GitHub. An **authentication key** is used for `git` operations — the classic use-case. A **signing key** is used to cryptographically sign commits and tags (the “Verified” badge on GitHub). A single public key can be registered for one purpose but not both; if you want the same physical key to do both jobs you must upload it twice, once as each type.

`gh ssh-key` talks directly to the GitHub API, so the changes take effect immediately — no browser login required. The subcommands mirror the three things you need to do: `list` to see, `add` to register, `delete` to remove.

Synopsis

```
gh ssh-key list
gh ssh-key add  [<key-file>] [-t title] [--type authentication|signing]
gh ssh-key delete <id>          [-y]
```

Everyday usage

Listing registered keys

Show every SSH key on your account, including its ID, title, and type:

```
gh ssh-key list
```

ID	TITLE	KEY	CREATED
12345678	Work MacBook	ssh-ed25519 AAAAC3...	2024-01-10
12345679	Personal M3	ssh-ed25519 AAAAC3...	2024-03-22

The ID column is what you pass to `delete`.

Adding a key

Upload your default public key and give it a recognisable title:

```
gh ssh-key add ~/.ssh/id_ed25519.pub --title "Work MacBook"
```

If you omit the file argument, `gh` reads from stdin — useful when piping:

```
cat ~/.ssh/id_ed25519.pub | gh ssh-key add --title "Work MacBook"
```

Register the same key as a signing key (to sign commits):

```
gh ssh-key add ~/.ssh/id_ed25519.pub --title "Work MacBook signing" --type signing
```

Deleting a key

Find the ID with `list`, then delete it. The command asks for confirmation by default:

```
gh ssh-key delete 12345678
```

```
Are you sure you want to delete key 12345678 (Work MacBook)? (y/N)
```

Skip the prompt in scripts with `--yes`:

```
gh ssh-key delete 12345678 --yes
```

Key options

add

Flag	What it does	When to use it
<code>-t / --title</code>	Label the key on GitHub	Always — names make <code>list</code> output readable
<code>--type</code>	authentication (default) or signing	When uploading a key exclusively for commit signing

delete

Flag	What it does	When to use it
<code>-y / --yes</code>	Skip the confirmation prompt	Automation and scripts

list

`list` has no flags beyond `--help`. It has the alias `ls`:

```
gh ssh-key ls
```

Best practices

Give keys descriptive titles. The default `id_ed25519` title tells you nothing six months later. Include the machine and, if useful, the purpose — `Work MacBook signing`, `CI runner deploy key`. Good titles make it obvious which row to delete when a machine is decommissioned.

Register signing keys separately. Even if your authentication and signing keys are the same physical key pair, GitHub requires a separate entry for each type. Keep the titles consistent — `Personal M3 auth` and `Personal M3 signing` — so they pair up visually in `gh ssh-key list`.

Audit keys periodically. Run `gh ssh-key list` every few months and delete keys from machines you no longer use. Each key is a potential entry point; a decommissioned laptop's key should not remain active.

Prefer `--yes` only in automation. The confirmation prompt on `delete` exists to prevent accidents. In CI or scripts where you have already validated the ID, passing `--yes` is fine. Do not use it interactively out of habit.

Generate the key locally before uploading. `gh ssh-key add` only *registers* a public key — it does not generate one for you. Generate the key pair first:

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

Then upload the public half with `gh ssh-key add`.

Pitfalls & gotchas

Uploading the private key by mistake. The `add` subcommand expects a public key (the file ending in `.pub`). If you accidentally pass the private key file the command will reject it with an error, but double-check before running:

```
# Wrong — private key
gh ssh-key add ~/.ssh/id_ed25519

# Correct — public key
gh ssh-key add ~/.ssh/id_ed25519.pub
```

A key registered as authentication cannot serve as a signing key, and vice versa.

GitHub enforces the type at registration time. If you need both, upload the same public key twice with different `--type` values and different titles. Attempting to use an authentication key for commit signing will produce a signing key not found error on the GitHub side.

`list` shows no output when no keys are registered. The command exits cleanly with an empty table. If you expected to see keys and the output is blank, check that you are authenticated as the correct account — see *auth*.

IDs are numeric, not fingerprints. `gh ssh-key delete` takes the integer ID shown in `gh ssh-key list`, not the key fingerprint from `ssh-keygen -lf`. Confirm the ID from `list` before deleting.

`gh ssh-key add` with stdin requires the key on a single line. Multi-line input or extra whitespace will cause the API to reject the key. If you are constructing the key string programmatically, strip trailing newlines before piping.

Worked examples

Setting up a new machine from scratch

Generate a key, upload it, and verify it is registered:

```
# Generate a new Ed25519 key pair
ssh-keygen -t ed25519 -C "ada@example.com" -f ~/.ssh/id_ed25519

# Add to the SSH agent
ssh-add ~/.ssh/id_ed25519

# Upload the public key to GitHub
gh ssh-key add ~/.ssh/id_ed25519.pub --title "Ada's MacBook Pro"

# Confirm it appears
gh ssh-key list
```

ID	TITLE	KEY	CREATED
87654321	Ada's MacBook Pro	ssh-ed25519 AAAAC3...	2026-06-17

Test the connection:

```
ssh -T git@github.com
```

Hi ada-lovelace! You've successfully authenticated, but GitHub does not provide shell access.

Enabling verified commit signing

Register the same key for signing (GitHub requires a separate entry):

```
gh ssh-key add ~/.ssh/id_ed25519.pub \
  --title "Ada's MacBook Pro signing" \
  --type signing
```

Then configure git locally to sign commits with that key:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
```

Now commits pushed from this machine will show the “Verified” badge on GitHub.

Rotating a compromised key

If a private key is exposed, remove the corresponding public key from GitHub immediately:

```
# Find the ID of the compromised key
gh ssh-key list
```

ID	TITLE	KEY	CREATED
11111111	Old Laptop	ssh-ed25519 AAAAB3...	2023-09-01
22222222	Work MacBook	ssh-ed25519 AAAAC3...	2025-01-15

```
# Delete without waiting for a prompt
gh ssh-key delete 11111111 --yes
```

Generate a fresh key pair on your current machine and upload it:

```
ssh-keygen -t ed25519 -C "ada@example.com" -f ~/.ssh/id_ed25519_new
gh ssh-key add ~/.ssh/id_ed25519_new.pub --title "Work MacBook (rotated 2026-06)"
```

Update any deployment targets or CI runners that used the old key.

Removing stale keys in a script

Fetch the list of key IDs matching a known title pattern and delete each one:

```
# List all keys, filter by title, extract ID, delete
gh ssh-key list | grep "Old CI runner" | awk '{print $1}' | while read id; do
  gh ssh-key delete "$id" --yes
done
```

Recovery

Deleted a key by accident? There is no undo — `gh ssh-key delete` is permanent. Re-add the public key file if you still have it:

```
gh ssh-key add ~/.ssh/id_ed25519.pub --title "Work MacBook"
```

If the key is gone from disk as well, generate a new pair with `ssh-keygen`, upload the public half with `gh ssh-key add`, update your `~/.ssh/config` or run `ssh-add`, and test with `ssh -T git@github.com`.

For authentication failures after a key rotation, see *Getting out of jams*.

See also

- *auth* — log in to GitHub and configure the active account that `gh ssh-key` acts on.
- *gpg-key* — manage GPG keys for commit signing as an alternative to SSH signing keys.
- *config* — set default hosts and other gh-wide settings.

gpg-key

Manage the GPG keys registered with your GitHub account — list what is there, add a new public key, and delete one you no longer need.

Mental model

GitHub uses GPG keys to verify signed commits and tags. When you sign a commit with `git commit -S`, Git attaches your GPG signature to the object. Anyone who fetches that commit can check the signature against the public key GitHub holds on file for your account; if they match, GitHub displays the green **Verified** badge in the UI.

`gh gpg-key` is the command-line shortcut to the “SSH and GPG keys” section of your GitHub account settings. The three subcommands mirror the three operations available there:

`list` reads what is stored, `add` uploads a new public key, and `delete` removes one by its numeric ID.

The keys managed here are **account-level** — they belong to your GitHub user profile, not to any single repository. Every repository you push signed commits to will verify against the same set of stored keys.

Synopsis

```
gh gpg-key list
gh gpg-key add  [<key-file>] [-t <title>]
gh gpg-key delete <key-id>  [-y]
```

Everyday usage

List all GPG keys currently registered on your account:

```
gh gpg-key list
```

Upload a new GPG public-key file:

```
gh gpg-key add ~/.gnupg/my-key.pub --title "Personal laptop 2024"
```

Read the public key from stdin instead of a file:

```
gpg --armor --export alice@example.com | gh gpg-key add --title "alice work key"
```

Delete a key by its numeric ID (get the ID from `gh gpg-key list`):

```
gh gpg-key delete 12345678
```

Skip the confirmation prompt when scripting:

```
gh gpg-key delete 12345678 --yes
```

Key options

add

Flag	What it does	When to use it
[<key-file>]	Path to the ASCII-armored public-key file; omit to read from stdin	Use a file path for one-off uploads; stdin for piping from <code>gpg --export</code>
-t / --title	Human-readable label for the key on GitHub	Always set this — it is the only way to distinguish keys in the list

delete

Flag	What it does	When to use it
<key-id>	Numeric ID of the key to remove (required)	Obtain it from <code>gh gpg-key list</code>
-y / --yes	Skip the interactive confirmation prompt	Automation and CI scripts

list

`gh gpg-key list` (alias: `gh gpg-key ls`) has no additional flags. It prints each key's ID, title, key ID fingerprint, and creation date.

Best practices

Always supply `--title` when adding a key. GitHub shows the title in the settings UI and in `gh gpg-key list` output. Without a descriptive title, rows like "Added 2024-01-15" are indistinguishable from each other once you have more than one key — you will not know which ID to delete when a key is retired.

Export only the public key, never the private key. The standard GPG export for uploading is:

```
gpg --armor --export <fingerprint-or-email>
```

The `--armor` flag produces ASCII output that `gh gpg-key add` expects. Never run `gpg --export-secret-keys` and pipe that to GitHub.

Pair this with `git commit -S` configuration. Uploading a key to GitHub has no effect until Git is actually signing commits. Configure Git to sign automatically once the key is in place:

```
git config --global user.signingkey <fingerprint>
git config --global commit.gpgSign true
```

Rotate keys on a schedule. GPG subkeys can be set to expire. When a subkey expires, remove the old entry with `gh gpg-key delete` and upload the renewed public key (after running `gpg --send-keys` to your keyserver). This keeps the number of active keys small and verifiable.

Use `--yes` only in automation. The confirmation prompt on `delete` exists to prevent accidental removal. In a CI context where you are scripting key rotation, pass `--yes` so the script does not hang waiting for input.

Pitfalls & gotchas

`gh gpg-key list` returns the **GitHub key ID**, not the **GPG fingerprint**. The numeric ID printed by `list` (e.g., 12345678) is GitHub's internal identifier for the uploaded record. It is not the 40-character GPG fingerprint you see in `gpg --list-keys`. Always run `gh gpg-key list` to get the correct ID before running `gh gpg-key delete`.

Uploading the wrong key (private vs. public) will fail. The API validates that the submitted key is a valid OpenPGP public-key block. If you accidentally pipe a secret key export, GitHub will reject the request with an error message.

A key can be uploaded but still not verify commits if the email does not match. GitHub links a GPG key to the email addresses embedded in that key. If the email in your GPG user ID does not match a verified email address on your GitHub account, commits signed with that key will show **Unverified** even after upload. Run `gpg --edit-key <fingerprint>` to inspect the UIDs and add a matching email if needed.

Deleting a key does not un-verify historical commits. Once a commit was verified against a key, that verification is stored. Removing the key from your account affects future verification checks, not past ones.

`gh gpg-key add` reads stdin only when no file argument is given. If you accidentally supply both a file path and a pipe, the file path takes precedence and the piped data is ignored without a warning.

Worked examples

Adding a freshly generated GPG key

Generate a new key, export the public half, and upload it in one pipeline:

```
# Generate a key (interactive – choose RSA 4096 and your email)
gpg --full-generate-key

# Find the fingerprint of the new key
gpg --list-secret-keys --keyid-format LONG

# Export and upload in one step
gpg --armor --export 3AA5C34371567BD2 | gh gpg-key add --title "MacBook Pro 2025"
```

Confirm the upload worked:

```
gh gpg-key list
```

ID	Title	Key ID	Created
12345678	MacBook Pro 2025	3AA5C34371567BD2	2025-03-10

Now configure Git to sign all commits automatically:

```
git config --global user.signingkey 3AA5C34371567BD2
git config --global commit.gpgSign true
```

Make a signed commit and check the GitHub UI for the green **Verified** badge.

Rotating an expired key

Your GPG subkey has expired. Generate a new one, then swap the GitHub record:

```
# Extend or create a new subkey in GPG
gpg --edit-key 3AA5C34371567BD2
# Inside gpg> prompt: addkey, then save

# Re-export the full public key (includes the new subkey)
gpg --armor --export 3AA5C34371567BD2 | gh gpg-key add --title "MacBook Pro 2025 (renewed)"

# List keys to see both old and new entries
gh gpg-key list
```

ID	Title	Key ID	Created
12345678	MacBook Pro 2025	3AA5C34371567BD2	2025-03-10
23456789	MacBook Pro 2025 (renewed)	3AA5C34371567BD2	2025-11-01

Delete the old entry:

```
gh gpg-key delete 12345678
```

```
! Are you sure you want to delete GPG key 12345678? (y/N)
```

Type `y` to confirm, or pass `--yes` to skip the prompt.

Auditing and cleaning up stale keys

List every key currently on file:

```
gh gpg-key list
```

Cross-reference with the local keyring:

```
gpg --list-secret-keys --keyid-format LONG
```

For any key ID that no longer has a corresponding private key on this machine (decommissioned laptop, revoked key), delete it:

```
gh gpg-key delete 99887766 --yes
```

Recovery

If you delete a key by mistake, re-add it from your local keyring immediately:

```
gpg --armor --export <fingerprint> | gh gpg-key add --title "Restored key"
```

If you have lost the private key entirely and cannot regenerate the public key, you cannot restore the GitHub entry. Future commits will need to be signed with a new key — generate one, upload it, and update `user.signingkey` in your Git config.

If signed commits are showing **Unverified** after adding a key, verify that the email address embedded in the GPG key UID is listed as a verified email in your GitHub account settings. See *auth* for how to manage the GitHub account that `gh` is authenticated as, in case you are uploading the key to the wrong account.

See also

- *auth* — manage the GitHub account `gh` acts on; ensure you are logged in as the correct user before adding or deleting keys.
- *ssh-key* — the parallel command group for SSH authentication keys; GPG keys are for commit signing, SSH keys are for repository access.
- *config* — configure `gh` behavior and defaults beyond authentication.

org

List and browse the GitHub organizations your authenticated account belongs to.

Mental model

GitHub organizations are shared accounts where teams collaborate on repositories under a common namespace. Your personal account can belong to many organizations — open-source projects, employers, client accounts — each with its own member list, repositories, and billing.

`gh org` is a focused command group. It does not administer membership, settings, or billing through the CLI; those operations belong in the GitHub web UI or the REST API (see *api*). What `gh org list` gives you is a fast, scriptable answer to “which orgs can I act as?” — useful when you are switching contexts, writing automation, or building prompts for other `gh` subcommands that require an org name.

The results reflect the account that is currently **active** on a given host. If you are authenticated as multiple GitHub accounts, `gh org list` shows the organizations for whichever account is active. Use `gh auth switch` to change the active account before running `gh org list` when you need the org list for a different identity.

Synopsis

```
gh org list [--limit <max>]
gh org ls   [--limit <max>]    # alias
```

Everyday usage

List the organizations your active account belongs to (up to 30):

```
gh org list
```

Raise the cap to see more than 30 organizations:

```
gh org list --limit 100
```

Capture the org names for use in a script:

```
gh org list --limit 200 | while read -r org; do
  echo "Processing $org"
  # gh repo list "$org" ...
done
```

Check which orgs are available under a specific account without permanently switching the active account:

```
# Switch, inspect, switch back
gh auth switch --user work-user
gh org list
gh auth switch --user personal-user
```

Key options

list

Flag	What it does	When to use it
<code>-L / --limit</code>	Maximum number of organizations to list (default: 30)	When you belong to more than 30 orgs, or want to cap output in scripts

Best practices

Always confirm the active account before scripting. `gh org list` is only as useful as the account it reflects. Run `gh auth status --active` first, or embed the switch inside your script, so you never accidentally operate on the wrong org set.

Use `--limit` defensively in automation. The default cap of 30 is appropriate for interactive inspection. Automation that iterates over all orgs should pass an explicit `--limit` large enough to cover the real count — otherwise it silently processes only the first 30 and drops the rest:

```
gh org list --limit 500 | sort
```

Pipe into other `gh` commands to scope operations. `gh org list` is most powerful as a source of org names for subsequent commands. Combining it with `gh repo list <org>` or `gh api` calls lets you write org-wide automation without hard-coding names:

```
for org in $(gh org list --limit 200); do
  echo "=== $org ==="
  gh repo list "$org" --limit 5
done
```

Prefer the `gh org ls` alias in interactive shells to save keystrokes. Use the full `gh org list` spelling in scripts so the intent is self-documenting.

Pitfalls & gotchas

The 30-organization default limit silently truncates. If your account belongs to 40 organizations and you run `gh org list` without `--limit`, you see only 30. There is no warning. Scripts that assume the list is complete will miss organizations without notice.

Results are tied to the active account, not the current repository. Unlike many `gh` commands, `gh org list` does not infer an owner from the current directory's git remote. It lists orgs for the active account on the target host regardless of where you run it.

Organization membership visibility may be restricted. GitHub allows organization members to hide their membership publicly. `gh org list` returns the orgs the authenticated account genuinely belongs to, which may differ from what an external observer can see on your public profile.

Enterprise Server hosts need explicit account switching. If you are authenticated against a GitHub Enterprise Server instance as well as `github.com`, `gh org list` only queries the host associated with the active account. To list orgs on the Enterprise host, switch to that account first:

```
gh auth switch --hostname github.mycompany.com
gh org list
```

Worked examples

Auditing all organizations you belong to

You want a quick inventory before starting a cross-org search.

```
gh auth status --active
```

```
github.com
✓ Logged in to github.com account ada-lovelace (keyring)
- Active account: true
```

```
gh org list --limit 200
```

```
acme-corp
open-widgets
my-oss-collective
```

You now have a list you can feed into other commands or share with your team.

Listing recent repositories across every org you belong to

You want to find recently pushed repositories without knowing which org owns each one.

```
gh org list --limit 200 | while read -r org; do
  gh repo list "$org" --limit 5 --json name,pushedAt \
    --jq ".[] | \"$org/\"(.name) \"(.pushedAt)\""
done | sort -k2 -r | head -20
```

This pipes every org name into `gh repo list`, extracts the five most-recently pushed repos in each, and sorts by push date to surface the most active repositories across all your organizations.

Switching accounts and comparing org memberships

You manage a personal account and a work account on `github.com` and need to confirm which org is accessible from which account.

```
gh auth switch --user personal-user
gh org list
```

```
my-oss-collective
open-widgets
```

```
gh auth switch --user work-user
gh org list
```

```
acme-corp  
acme-internal
```

Now you know exactly which account to use when running commands against each organization. See *auth* for full details on managing multiple accounts.

Recovery

`gh org list` is a read-only command — it cannot modify anything, so there is nothing to undo.

If the command returns an empty list or an authentication error, the most likely cause is an expired or insufficient token. Refresh credentials with:

```
gh auth refresh
```

If you need the `read:org` scope explicitly (required to see private org membership), add it:

```
gh auth refresh --scopes read:org
```

Then retry `gh org list`.

See also

- *auth* — manage accounts and switch the active identity before listing orgs.
- *repo* — `gh repo list <org>` lists repositories inside an organization.
- *api* — perform org-level administration (members, settings, webhooks) via the REST and GraphQL APIs that `gh org list` does not expose.
- *search* — `gh search repos --owner <org>` searches public repositories within a specific organization.

api

Make authenticated HTTP requests to the GitHub REST and GraphQL APIs directly from the terminal, without leaving your workflow to open a browser or reach for `curl`.

Mental model

`gh api` is a thin, authenticated wrapper around GitHub's HTTP API. When you run it, `gh` injects the stored OAuth token (or the value of `GH_TOKEN`) as the `Authorization` header and sends the request to `api.github.com` — or a GitHub Enterprise Server host if you specify one.

The command exposes both API flavors:

- **REST (v3)** — Pass a path such as `repos/{owner}/{repo}/issues`. The HTTP method defaults to `GET` adding field flags (`-f`, `-F`) switches it to `POST`. Override the method explicitly with `-X`.
- **GraphQL (v4)** — Use the special endpoint `graphql` and supply your query via `-f query='...'`. Pagination, variable injection, and the `--slurp` helper all work with GraphQL too.

The curly-brace placeholders `{owner}`, `{repo}`, and `{branch}` are substituted automatically from the repository in the current directory or from the `GH_REPO` environment variable. This lets you write portable scripts that work in any repo.

Response bodies are printed as-is (usually JSON). Two flags narrow the output without requiring a separate `jq` installation: `--jq` applies a `jq` expression inline, and `--template` renders a Go template.

Synopsis

```
gh api <endpoint> [flags]
gh api graphql    [flags]
```

Everyday usage

Fetch a repository's metadata (uses placeholders):

```
gh api repos/{owner}/{repo}
```

List the open issues in the current repo:

```
gh api repos/{owner}/{repo}/issues
```

Post a comment on issue #42:

```
gh api repos/{owner}/{repo}/issues/42/comments \
-f body='Looks good to me!'
```

Run a GraphQL query:

```
gh api graphql -f query='
  query {
    viewer {
```

```

    login
    name
  }
}

```

Print only the login field with `--jq`:

```
gh api /user --jq '.login'
```

Fetch all pages of a paginated REST endpoint and collect them into one array:

```
gh api repos/{owner}/{repo}/issues --paginate --slurp
```

Key options

Option	What it does	When to use it
<code>-X / --method</code>	Override the HTTP method (GET , POST , PATCH , PUT , DELETE)	Any non-default verb
<code>-f / --raw-field</code>	Add a string parameter as <code>key=value</code>	Simple string request body fields
<code>-F / --field</code>	Add a typed parameter; converts <code>true / false / null / integers</code> to JSON; <code>@file</code> reads from a file	Non-string fields, nested params, file content
<code>-H / --header</code>	Add a custom HTTP request header as <code>key:value</code>	Custom <code>Accept</code> headers, API versioning
<code>--input</code>	Read the request body from a file (use <code>-</code> for stdin)	Pre-built JSON payloads
<code>-q / --jq</code>	Filter/transform the response with a jq expression	Extracting specific fields
<code>-t / --template</code>	Format the response with a Go template	Tabular or custom output
<code>--paginate</code>	Fetch all pages of results automatically	Complete data sets
<code>--slurp</code>	Wrap all paginated pages into a single outer JSON array	Use together with <code>--paginate</code>
<code>-p / --preview</code>	Opt into a named GitHub API preview (names must omit the <code>-preview</code> suffix, e.g. <code>baptiste</code> not <code>baptiste-preview</code> repeat the flag or comma-separate for multiple)	Experimental endpoints
<code>--cache</code>	Cache the response for a duration (e.g. <code>60m</code> , <code>1h</code>)	Repeated calls to rate-limited endpoints
<code>-i / --include</code>	Print the HTTP status line and response headers	Debugging

<code>--verbose</code>	Print the full request and response, including headers	Deep debugging
<code>--silent</code>	Suppress the response body	Fire-and-forget writes
<code>--hostname</code>	Override the target host (default: <code>github.com</code>)	GitHub Enterprise Server

Typed fields with `-F`

The `-F` flag inspects the value and converts it before sending:

Value form	What <code>gh</code> sends
<code>true / false</code>	JSON boolean
<code>null</code>	JSON null
Integer string	JSON number
<code>@path/to/file</code>	Contents of that file
<code>@-</code>	Contents of stdin
<code>{owner}</code> , <code>{repo}</code> , <code>{branch}</code>	Substituted from the current repo

Nested and array parameters

Use bracket syntax to build nested objects and arrays:

```
# Nested object
gh api some/endpoint -F 'parent[child]=value'

# Array – repeat the flag
gh api some/endpoint -F 'tags[]=bug' -F 'tags[]=help-wanted'

# Empty array
gh api some/endpoint -F 'tags[]'
```

GraphQL-specific behaviour

For graphql requests, every field other than `query` and `operationName` is treated as a GraphQL variable. The `--paginate` flag requires the query to accept an `$endCursor: String` variable and to fetch `pageInfo { hasNextPage endCursor }`.

Best practices

Use placeholders instead of hard-coding owner and repo. Scripts written with `{owner}` and `{repo}` work in any checkout without modification and are safe to share:

```
# Portable – works in any repo
gh api repos/{owner}/{repo}/releases

# Fragile – breaks when copied to another repo
gh api repos/acme/backend/releases
```

Prefer `-F` over `-f` for non-string values. Sending the string `"true"` where the API expects a boolean causes a 422 Unprocessable Entity error. Use `-F` and let `gh` handle the conversion:

```
# Wrong – sends the string "true"
gh api repos/{owner}/{repo} -X PATCH -f has_wiki=true

# Correct – sends JSON boolean true
gh api repos/{owner}/{repo} -X PATCH -F has_wiki=true
```

Use `--jq` to avoid a `jq` dependency. The filter runs inside `gh` itself, so your scripts work even where `jq` is not installed:

```
gh api repos/{owner}/{repo}/tags --jq '.[].name'
```

Cache read-heavy calls. The REST API allows 5 000 authenticated requests per hour. Polling in a loop or in CI quickly burns through that budget. Add `--cache` for any idempotent read:

```
gh api repos/{owner}/{repo} --cache 300s
```

Use `--input` for large or complex payloads. Building a deeply nested JSON body with `-F` flags is error-prone. Write the JSON to a file and pass it with `--input`. Note: when `--input` is used, any `-f` / `-F` flags are appended as query-string parameters rather than merged into the body:

```
gh api repos/{owner}/{repo}/rulesets --input ruleset.json
```

Set `GH_TOKEN` in CI rather than relying on stored credentials. In GitHub Actions the `GITHUB_TOKEN` secret is already available. Map it to `GH_TOKEN` and `gh api` will use it without any stored credential. See *auth* for details on environment-variable authentication:

```
- name: Close a stale issue
  env:
    GH_TOKEN: ${GITHUB_TOKEN}
  run: gh api repos/{owner}/{repo}/issues/99 -X PATCH -F state=closed
```

Pitfalls & gotchas

The HTTP method defaults to `POST` when you add any fields. If you intend to pass parameters as a `GET` query string, set `-X GET` explicitly:

```
# Without -X GET this would incorrectly POST instead of search
gh api -X GET search/issues -f q='repo:cli/cli is:open label:bug'
```

Shells may expand `{...}` before `gh` sees it. PowerShell and some zsh configurations treat curly braces as brace-expansion or glob characters. Quote the argument:

```
gh api "repos/{owner}/{repo}/issues"
```

`--paginate` returns each page as a separate JSON document, not one merged result. Add `--slurp` to wrap all pages into a single array you can process as a whole:

```
# Without --slurp: multiple JSON arrays printed back-to-back
gh api repos/{owner}/{repo}/issues --paginate

# With --slurp: one outer array containing every item
gh api repos/{owner}/{repo}/issues --paginate --slurp
```

GraphQL pagination requires the query to cooperate. If your query does not fetch `pageInfo { hasNextPage endCursor }` and accept `$endCursor: String`, `--paginate` silently returns only the first page without any warning.

--verbose, --silent, --template, and --jq are mutually exclusive. Only one of these output-control flags may be used at a time; combining any two causes an error. Use `--verbose` for debugging, `--silent` for fire-and-forget writes, and `--jq` / `--template` for formatted output.

Rate-limit errors return HTTP 403 or 429 with a JSON body. Check the remaining quota before a long-running loop:

```
gh api rate_limit --jq '.rate | {limit, remaining, reset}'
```

Worked examples

Listing and filtering releases

Fetch all release tag names from the current repo and print one per line:

```
gh api repos/{owner}/{repo}/releases --jq '.[].tag_name'
```

```
v2.4.0
v2.3.1
v2.3.0
v2.2.0
```

Limit to the three most recent:

```
gh api repos/{owner}/{repo}/releases --jq '.[0:3][].tag_name'
```

Creating a label via the REST API

```
gh api repos/{owner}/{repo}/labels \
  -f name='needs-triage' \
  -f color='e4e669' \
  -f description='Waiting for a maintainer to review'
```

```
{
  "id": 4028982537,
  "name": "needs-triage",
  "color": "e4e669",
  "description": "Waiting for a maintainer to review",
  ...
}
```

Bulk-closing issues with `--paginate`

Close every open issue labelled `wontfix` in the current repository. Fetch all matching issue numbers with one paginated search call, then loop:

```
gh api -X GET search/issues \
  -f q='repo:{owner}/{repo} is:open label:wontfix' \
  --paginate \
  --jq '.items[].number' \
  | while read -r number; do
    gh api repos/{owner}/{repo}/issues/"$number" \
      -X PATCH -F state=closed --silent
    echo "Closed #$number"
  done
```

Querying the GraphQL API with pagination

List every repository the authenticated user owns, paginating automatically, and collect the results into one JSON array:

```
gh api graphql --paginate --slurp -f query='
query($endCursor: String) {
  viewer {
    repositories(first: 100, after: $endCursor) {
      nodes { nameWithOwner isPrivate }
      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
}'
' | jq '[[.].data.viewer.repositories.nodes[]]'
```

Tabular output with `--template`

Print the number, title, and state of the last five closed pull requests in a tidy table:

```
gh api repos/{owner}/{repo}/pulls \
  -X GET \
  -f state=closed \
  -f per_page=5 \
  --template '{{range .}}{{tablerow .number .title .state}}{{end}}'
```

```
47 Fix memory leak in parser      closed
46 Add dark-mode support          closed
45 Bump dependencies              closed
44 Remove deprecated endpoint     closed
43 Initial CI setup               closed
```

Debugging a failed request

Add `--verbose` to see exactly what `gh` sent and what the server returned:

```
gh api repos/{owner}/{repo}/nonexistent --verbose
```

```
* Request at 2024-11-01 10:32:01.123 +0000 UTC
> GET /repos/acme/backend/nonexistent HTTP/1.1
> Host: api.github.com
> Authorization: token ghp_...
< HTTP/1.1 404 Not Found
{
  "message": "Not Found",
  "documentation_url": "https://docs.github.com/rest"
}
```

Recovery

If a request fails with **401 Unauthorized**, the stored token has expired or been revoked. Refresh credentials with `gh auth refresh` or log in again with `gh auth login`. See *auth* for the full credential-management workflow.

If you hit a **403 or 429 rate-limit** error, check remaining quota and the reset time:

```
gh api rate_limit --jq '.rate | {limit, remaining, reset}'
```

The `reset` value is a Unix timestamp. Wait until that time or switch to a different token with its own quota.

If a write operation partially succeeded — for example, some labels were created before the script was interrupted — use `--include` on a subsequent call to inspect the server state, then issue the corrective `PATCH` or `DELETE` requests to bring the resource to the intended state.

See also

- *auth* — managing the OAuth token that `gh api` injects automatically into every request.
- *config* — setting a default hostname or adjusting the git credential helper.
- *search* — `gh search` uses the REST search endpoint under the hood;
`gh api -X GET search/...` gives the same data with full field control.
- *run* and *workflow* — `gh api` can trigger or inspect Actions runs via the REST API when the built-in subcommands do not expose the exact endpoint you need.

extension

Extend the GitHub CLI with community-built commands that integrate directly into the `gh` command surface.

Mental model

The `gh` CLI ships with a fixed set of built-in commands, but the ecosystem adds dozens more through **extensions**: ordinary Git repositories whose name starts with `gh-` and whose root contains an executable of the same name. When you run `gh <extname>`, the CLI finds the matching extension and forwards all remaining arguments to that executable.

Extensions live in `~/.local/share/gh/extensions/` (or the platform equivalent). Installing one is a one-step operation — `gh extension install owner/gh-extname` — and from that moment `gh extname` becomes a first-class command on your machine.

There are two kinds of extensions:

- **Script extensions** — a shell script or any interpreted executable committed directly to the repository root. `gh` clones the repo and runs the script.
- **Precompiled (binary) extensions** — the repository ships compiled binaries as GitHub release assets. `gh` downloads the appropriate binary for your OS and architecture. Precompiled extensions start faster and have no runtime dependency requirements.

An extension cannot override a built-in `gh` command. If a naming conflict arises — say you install `gh-label` and `gh label` already exists — use `gh extension exec label` to invoke the extension version.

`gh` checks for new extension versions at most once every 24 hours and prints an upgrade notice when one is available. Set the environment variable `GH_NO_EXTENSION_UPDATE_NOTIFIER=1` to suppress these notices.

Synopsis

```
gh extension browse
gh extension create [<name>] [--precompiled=go|other]
gh extension exec  <name> [args]
gh extension install <repository> [--pin <ref>] [--force]
gh extension list
gh extension remove <name>
gh extension search [<query>] [flags]
gh extension upgrade {<name> | --all} [--dry-run] [--force]
```

Aliases: `gh ext`, `gh extensions`

Everyday usage

Find extensions to install:

```
gh extension search
```

Install an extension:

```
gh extension install dlvdhr/gh-dash
```

List what is currently installed:

```
gh extension list
```

Upgrade a single extension:

```
gh extension upgrade gh-dash
```

Upgrade everything at once:

```
gh extension upgrade --all
```

Remove an extension you no longer need:

```
gh extension remove gh-dash
```

Discovering extensions with browse

`browse` opens a full-screen terminal UI for finding, installing, and removing extensions:

```
gh extension browse
```

Press `?` inside the TUI to see the keyboard shortcuts. Press `q` to quit. The TUI requires a terminal at least 100 columns wide; on a narrower terminal (or when using a screen reader) pass `--single-column`:

```
gh extension browse --single-column
```

Searching from the command line

`search` queries the GitHub API for repositories tagged as `gh-extension`. With no arguments it returns the top 30 by star count:

```
gh extension search
gh extension search branch      # filter by keyword
gh extension search --owner github # filter by owner
gh extension search --limit 100  # fetch more results
```

The first column shows a checkmark (✓) next to extensions that are already installed. Pipe to `--json` / `--jq` for scripting:

```
gh extension search --json fullName,description,stargazersCount \
  --jq '.[0] | [.stargazersCount, .fullName, .description] | @tsv'
```

Open the results in the browser instead:

```
gh extension search -w
```

Installing with a pinned version

Pin a binary extension to a specific release tag:

```
gh extension install dlvdhr/gh-dash --pin v5.9.0
```

Pin a script extension to a specific commit:

```
gh extension install owner/gh-script --pin alb2c3d
```

Installing a local development extension

During development, install from the current directory. `gh` creates a symlink rather than a copy, so edits take effect immediately:

```
cd ~/code/gh-myscript
gh extension install .
```

Running an extension that conflicts with a built-in

If your installed extension shares a name with a built-in `gh` command, use `exec` to call the extension explicitly:

```
gh extension exec label      # invokes the gh-label extension
gh label                     # still invokes the built-in
```

Creating a new extension

Scaffold a shell-script extension:

```
gh extension create gh-hello
```

Scaffold a Go precompiled extension:

```
gh extension create --precompiled=go gh-hello
```

Scaffold a non-Go precompiled extension (provides the build skeleton without Go-specific tooling):

```
gh extension create --precompiled=other gh-hello
```

Run `gh extension create` with no arguments to use the interactive wizard instead.

Key options

install

Flag	What it does	When to use it
<code>--pin <ref></code>	Pin the extension to a tag or commit	Reproducible environments; avoid surprise upgrades
<code>--force</code>	Force the install even if the extension is already installed	Re-install after a broken upgrade

search

Flag	What it does	When to use it
<code>-L / --limit <n></code>	Maximum results to return (default 30)	Browsing beyond the top 30

<code>--owner <org></code>	Filter by repository owner	Finding all extensions from a trusted org
<code>--sort <field></code>	Sort by <code>forks</code> , <code>help-wanted-issues</code> , <code>stars</code> , or <code>updated</code>	Surface recently-maintained extensions
<code>--order <asc desc></code>	Sort direction (default <code>desc</code>)	Find oldest or least-starred first
<code>--license <type></code>	Filter by SPDX license identifier	Compliance requirements
<code>--json <fields></code>	Output JSON with the specified fields	Scripting
<code>-q / --jq <expr></code>	Filter JSON output with a jq expression	Inline scripting
<code>-t / --template <tmpl></code>	Format JSON output with a Go template	Custom formatting
<code>-w / --web</code>	Open results in the browser	Visual browsing

upgrade

Flag	What it does	When to use it
<code>--all</code>	Upgrade every installed extension	Maintenance pass
<code>--dry-run</code>	Print what would be upgraded without doing it	Preview before committing
<code>--force</code>	Downgrade or reinstall even if already current	Recovering from a broken state

create

Flag	What it does	When to use it
<code>--precompiled=go</code>	Scaffold a Go binary extension	Extensions that need performance or cross-platform distribution
<code>--precompiled=other</code>	Scaffold a non-Go binary extension	Rust, Zig, or other compiled languages

browse

Flag	What it does	When to use it
<code>-s / --single-column</code>	Render the TUI with a single column	Screen readers, narrow terminals
<code>--debug</code>	Write a debug log to <code>/tmp/extBrowse-*</code>	Diagnosing TUI rendering issues

Best practices

Pin extensions in shared or CI environments. Without `--pin` , every fresh install fetches the latest version, which can introduce unexpected behaviour when the upstream author

pushes a breaking change. Record the pin in a setup script or a Makefile target so the whole team runs the same version.

Prefer precompiled extensions for speed. Script extensions execute through the shell and carry runtime dependencies. Precompiled extensions are single static binaries — faster cold starts and no interpreter requirement.

Review extension source before installing. Extensions are not vetted, signed, or endorsed by GitHub. Read the repository (at minimum, the main executable and any `install.sh`) before running code from an unfamiliar publisher. Check the star count and recent commit activity as a rough signal of community trust.

Suppress the daily update check in automation. The once-per-24-hour version check writes to the filesystem and prints output that can pollute script logs. Suppress it:

```
export GH_NO_EXTENSION_UPDATE_NOTIFIER=1
```

Run `gh extension upgrade --all` regularly. Extension authors fix bugs, add features, and patch security issues. A quick weekly upgrade sweep keeps the risk surface small without requiring per-extension tracking.

Develop with a local install. Use `gh extension install .` during development. `gh` follows the symlink on every invocation, so you iterate without reinstalling.

Pitfalls & gotchas

Extension names must not conflict with built-in commands — or you must use `exec`. If you install `gh-run` and the built-in `gh run` exists, typing `gh run` always invokes the built-in. Use `gh extension exec run` to reach your extension. This surprises people who expect the extension to shadow the core command.

`gh extension remove` uses the short name, not the OWNER/REPO form. Remove `dlvhdr/gh-dash` like this:

```
gh extension remove gh-dash          # correct
gh extension remove dlvhdr/gh-dash    # WRONG — not found
```

Pinned script extensions track commits, not tags. For binary extensions, `--pin` refers to a release tag (e.g. `v1.2.3`). For script extensions, it refers to a commit SHA. If you pin a script extension to a branch name it may still drift because `gh` stores the branch tip at install time, not the branch reference.

Local installs become broken symlinks if you move the source repo. Because `gh extension install .` creates a symlink to the source directory, relocating or deleting that directory breaks the extension. Run `gh extension remove` then `gh extension install .` from the new location.

`--force on upgrade` can downgrade. If there is no pin and the latest release asset is older than what is installed (a yanked-release scenario), the force flag overwrites the newer local copy.

The TUI in `browse` may render badly in small terminals. The recommended minimum is 100 columns. On a terminal below that threshold the columns overlap. Pass `--single-column` as a workaround.

Worked examples

Installing and using gh-dash

`gh-dash` is a popular TUI dashboard for pull requests and issues.

```
gh extension install dlvhdr/gh-dash
```

```
✓ Installed extension dlvhdr/gh-dash
```

Run it:

```
gh dash
```

Upgrade it later:

```
gh extension upgrade gh-dash
```

```
[gh-dash]: upgraded from v5.9.0 to v5.10.0
```

Remove it:

```
gh extension remove gh-dash
```

Scripting a pinned extension setup

A team setup script that installs two extensions at known versions and suppresses the update notifier:

```
#!/usr/bin/env bash
set -euo pipefail

export GH_NO_EXTENSION_UPDATE_NOTIFIER=1

gh extension install dlvhdr/gh-dash --pin v5.9.0
gh extension install nicholasgastor/gsfmt --pin a3f1c9e

gh extension list
```

Running this script on any machine produces the same installed set, regardless of what the upstream repos have published since.

Creating and publishing a shell-script extension

Scaffold the extension:

```
gh extension create gh-greet
cd gh-greet
```

`gh extension create` generates a starter script named `gh-greet`. Edit it to call the GitHub API:

```
#!/usr/bin/env bash
echo "Hello, $(gh api user --jq .login)!"
```

Make it executable and test locally:

```
chmod +x gh-greet
gh extension install .
gh greet
```

```
Hello, ada-lovelace!
```

Publish by pushing the repository to GitHub — `gh repo create` from the *repo* command group handles this:

```
gh repo create gh-greet --public --source=. --push
```

Anyone can now install it with:

```
gh extension install ada-lovelace/gh-greet
```

Discovering high-quality extensions

Find the top extensions in a category, sorted by stars:

```
gh extension search dashboard --sort stars --limit 10
```

REPO	DESCRIPTION
✓ dlvdhr/gh-dash	A beautiful CLI dashboard for GitHub
dlvdhr/gh-prs	Review GitHub pull requests in your terminal
...	

Extensions already installed are marked with ✓. Filter to see only extensions published by a trusted organisation:

```
gh extension search --owner github
```

Recovery

An upgrade broke an extension. Roll back by reinstalling the last known-good version with `--pin`:

```
gh extension remove gh-dash
gh extension install dlvdhr/gh-dash --pin v5.9.0
```

An extension executable is missing or corrupt. Force a clean reinstall:

```
gh extension upgrade gh-dash --force
```

A local-install extension stopped working after moving the repo. The symlink is stale. Remove and reinstall from the new location:

```
gh extension remove my-ext  
cd /new/path/to/gh-my-ext  
gh extension install .
```

gh extension list shows nothing after migrating machines. Extensions are machine-local and are not synced through your GitHub account. Re-run your setup script, or reinstall manually.

For general `gh` authentication problems that prevent installing or publishing extensions, see *auth*.

See also

- *auth* — authenticate `gh` before installing or publishing extensions.
- *search* — `gh search repos --topic gh-extension` for a finer-grained extension search with full qualifier support.
- *repo* — create and manage the repository that backs an extension.
- *release* — publish precompiled extension binaries as release assets.

status

Show a cross-repository summary of your open work on GitHub: assigned issues, assigned pull requests, review requests, mentions, and recent repository activity — all in one view.

Mental model

Every GitHub user accumulates work scattered across many repositories: issues assigned to you in one org, PRs waiting for your review in another, comments where someone mentioned your handle a third place. Normally you would have to visit each repository or rely on GitHub’s notification inbox. `gh status` pulls all of that together and prints it to the terminal in a single dashboard.

The data comes from GitHub’s REST API notifications and activity endpoints. It reflects the repositories you watch or have been active in, filtered to items that are directly relevant to you. It is a read-only snapshot — nothing is modified when you run it.

Think of `gh status` as the terminal equivalent of the GitHub homepage’s “Your activity” feed, but scoped to actionable items rather than general social activity.

Synopsis

```
gh status [-e <owner/repo>,...] [-o <org>]
```

Everyday usage

Print your full work dashboard:

```
gh status
```

The output is divided into five panels:

Assigned Issues Nothing here ^_ ^	Assigned Pull Requests #142 Fix login timeout owner/repo
Review Requests #88 Add dark mode owner/repo	Mentions Nothing here ^_ ^
Repository Activity #201 opened by colleague owner/repo	

Limit the dashboard to a single organisation:

```
gh status -o my-company
```

Exclude noisy repositories you do not want cluttering the output:

```
gh status -e cli/cli -e cli/go-gh
```

Combine both flags — show only one org while excluding a specific repo inside it:

```
gh status -o my-company -e my-company/legacy-monolith
```

Key options

Option	What it does	When to use it
<code>-e / --exclude</code>	Comma-separated list of <code>owner/name</code> repos to hide	Silence high-traffic repos that drown out real work
<code>-o / --org</code>	Restrict results to one GitHub organisation	Focus on a single employer or project group

Best practices

Run `gh status` before starting your day. It surfaces review requests and assigned issues that might otherwise sit unnoticed across many repositories. Making it a morning habit is faster than checking each repository's notification bell individually.

Exclude high-traffic repositories with `-e`. Monorepos and active open-source repositories generate constant noise. Add them to your exclude list so that items you actually own do not scroll off screen:

```
gh status -e my-company/platform -e my-company/frontend
```

Scope to an org on large GitHub.com accounts. If you contribute to both employer and personal open-source repositories, use `-o <org>` when triaging work tasks to avoid mixing contexts.

Pair `gh status` with `gh pr status` for pull-request detail. `gh status` gives you the high-level overview; see *pr* for `gh pr status`, which adds CI check results and merge-conflict indicators for pull requests in the current repository.

Pitfalls & gotchas

Only repositories you watch or have been active in appear. If you are added to an issue in a repository you have never interacted with and do not watch, it may not show up. Go to the repository on GitHub and subscribe to the relevant notifications to make it appear.

`--exclude` requires the full `owner/name` format. Passing just the repository name without the owner produces no error but also silences nothing. Always use the slash-separated form:

```
# WRONG – silently ignored
gh status -e my-repo

# Correct
gh status -e my-company/my-repo
```

Multiple repos with `-e` can be comma-separated or repeated. Both forms work; use whichever is easier in scripts:

```
gh status -e cli/cli,cli/go-gh      # comma-separated
gh status -e cli/cli -e cli/go-gh   # repeated flag
```

The output is a terminal dashboard, not machine-parseable. `gh status` has no `--json` flag. If you need structured data about pull requests or issues, use `gh pr list --json` or `gh issue list --json` instead (see *pr* and *issue*).

Activity reflects your notification subscriptions, not all public activity. Removing yourself from a repository's watchers will drop its items from the Repository Activity panel even if you have open PRs there.

Worked examples

Morning triage across all repositories

Run the dashboard with no flags to see everything:

```
gh status
```

Assigned Issues		Assigned Pull Requests	
#34 Update README	myorg/docs	#201 Add rate limiting	myorg/api
#67 Fix nav bar	myorg/frontend		
Review Requests		Mentions	
#88 Improve caching	myorg/api	#12 @you LGTM?	myorg/infra
#91 Bump deps	myorg/frontend		
Repository Activity			
#205 opened by alice	myorg/api		
#31 closed by bob	myorg/docs		

From here you can jump directly to any item with `gh pr view 88 --repo myorg/api` or `gh issue view 34 --repo myorg/docs`.

Focused org triage with noisy repos excluded

You work mainly in `myorg` but the `myorg/data-pipeline` repository has continuous-integration bots opening hundreds of issues:

```
gh status -o myorg -e myorg/data-pipeline
```

Assigned Issues		Assigned Pull Requests	
#34 Update README	myorg/docs	#201 Add rate limiting	myorg/api
Review Requests		Mentions	
#88 Improve caching	myorg/api	Nothing here ^_^	
Repository Activity			
#205 opened by alice	myorg/api		

The data-pipeline items are hidden; everything else in `myorg` still appears.

Checking status in a CI or scripted context

`gh status` does not offer JSON output. For automated pipelines, compose the purpose-built commands instead:

```
# All open PRs assigned to the authenticated user in one repo
gh pr list --assignee @me --state open --json number,title,url

# All open issues assigned to you across an org (via gh api)
```

```
gh api graphql -f query='
  query {
    search(query: "assignee:@me is:issue is:open org:myorg", type: ISSUE, first:
20) {
      nodes { ... on Issue { number title url } }
    }
  }
'
```

See *api* for using `gh api` to build custom queries when the dashboard's fixed format is not enough.

Recovery

`gh status` is purely read-only — it changes nothing. If the command errors:

- **Authentication error:** your token may have expired. Run `gh auth status` and re-authenticate with `gh auth login` if needed (see *auth*).
- **No results when you expect some:** check that the affected repository appears in your GitHub notification subscriptions. Visit the repository on GitHub and set your watch level to “All Activity” or “Participating”.
- **Output truncated:** there is no pagination flag. Switch to `gh issue list` or `gh pr list` with `--limit` to retrieve full result sets.

See also

- *auth* — authenticate `gh` and manage stored credentials used by every command including `gh status`.
- *pr* — `gh pr status` shows per-repository pull request detail with CI results and conflict indicators.
- *issue* — `gh issue list --assignee @me` lists assigned issues with full filtering and JSON output.
- *api* — build custom GraphQL queries when the fixed dashboard format is too limited.

completion

Generate shell completion scripts so your shell can tab-complete `gh` commands, subcommands, flags, and arguments.

Mental model

When you press Tab after typing `gh pr`, your shell does not ask GitHub for a list of subcommands. Instead, it consults a completion script — a chunk of shell code that was loaded at startup time, describing every command, flag, and accepted value. `gh completion` outputs that script for your shell.

The script is static: it encodes the structure of `gh` at the time the CLI was built. It does not query the network. Once loaded, it makes the entire `gh` command tree navigable with Tab without any runtime cost.

Most package managers install the completion script automatically. If you installed `gh` with Homebrew on macOS, completions are already wired up through Homebrew's own completion infrastructure. `gh completion` matters most when you installed the binary manually, upgraded to a new version whose new subcommands are not yet reflected in the system-installed script, or need to set up completions in a non-standard shell profile.

Synopsis

```
gh completion -s <shell>
```

The `-s` flag is required. There are no subcommands.

Everyday usage

Print a completion script for the current session (useful for testing):

```
gh completion -s zsh
```

Install completions for zsh permanently:

```
gh completion -s zsh > /usr/local/share/zsh/site-functions/_gh
```

Install completions for bash (requires `bash-completion` to be installed):

```
# Add to ~/.bash_profile
eval "$(gh completion -s bash)"
```

Install completions for fish:

```
gh completion -s fish > ~/.config/fish/completions/gh.fish
```

Install completions for PowerShell. Open your profile with:

```
notepad $profile
```

Then add and save:

```
Invoke-Expression -Command $(gh completion -s powershell | Out-String)
```

After writing or sourcing any of these files, restart your shell before testing Tab completion.

Key options

Flag	What it does	When to use it
<code>-s / --shell</code>	Shell type: <code>bash</code> , <code>zsh</code> , <code>fish</code> , or <code>powershell</code>	Required on every invocation

There are no other flags beyond `--help`.

Best practices

Write the script to a file rather than eval-ing it on every startup. Evaluating a script at shell startup — `eval "$(gh completion -s zsh)"` — works but re-runs the script on every new shell. For `zsh` and `fish`, writing once to a file in `$fpath` (`zsh`) or `~/.config/fish/completions/` (`fish`) loads faster and survives `gh` upgrades as-is until you explicitly refresh it.

Re-run after upgrading `gh`. The script encodes the subcommands and flags of the exact version that generated it. After a major upgrade that adds new commands, overwrite the completion file so Tab discovers the new additions:

```
gh completion -s zsh > /usr/local/share/zsh/site-functions/_gh
exec zsh # reload the shell
```

Let Homebrew manage completions when possible. If you installed `gh` via Homebrew, follow the Homebrew shell-completion setup instead of running `gh completion` directly. Homebrew refreshes the script automatically on upgrades. See <https://docs.brew.sh/Shell-Completion> for setup instructions.

Verify `$fpath` contains your target directory before writing. For `zsh`, the file must land in a directory that appears in `$fpath`. Print the list to confirm:

```
print -l $fpath
```

If `/usr/local/share/zsh/site-functions` is absent, pick a directory that is present, or add a custom one and ensure `compinit` runs after it:

```
fpath=(~/.zsh/completions $fpath)
autoload -U compinit && compinit
```

Pitfalls & gotchas

Completions not working after installation. The most common cause is forgetting to restart the shell (or run `exec zsh` / `exec bash`). The completion script is sourced at shell startup; an already-running shell does not pick up the new file automatically. If a restart does not help, check that `compinit` is called in `~/.zshrc` *after* the `$fpath` addition:

```
autoload -U compinit
compinit -i
```

zsh compinit: insecure directories warning. The `-i` flag to `compinit` suppresses this warning by ignoring insecure directories. Without it, zsh may refuse to load completions from world-writable directories. Alternatively, fix the directory permissions with `chmod go-w .`

The generated script does not include gh extensions. Completion scripts cover only built-in commands. Tab completion for commands added through `gh extension install` (see *extension*) is not generated by `gh completion` — each extension would need to provide its own completion logic.

Homebrew completions and a manual file conflict. If Homebrew is managing a completion file and you also write one manually to the same path, they will overwrite each other on the next `brew upgrade`. Choose one approach and stick to it. A safe manual location outside Homebrew's reach is `~/.zsh/completions/`.

PowerShell profile directory may not exist. The `notepad $profile` step will fail if the parent directory has not been created yet. Create it first:

```
mkdir -Path (Split-Path -Parent $profile) -ErrorAction SilentlyContinue
```

Worked examples

Setting up zsh completions from scratch

Check whether a suitable directory exists in `$fpath`:

```
print -l $fpath | grep site-functions
```

```
/usr/local/share/zsh/site-functions
```

Write the script there:

```
gh completion -s zsh > /usr/local/share/zsh/site-functions/_gh
```

Ensure `~/.zshrc` contains the required lines (add them if missing):

```
autoload -U compinit
compinit -i
```

Reload the shell and test:

```
exec zsh
gh pr <Tab>
```

```
checkout -- Check out a pull request in git
close    -- Close a pull request
```

```
comment    -- Add a comment to a pull request
...
```

Refreshing completions after a gh upgrade

After upgrading `gh` the new `gh attestation` subcommands do not Tab-complete because the on-disk script was generated by the old binary:

```
# Overwrite the stale file with the freshly-built script
gh completion -s zsh > /usr/local/share/zsh/site-functions/_gh

# Reload
exec zsh

# Verify the new subcommand appears
gh attestation <Tab>
```

```
download  -- Download an artifact attestation
inspect   -- Inspect a sigstore bundle used for attestation verification
...
```

Adding fish completions

```
# Create the directory if it does not exist
mkdir -p ~/.config/fish/completions

# Write the script
gh completion -s fish > ~/.config/fish/completions/gh.fish
```

Open a new fish session (or run `exec fish`) and test:

```
gh repo <Tab>
```

```
archive  clone  create  delete  deploy-key  edit  fork ...
```

Recovery

If Tab completion stops working after a `gh` upgrade, regenerate the script and reload the shell:

```
gh completion -s zsh > /usr/local/share/zsh/site-functions/_gh
exec zsh
```

If the file was written to the wrong location, identify where the shell looks for completion scripts:

```
# zsh: list every directory searched for completion files
print -l $fpath

# bash: check the compat directory
echo $BASH_COMPLETION_COMPAT_DIR
```

Write the script to a directory that appears in that list, then reload.

For Homebrew-managed installations, let Homebrew regenerate the script by running `brew reinstall gh` rather than writing the file manually.

See also

- *auth* — authenticate `gh` before any command can be tab-completed and executed.
- *alias* — custom aliases defined with `gh alias set` are not covered by the generated completion script.
- *extension* — third-party extensions installed with `gh extension install` require their own completion setup outside of `gh completion`.
- *config* — `gh config set` controls shell and editor preferences that complement the completion setup.

Getting out of jams: gh

Authentication, API, and workflow problems have clear causes and repeatable fixes. Work through the symptom that matches your error.

HTTP 403 / “Permission denied” on a repo command

What it means. GitHub accepted your token but refused the request. The authenticated user does not have the access the operation requires.

Why it happens. The most common cause on a machine with multiple accounts is the wrong account being active. `gh` marks one account per host as active and uses it for every command. If you last worked on a personal repo and forgot to switch back before opening a work repo, every command runs as the wrong user. A fine-grained personal access token scoped to specific repositories produces the same 403 when you target a repo outside its scope.

How to get out.

1. Check which account is currently active:

```
gh auth status --active
```

2. If the wrong account is active, switch to the correct one:

```
gh auth switch --user correct-username
```

On a host with exactly two accounts, omit `--user` to toggle between them.

1. Retry the original command.
2. If the token is a fine-grained PAT and the repo is simply outside its scope, generate a new token that covers the repo, or use a classic PAT, then re-authenticate:

```
echo "ghp_newtoken..." | gh auth login --with-token
```

How to avoid it. After finishing work in one account’s repos, run `gh auth switch` to return to your default account. Confirm the active account with `gh auth status --active` before any operation on a sensitive repo. Use `GH_TOKEN` in the environment when you need a one-off override without touching the stored credentials:

```
GH_TOKEN="ghp_work_token..." gh pr list --repo my-company/backend
```

Not logged in / “authentication required” / expired token

What it means. `gh` has no stored credential for the host, or the credential it holds has been revoked or expired.

Why it happens. Tokens expire (especially fine-grained PATs), get revoked in the GitHub settings UI, or were never created because this is a new machine. In a CI environment the secret may have been rotated and not updated in the workflow.

How to get out.

1. Confirm the state of stored credentials:

```
gh auth status
```

Any account with an authentication failure is flagged here.

1. Log in again (or log in for the first time):

```
gh auth login
```

In a headless environment, pass a fresh token on stdin:

```
echo "ghp_..." | gh auth login --with-token
```

1. If the old broken entry is still listed, log it out first:

```
gh auth logout --hostname github.com --user stale-username  
gh auth login
```

How to avoid it. Prefer classic OAuth tokens (browser flow) for interactive use — they do not expire on a fixed schedule. In CI, store the token in a secret and use the `GH_TOKEN` environment variable; rotate the secret whenever you rotate the underlying PAT.

Missing token scopes (“requires the X scope”)

What it means. The OAuth token `gh` holds was granted permissions at login time, and the operation you are attempting needs a scope that was not included.

Why it happens. `gh auth login` requests only the minimum required scopes (`repo` , `read:org` , `gist`). Commands that go beyond core PR/issue/repo work — such as managing org SSO, publishing packages, or reading audit logs — need additional scopes.

How to get out.

Add the missing scope without logging out:

```
gh auth refresh --scopes <scope-name>
```

For example, adding package write access:

```
gh auth refresh --scopes write:packages
```

This opens a browser to re-authorise the existing token with the new scope. When a command error message names the exact scope needed, paste it directly into `--scopes` .

To add multiple scopes at once, separate them with commas:

```
gh auth refresh --scopes write:org,admin:public_key
```

How to avoid it. Grant scopes only as needed — a token with fewer scopes does less damage if it leaks. Keep note of which projects need which extra scopes; you can inspect the current set with `gh auth status --show-token` and decode the token on GitHub’s token settings page.

SSO authorization required for an org's resources

What it means. The organization has SAML Single Sign-On enabled. Your token is valid for github.com in general but has not been authorized for this specific org, so the API returns a 403 with a `X-GitHub-SSO` response header.

Why it happens. After authenticating with `gh auth login` or creating a PAT, each organization that enforces SAML SSO requires a separate, explicit authorization step before the token is allowed to access that org's resources.

How to get out.

1. Run the refresh command so that the browser flow can handle SSO:

```
gh auth refresh
```

2. In the browser, complete the SAML login for the organization when prompted.
3. If the token is a PAT rather than an OAuth app token, go to **GitHub → Settings → Personal access tokens**, find the token, and click **Authorize** next to the organization.

How to avoid it. After first authenticating on a machine used for work at an SSO-protected org, immediately run `gh auth refresh` and complete the SSO authorization. Automate CI tokens by creating a service-account PAT that is pre-authorized for the org and stored as a secret.

HTTPS git push fails because gh credential helper uses the wrong account

What it means. A plain `git push` over HTTPS fails with a 403 or “remote: Permission to ... denied” message even though `gh auth status` shows you are logged in.

Why it happens. `gh auth setup-git` registers `gh` as git's HTTPS credential helper. When git asks for credentials, `gh` supplies the token for whichever account is currently active. If you have two accounts and the active one does not have write access to the repo you are pushing to, the push is rejected. The active account is a global setting, so it is easy to push to the wrong identity after working in another account's repos.

How to get out.

1. Check the active account:

```
gh auth status --active
```

2. Switch to the account that owns (or has write access to) the repo:

```
gh auth switch --user correct-username
```

3. Push again:

```
git push
```

4. If the remote URL uses SSH (starts with `git@github.com:` or uses an SSH config alias), the credential helper is not involved and the issue is with your SSH key, not `gh`. Check `ssh -T git@github.com` to confirm SSH identity.

How to avoid it. For repos you regularly push to from two different accounts, consider using SSH remotes with separate host aliases in `~/.ssh/config`. That way each remote carries its own identity and the active `gh` account is irrelevant for git pushes.

Rate limiting (HTTP 429 / X-RateLimit headers)

What it means. GitHub's API has returned a 429 Too Many Requests response, or a 200/403 with `X-RateLimit-Remaining: 0`, indicating you have exhausted your rate-limit quota for the current window.

Why it happens. Unauthenticated requests are limited to 60 per hour. An authenticated token is limited to 5,000 requests per hour for the REST API. Scripts that loop over repos, issues, or commits in tight loops can exhaust the limit quickly. GraphQL has a separate point-based budget.

How to get out.

1. Inspect the current limit and when it resets:

```
gh api rate_limit
```

The response body shows `resources.core.remaining` and `resources.core.reset` (a Unix timestamp).

1. Use `gh api --include` to see the response headers on a live request:

```
gh api --include repos/{owner}/{repo} 2>&1 | grep -i ratelimit
```

2. Wait until the reset time, then retry. The window resets on a rolling one-hour basis.
3. If the rate limit is from an unauthenticated request path, confirm `gh` is authenticated and the correct token is in use:

```
gh auth status --active
```

How to avoid it. Add `--cache` to `gh api` calls that fetch stable data:

```
gh api --cache 1h repos/{owner}/{repo}
```

Use `--paginate` instead of writing your own loop so that `gh` handles cursor management efficiently. In scripts, check `X-RateLimit-Remaining` and back off before the limit hits zero.

`gh pr create` — “no commits between base and head” / wrong base branch

What it means. `gh pr create` refuses to open a PR because the head branch has no commits that are not already on the base branch.

Why it happens. Two common causes: the branch was branched from — or has already been merged into — the target base, so there is nothing new to include; or `gh pr create` inferred the wrong base branch (e.g., `main` when you intended `develop`).

How to get out.

1. Confirm what base branch was inferred:

```
gh pr create --dry-run --fill
```

The dry-run output shows the base and head without creating the PR.

1. Specify the intended base explicitly:

```
gh pr create --base develop --fill
```

2. If the branch genuinely has no new commits, check your git log:

```
git log origin/main..HEAD --oneline
```

If this is empty, the branch needs commits before a PR makes sense.

How to avoid it. Configure a per-branch merge base so that `gh pr create` always picks it up:

```
git config branch.my-feature.gh-merge-base develop
```

From then on, `gh pr create` on that branch defaults to `develop` as the base without needing the flag.

Merge blocked by required checks / branch protection

What it means. GitHub refused a `gh pr merge` because required status checks have not passed, or a branch protection rule demands reviews that have not been completed.

Why it happens. The repository's branch protection rules require one or more of: all CI checks to pass, a minimum number of approving reviews, no unresolved conversations, or a signed-off commit policy. Attempting to merge before these conditions are satisfied fails even via the CLI.

How to get out.

1. See which checks are pending or failing:

```
gh pr checks
```

2. If checks are still running, wait for them or watch live:

```
gh pr checks --watch
```

3. If a required check failed, fix the code, push, and wait for re-runs.
4. If you are waiting for a review, request one explicitly:

```
gh pr edit --add-reviewer teammate-username
```

5. Enable auto-merge so that `gh` merges automatically once all requirements are met:

```
gh pr merge --auto --squash
```

6. As a last resort, administrators can bypass protections with `--admin`:

```
gh pr merge --admin --squash # requires admin rights on the repo
```

How to avoid it. Use `gh pr merge --auto` as a habit. It queues the merge immediately but waits for checks and reviews, so you do not have to poll manually.

gh api — distinguishing 404 vs 403 vs 422

What it means. `gh api` exits non-zero when the server returns an error status. Each code means something different and demands a different fix.

Why it happens.

Status	Meaning	Typical cause
404 Not Found	The resource does not exist, or the authenticated user cannot see it	Wrong endpoint path, resource deleted, or insufficient read access
403 Forbidden	The resource exists but the operation is not allowed	Token lacks a required scope, org SSO not authorized, or write-protected resource
422 Unprocessable Entity	The request was well-formed but semantically invalid	Required fields missing, conflicting state (e.g., trying to merge an already-merged PR), or validation failure

How to get out.

Use `--include` to see the full HTTP response including status code and headers:

```
gh api --include repos/{owner}/{repo}/pulls/999
```

```
HTTP/2.0 404 Not Found
...
{"message":"Not Found","documentation_url":"..."}
```

- **404:** Verify the endpoint path and that the resource exists. A private repo you cannot see returns 404, not 403, to avoid leaking its existence. Check that the active account has at least read access.
- **403:** Run `gh auth status --show-token` and check the token's scopes. See the *HTTP 403 / "Permission denied"* and *Missing token scopes* entries above.
- **422:** Read the response body carefully — it contains a `message` and often an `errors` array describing which field or constraint failed. Fix the request parameters accordingly.

How to avoid it. In scripts, capture the HTTP status with `--include` and branch on it rather than letting a non-zero exit silently abort your script. Use `--silent` combined with exit-code checking when you only care about success vs failure:

```
if gh api repos/{owner}/{repo} --silent 2>/dev/null; then
  echo "repo accessible"
else
  echo "repo not accessible (check auth and path)"
fi
```

Targeting a GitHub Enterprise Server host

What it means. Commands that work fine against `github.com` fail or hit the wrong host when the target repository is on a GitHub Enterprise Server (GHES) instance.

Why it happens. `gh` defaults every request to `github.com`. If your GHES host is not specified, API calls go to the wrong server and return 404 or refuse to authenticate.

How to get out.

1. Log in to the Enterprise host if you have not already:

```
gh auth login --hostname github.mycompany.com
```

2. For `gh api` calls, pass `--hostname` to target the GHES instance:

```
gh api --hostname github.mycompany.com repos/{owner}/{repo}
```

For other commands that do not accept `--hostname`, set `GH_HOST` for the invocation, or use the `[HOST/]OWNER/REPO` syntax with the `-R` flag:

```
GH_HOST=github.mycompany.com gh repo list my-org
gh pr list -R github.mycompany.com/my-org/backend
```

1. To check which hosts are authenticated:

```
gh auth status
```

How to avoid it. When a project always targets a GHES host, add `GH_HOST=github.mycompany.com` to a project-level `.env` or your shell profile for that work context. This avoids having to pass `--hostname` on every invocation. See *auth* for details on managing credentials across multiple hosts.